Part 1: Data from Raw to Ready

Data rarely arrive to us in a form such that they are ready for formal analysis. Typically, steps of preprocessing, cleaning, checking, etc. are required before we have what we can call **ready data**.

You will note that I avoid using the term **clean data**, as this can imply a complete lack of issues in the data set. Completely "clean" data is not realistic, and should not be expected. The best we can hope is that a data set is sufficiently processed to minimize any biases that may result from errors in the data set.

Ultimately, this process is quite situation-specific. We will present some key concepts and stress the importance of taking this step, but each data set will present unique challenges. This topic will be revisited throughout the course as we encounter new data sets.

Our discussion will initially focus on data sets that are naturally represented in a **flat** or **spreadsheet** form. Each column contains a different **variable** and each row gives one **instance** or **case**.

Exercise: Give examples of cases and associated variables in data sets typically encountered in finance.

When is a data set "ready?"

1. Each variable is properly named.
2. Each variable is understood.
3. Each variable is represented in its appropriate **form**.
4. Each variable has internal consistency, i.e., values have the same interpretation across all cases.
5. The source of any extreme values (outliers) is noted, and corrected as needed.
6. The source of any missing values is noted.

Example Data Set

First, we will read in a data set for use in some subsequent examples.

Visit the website

https://bit.ly/31QyYjK

and download the data from the first quarter of 2017. Create a directory for working on these examples, and place this file in that directory.

These data come from the U.S. Securities and Exchange Commission respository on market structure. This data set contains daily characteristics of a range of market variables over 5000 equities and ETFs, accumulated over several exchanges. See also

`https://www.sec.gov/marketstructure/mar_methodology.html`

Be sure that the working directory is correctly set in Python (use `chdir` from `os` if necessary) and use `read_csv` from `pandas`:

```
In [1]: import pandas as pd
        import matplotlib.pyplot as plt
        import numpy as np
        fulldata = pd.read_csv("q1_2017_all.csv")
```

Exercise: Visually inspect the data set and comment.

Background on the data can be found in the associated `README` file, available on Canvas in the "Data Sets" folder in the "Files" page. We will reference portions of it below.

Understanding the Variables

Whenever a new data set is encountered, one should fully understand its variables. Specific questions to address for each variable:

1. Describe the information that is stored in this column.
2. What type of variable is it? (Timestamp, categorical, ordinal, ratio, other?) Be sure that Python represents the variable correctly.
3. Are there any missing values? What is the source of the missingness?
4. Are there any extreme/inappropriate values? What is the source of the extreme value?

Exercise: Describe the distinctions between the variables which are on the categorical, ordinal, and ratio scales.

Column 1: Date

The first column in `fulldata` is a date stamp for the observation. Python/Pandas has special functions for handling dates which will prove useful later, including when working with time series.

After reading the file, the dates are simply integers.

```
In [2]: print(fulldata['Date'][0])
```

```
20170103
```

The function `to_datetime` in Pandas will make the conversion. The list on the following page shows all of the formatting characters.

```
In [3]: fulldata['Date'] = \
            pd.to_datetime(fulldata['Date'], \
                                format='%Y%m%d')
```

These strings can also be used in formatting output using `strftime`.

```
In [4]: print(fulldata.Date[0].strftime('%m/%y'))
        print(fulldata.Date[0].strftime('%A, %B %d'))
```

```
01/17
Tuesday, January 03
```

(From `https://stackabuse.com/how-to-format-dates-in-python/`.)

- `%b`: Returns the first three characters of the month name.
- `%d`: Returns day of the month, from 1 to 31.
- `%Y`: Returns the year in four-digit format.
- `%H`: Returns the hour.
- `%M`: Returns the minute, from 00 to 59.
- `%S`: Returns the second, from 00 to 59.
- `%a`: Returns the first three characters of the weekday, e.g. Wed.
- `%A`: Returns the full name of the weekday, e.g. Wednesday.
- `%B`: Returns the full name of the month, e.g. September.
- `%w`: Returns the weekday as a number, from 0 to 6, with Sunday as 0.
- `%m`: Returns the month as a number, from 01 to 12.
- `%p`: Returns AM/PM for time.
- `%y`: Returns the year in two-digit format, that is, without the century.
- `%f`: Returns microsecond from 000000 to 999999.
- `%Z`: Returns the timezone.
- `%z`: Returns UTC offset.
- `%j`: Returns the number of the day in the year, from 001 to 366.
- `%W`: Returns the week number of the year, from 00 to 53, with Monday being counted as the first day of the week.
- `%U`: Returns the week number of the year, from 00 to 53, with Sunday counted as the first day of each week.
- `%c`: Returns the local date and time version.
- `%x`: Returns the local version of date.
- `%X`: Returns the local version of time.

Columns 2 and 3: Security and Ticker

The variables `Security` and `Ticker` are currently of type `object`, which is how Pandas stores strings.

```
In [5]: fulldata[['Security','Ticker']].dtypes
```

```
Out[5]: Security    object
        Ticker      object
        dtype: object
```

Both of these variables are clearly categorical, and lacking in a natural ordering to the levels. The conversion to the appropriate data type is shown below. (By default, the categories are taken to be unordered.)

```
In [6]: fulldata['Security'] = \
          fulldata['Security'].astype('category')
        fulldata['Ticker'] = \
          fulldata['Ticker'].astype('category')

        fulldata[['Security','Ticker']].dtypes
```

```
Out[6]: Security    category
        Ticker      category
        dtype: object
```

We can see that there are two possible levels for `Security`, `Stock` or `ETF` (Exchange Traded Fund). There are over 5,000 symbols under consideration.

```
In [7]: print(fulldata['Security'].unique())
        print(len(fulldata['Ticker'].unique()))
```

```
[Stock, ETF]
Categories (2, object): [Stock, ETF]
5338
```

Exercise: How many of the different ticker symbols are ETFs?

Exercise: Execute the commands below, and discuss what you find.

```
In [8]: len(fulldata['Date'].unique())
        fulldata.groupby('Ticker')['Date'].\
          count().value_counts().sort_index()
        fulldata.groupby('Ticker')['Date'].\
          count().idxmax()
        fulldata[fulldata.duplicated(\
          subset=('Date','Ticker'),keep=False)]
        None
```

We can remove the duplicated rows using the following command.

```
In [9]: fulldata = fulldata.drop_duplicates(\
          subset=('Date','Ticker'), keep='last')
```

Columns 4 through 7: Rank Variables

The next four columns provide ranks of the stocks/ETFs with respect to key attributes of Market Capitalization, Turnover, Volatility, and Price.

In general, we would prefer to start with data in its most "raw" format, and one could construct variables such as these from that data. Alas, we are often left to work with what is avaiable.

Exercise: Inspect the output of

```
In [44]: fulldata.groupby(by=['Date','Security',\
             'VolatilityRank'])['Security'].count()
         fulldata.groupby(by=['Security','Ticker',\
             'VolatilityRank'])['VolatilityRank'].count()
         None
```

What does this tell us about the structure of these variables? Why should we be very careful with these variables?

We change these into **ordered categorical variables** to reflect their ordinal nature.

```
In [11]: fulldata = fulldata.copy()
         fulldata['McapRank'] = fulldata['McapRank'].\
           astype('category').cat.as_ordered()
         fulldata['TurnRank'] = fulldata['TurnRank'].\
           astype('category').cat.as_ordered()
         fulldata['VolatilityRank'] = \
           fulldata['VolatilityRank'].\
             astype('category').cat.as_ordered()
         fulldata['PriceRank'] = fulldata['PriceRank'].\
           astype('category').cat.as_ordered()
```

Columns 8 through 19: The Count Variables

The remaining column in the data set consist of trade and share counts of different types.

Note that some of the volume counts are reported in 1000's of shares, hence there are non-integer values among the counts.

Exercise: Execute the command below and comment on the results. It is especially important to focus on extreme or impossible values.

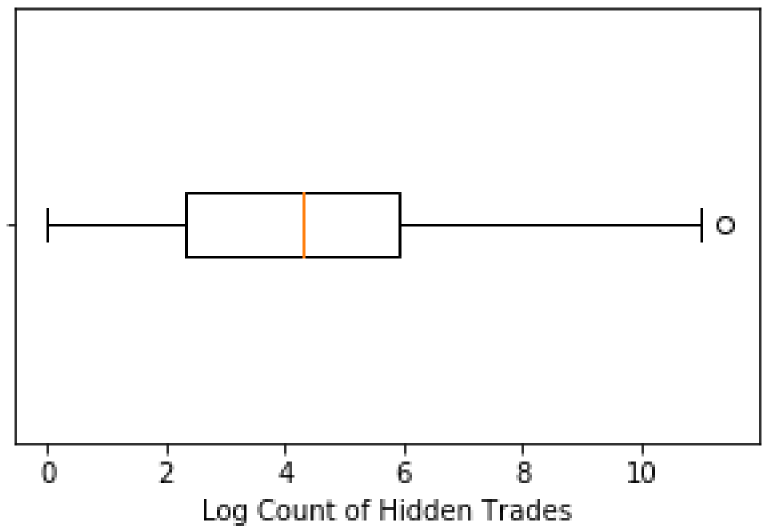
```
In [12]: fulldata.describe()
         None
```

Of course, graphical tools are useful for understanding the properties of variables. This topic will be covered more fully later, but here we consider a classic approach: the **boxplot**:

```
In [13]: plt.figure(figsize=(5,3))
         plt.boxplot(np.log(fulldata['Hidden'])\
             .dropna(),labels=[""], vert=False)
         plt.xlabel("Log Count of Hidden Trades")
         plt.show()
```

```
/Users/chadschafer/anaconda3/lib/python3.7/site-packages/ipyker

/Users/chadschafer/anaconda3/lib/python3.7/site-packages/ipyker
```



The “box” of the plot extends from the 25th to the 75th percentile of the data, while the solid line in the middle of the box is at the median. The two “arms” extend to the minimum and maximum value, **except** that any value labelled an **outlier** is plotted separately.

Exercise: How is an outlier defined in this case?

Exercise: Why was the logarithm of the variable utilized? What issues did that create?

Exercise: Consider the outlier in the previous plot. Is this observation also extreme in the other variables? Can you find the source of this behavior?

Exercise: Parse the syntax, and comment on the output, of the following command.

```
In [14]: fulldata.iloc[:, 7:19].\
            apply(lambda x: x.idxmax())
         None
```

Missing Data

Most data sets contain some amount of missing data, for many reasons.

As a first step, it is important to recognize how your data set represents missing values, so that they are read in properly.

In a `.csv` file such as that we are using here, it is common to simply have blank values to indiciate missingness, i.e., there are consecutive commas without a value between.

The `read_csv()` function has an argument `na_values` which allows the user to specify strings that should be considered as missing values. By default, the following are converted into `NaN`: `, #N/A, #N/A N/A, #NA, -1.#IND, -1.#QNAN, -NaN, -nan, 1.#IND, 1.#QNAN, N/A, NA, NULL, NaN, n/a, nan, null`. **Be careful:** It is not unusual for data sets to use numbers such as `-999` to indicate a missing value.

Pandas uses `NaN` to represent missing values.

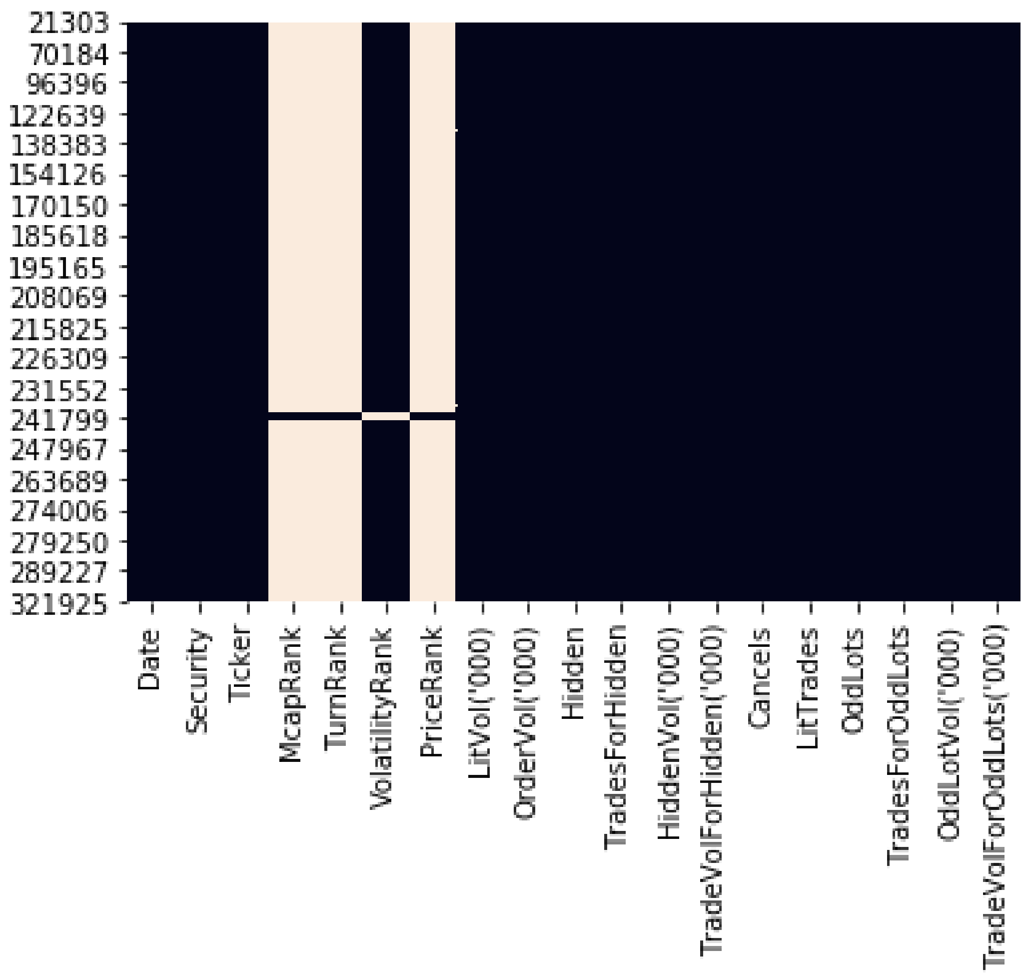
Exercise: Execute the following commands and discuss the results

```
In [15]: np.where(fulldata.isnull())
         len(np.unique(np.where(fulldata.isnull())[0]))
```

```
Out[15]: 77
```

The package `seaborn` has a function `heatmap()` that can be useful for inspecting any patterns in the missingness.

```
In [16]: import seaborn as sns
         sns.heatmap(fulldata.iloc[np.unique\
             (np.where(fulldata.isnull())[0]),:]\
             .isnull(),cbar=False)
         None
```



The `dropna` function will remove any row from the data frame with at least one `NaN` value.

```
In [17]: fulldatacomplete = fulldata.dropna()
         len(np.unique(np.where(fulldatacomplete.\
                  isnull())[0]))
```

```
Out[17]: 0
```

Working only with such data is referred to as **complete case analysis**.

Exercise: Discuss potential drawbacks to complete case analyses.

Exercise: Consider the output of the following and comment on any features of the incomplete cases that may be releveant.

```
In [18]: fulldata.iloc[np.unique(np.where(fulldata.\
             isnull())[0]),:]
         None
```

Part 2: Basic Visualization

This Part covers basic tools for visualizing data in Python.

Visualization techniques are critical to discovering and understanding the relationships between variables.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Example Data Set

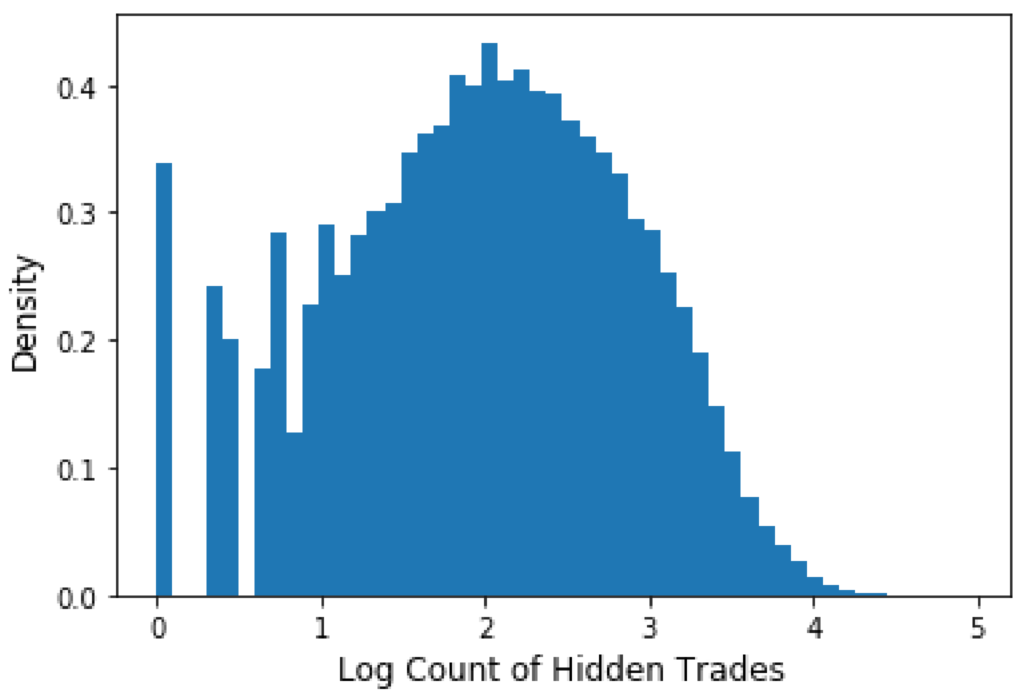
First, we will revisit the data set we considered in the previous Part, and the steps needed to properly format the data:

```
In [2]: fulldata =\
           pd.read_csv("q1_2017_all.csv", sep=",")
        fulldata['Date'] =\
           pd.to_datetime(fulldata['Date'],\
            format='%Y%m%d')
        fulldata =\
           fulldata.drop_duplicates(subset=\
            ('Date','Ticker'), keep='first')
        fulldata['McapRank'] =\
           fulldata['McapRank'].astype('category').\
            cat.as_ordered()
        fulldata['TurnRank'] =\
           fulldata['TurnRank'].astype('category').\
            cat.as_ordered()
        fulldata['VolatilityRank'] =\
           fulldata['VolatilityRank'].astype('category').\
            cat.as_ordered()
        fulldata['PriceRank'] =\
           fulldata['PriceRank'].astype('category').\
            cat.as_ordered()
```

Basic Plots

We saw the use of boxplot in a previous Part. A similar view of the shape of a distribution can be obtained using a **histogram**, formed using the `matplotlib` function `hist()`:

```
In [3]: fig = plt.figure(figsize=[6,4])
        axs = fig.subplots(1,1)
        axs.hist(np.log10(fulldata['Hidden']\
            [fulldata['Hidden'] > 0]),bins=50,density=True)
        axs.set_ylabel("Density", size=12)
        axs.set_xlabel("Log Count of Hidden Trades",size=12)
        plt.show()
```

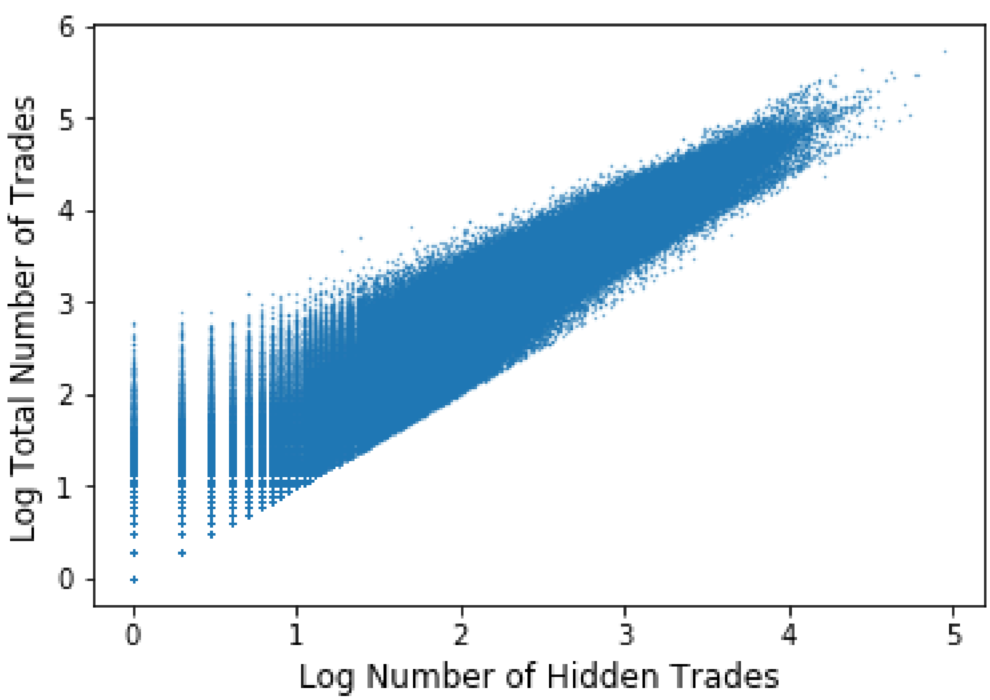


Exercise: Parse the syntax of the code above to understand some basic aspects of `matplotlib`.

Exercise: How could this plot be improved?

Scatter plots are ubiquitous, as well. They are created simply in Python using the `plot()` method.

```
In [4]: with np.errstate(divide='ignore', invalid='ignore'):
            fig, axs = plt.subplots(1, 1, figsize=[6,4])
            axs.plot(np.log10(fulldata['Hidden']),
                     np.log10(fulldata['TradesForHidden']),
                     marker='.', ms=0.5, linestyle="none")
            axs.set_xlabel("Log Number of Hidden Trades",
                   size=12)
            axs.set_ylabel("Log Total Number of Trades",
                   size=12)
            plt.show()
```



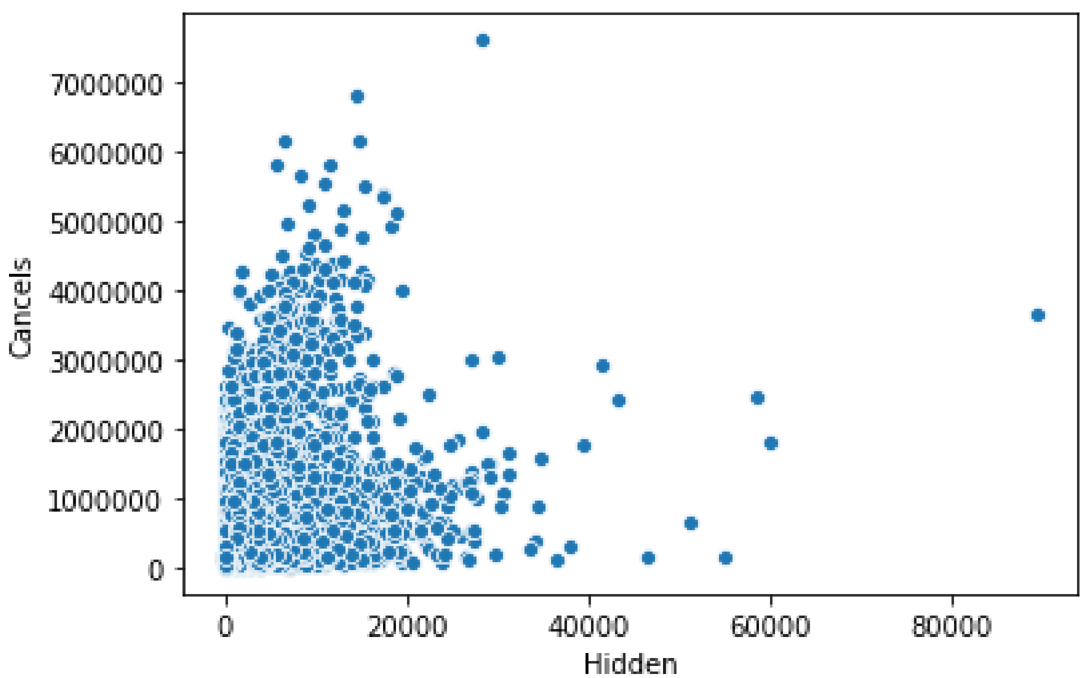
Exercise: Comment on the Python syntax in the previous example.

The Python package `Seaborn` allows for more sophisticated visualization. This package builds on `matplotlib` in useful ways. See https://seaborn.pydata.org/ for more detailed information and documentation.

```
In [5]: import seaborn as sns
```

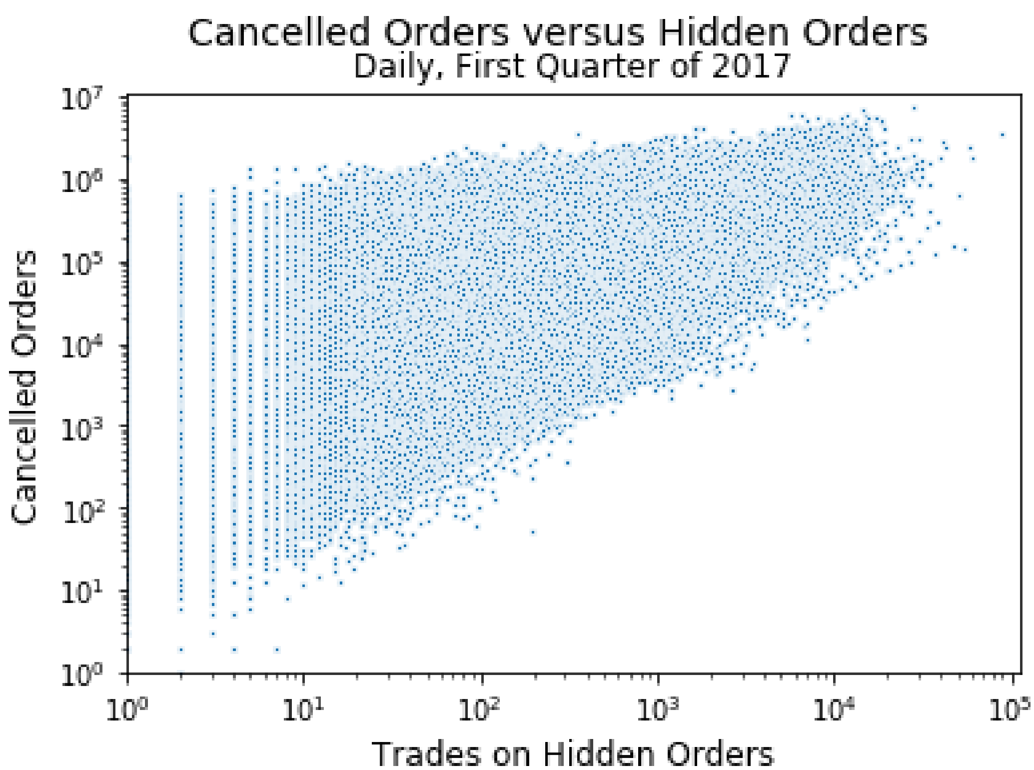
Let's start with a simple scatter plot, constucted via `scatterplot`:

```
In [6]: plt.figure(figsize=[6,4])
        ax = sns.scatterplot(x="Hidden", y="Cancels",
                    data=fulldata)
```



Let's make an improved version.

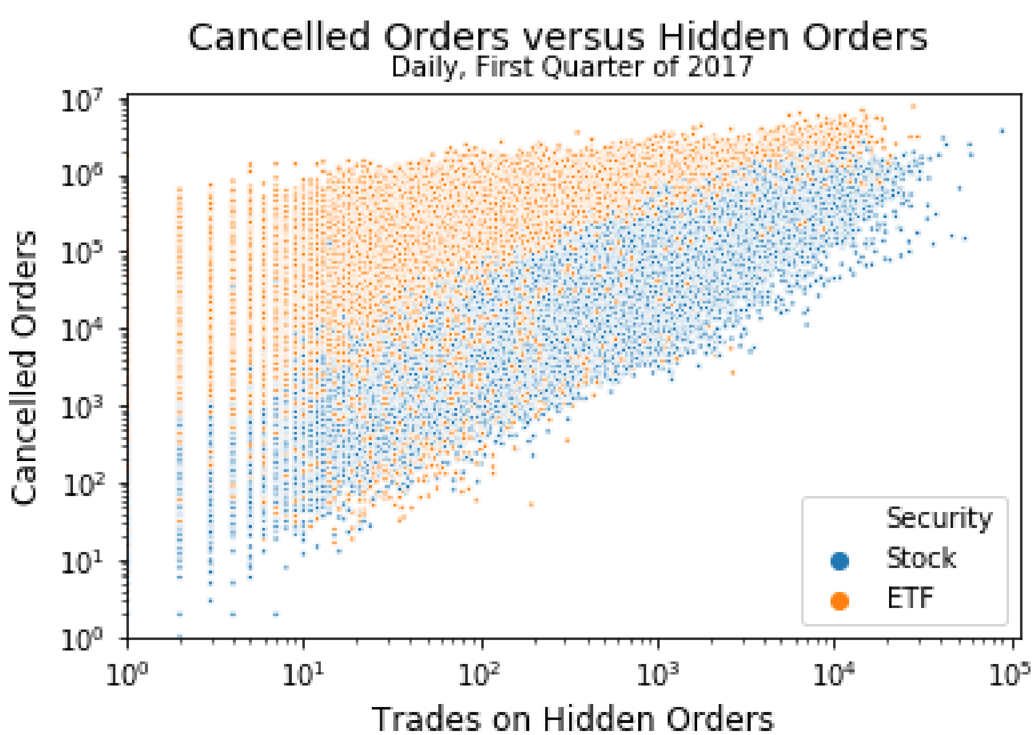
```
In [7]: plt.figure(figsize=[6,4])
        ax = sns.scatterplot(x="Hidden", y="Cancels",
             data=fulldata, s=5)
        ax.set(xscale="log", yscale="log",
               xlim=(1, None), ylim=(1, None))
        plt.suptitle("Cancelled Orders versus Hidden Orders",
               size=14)
        plt.title("Daily, First Quarter of 2017")
        plt.xlabel("Trades on Hidden Orders", size=12)
        plt.ylabel("Cancelled Orders", size=12)
        plt.show()
```



Exercise: Comment on the improvements made here.

The `hue` argument can be changed to vary with the levels of one of the categorical variables.

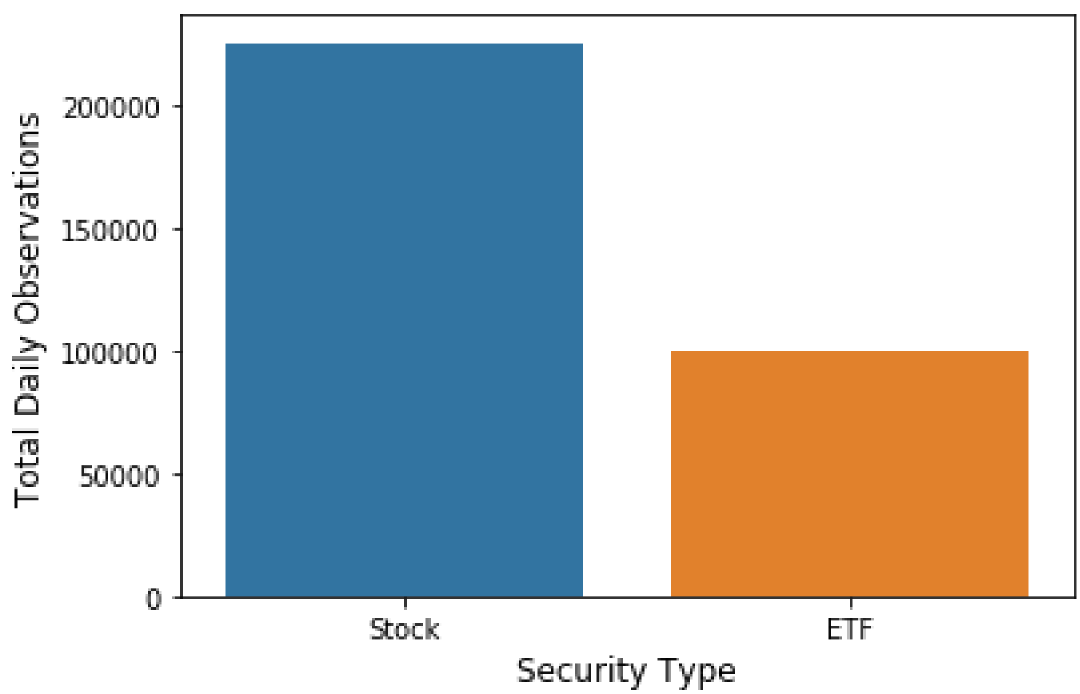
```
In [8]: plt.figure(figsize=[6,3.75])
        ax = sns.scatterplot(x="Hidden", y="Cancels",
            data=fulldata, s=5, hue="Security")
        ax.set(xscale="log", yscale="log",
               xlim=(1, None), ylim=(1, None))
        plt.suptitle("Cancelled Orders versus Hidden Orders",
               size=14)
        plt.title("Daily, First Quarter of 2017", size=10)
        plt.xlabel("Trades on Hidden Orders", size=12)
        plt.ylabel("Cancelled Orders", size=12)
        plt.show()
```



Distributions of Categorical Variables

Basic bar charts can be created to inspect the distribution of a categorical variable, using `countplot()`:

```
In [9]: plt.figure(figsize=[6,4])
        ax = sns.countplot(x="Security", data=fulldata)
        plt.xlabel("Security Type", size=12)
        plt.ylabel("Total Daily Observations", size=12)
        plt.show()
```



For the next plots, we will restrict to data that do not have missing values on `McapRank`, and are of type `Stock`:

```
In [10]: fulldataStock = fulldata[\
             (fulldata["Security"] == "Stock") \
              & (fulldata["McapRank"].notnull())]
```

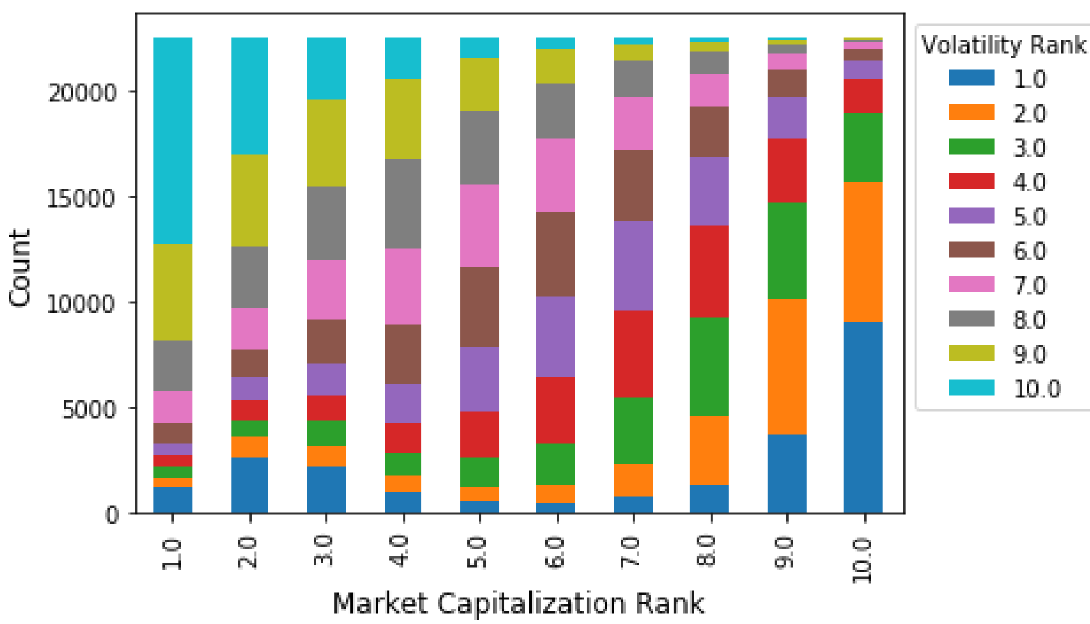
Exercise: Consider the command below. What is the result?

```
In [11]: df = fulldataStock.groupby(['McapRank',\
              'VolatilityRank'])['McapRank'].\
              count().unstack()
```

The **stacked bar chart** created below visualizes these data.

```
In [26]: plt.figure(figsize=[9,7])
         df.plot(kind='bar', stacked=True, legend=False)
         plt.xlabel("Market Capitalization Rank",
             size=12)
         plt.ylabel("Count", size=12)
         plt.legend(title="Volatility Rank",
             bbox_to_anchor=(1.0, 1.0))
         plt.show()
```

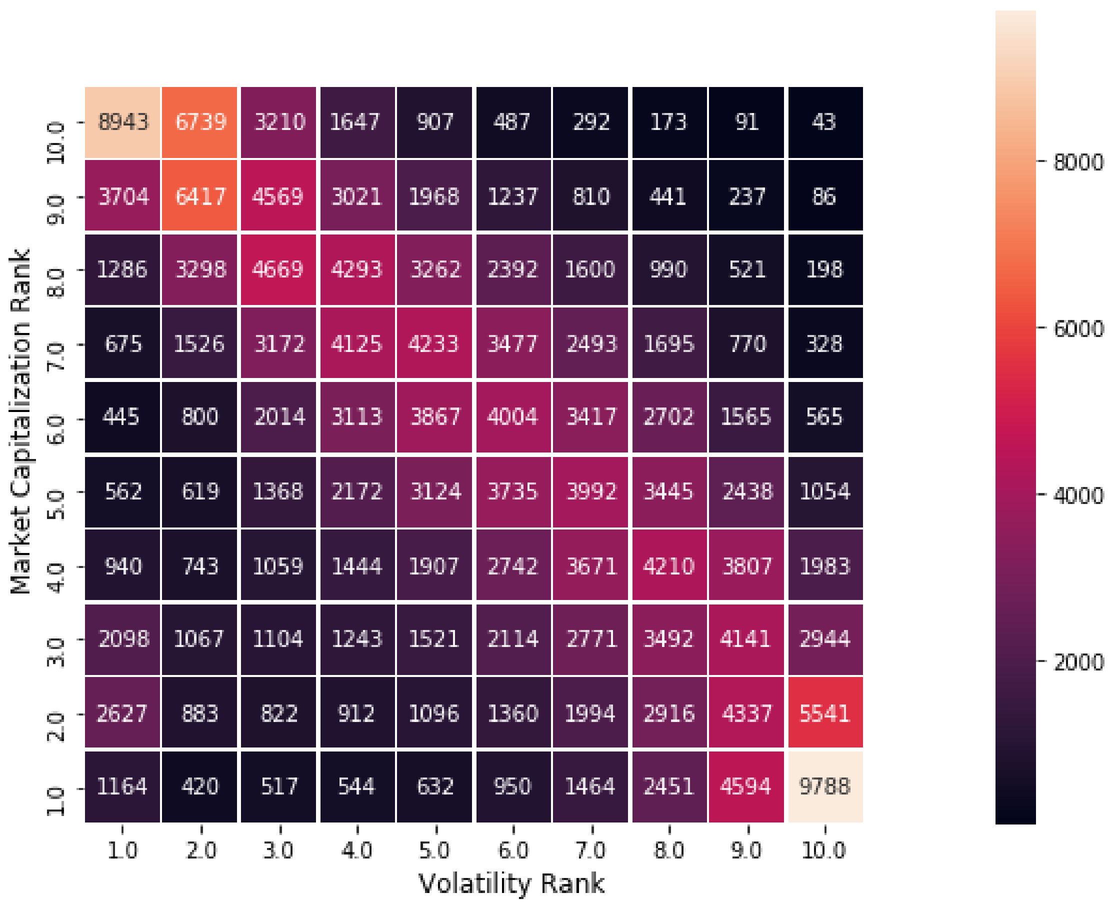
```
<Figure size 648x504 with 0 Axes>
```



Exercise: In this case, the two variables are ordinal. Would the plot make sense if either variable was categorical (and not ordinal)?

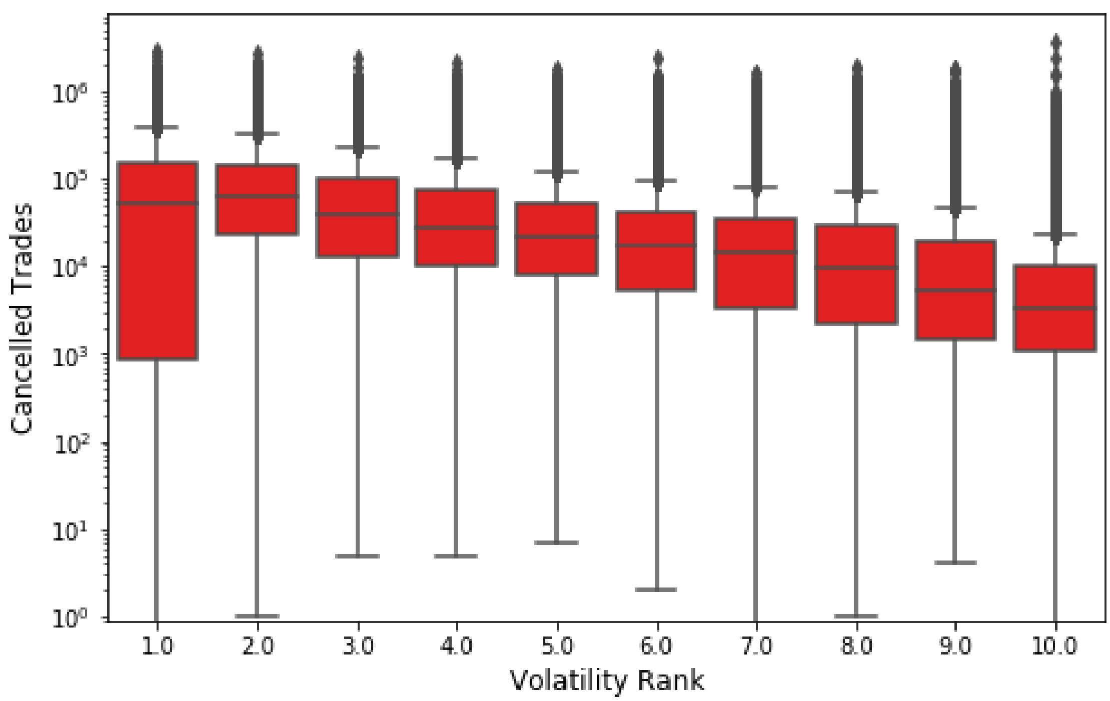
The **heat map** created below gives another visualization of the same data.

```
In [23]: plt.figure(figsize=[9,7])
         ax = sns.heatmap(df, annot=True,
            fmt='d', linewidths=.5)
         ax.invert_yaxis()
         plt.xlabel("Volatility Rank", size=12)
         plt.ylabel("Market Capitalization Rank", size=12)
         ax.set(xlim=(0, 11), ylim=(0, 11))
         plt.show()
```



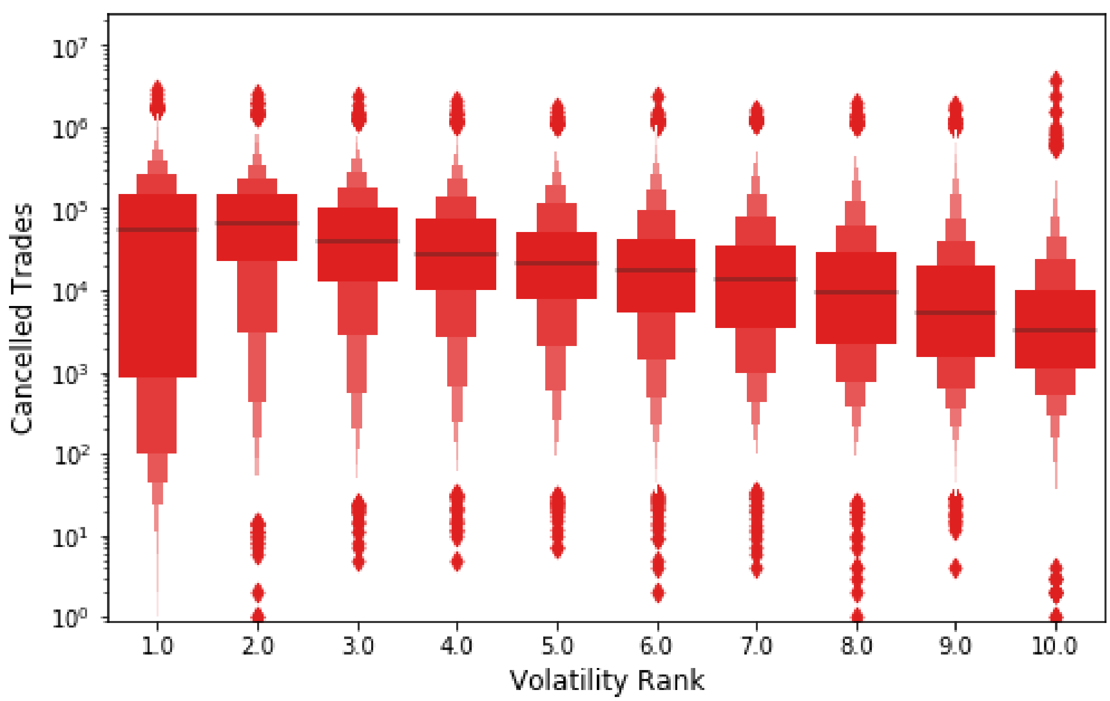
When comparing a ratio variable versus a categorical or ordinal variable, **side-by-side boxplots** are a standard choice.

```
In [15]: plt.figure(figsize=[8,5])
         ax = sns.boxplot(x="VolatilityRank", y="Cancels",
             data=fulldataStock,color="red")
         ax.set(yscale="log", ylim=(0.9, None))
         plt.xlabel("Volatility Rank",size=12)
         plt.ylabel("Cancelled Trades",size=12)
         plt.show()
```



Seaborn includes **boxenplots** to provide a similar, but more sophisticated, look at the data.

```
In [16]: plt.figure(figsize=[8,5])
         ax = sns.boxenplot(x="VolatilityRank",
             y="Cancels",
             data=fulldataStock,color="red")
         ax.set(yscale="log", ylim=(0.9, None))
         plt.xlabel("Volatility Rank",size=12)
         plt.ylabel("Cancelled Trades",size=12)
         plt.show()
```



Facets

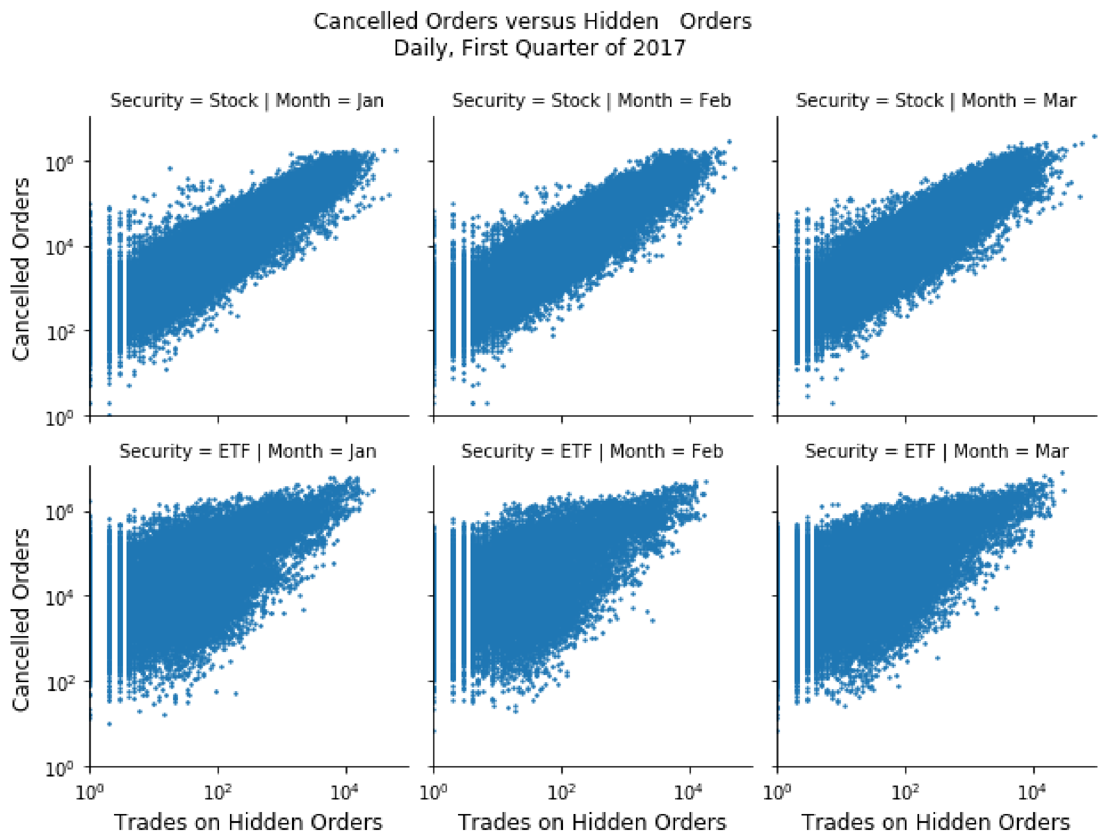
Facets refer to the plots that form a grid. Typically one or more categorical/ordinal variables vary on the rows and columns of the grid.

The example below compares the relationship between the number of cancelled orders and trades on hidden orders, as both a function of security type and month.

Note first how the `Month` column is added to the data frame.

```
In [17]: fulldata["Month"] = fulldata["Date"].\
            map(lambda x: x.strftime("%b"))
         plt.figure(figsize=[8,5])
         g = sns.FacetGrid(fulldata, col="Month",
            row="Security")
         g.map(plt.scatter, "Hidden", "Cancels", s=2)
         g.set(xscale="log", yscale="log",
            xlim=(1, None), ylim=(1, None))
         g.fig.suptitle("Cancelled Orders versus Hidden\
            Orders \n Daily, First Quarter of 2017", y=1.08)
         g.set_xlabels("Trades on Hidden Orders",
            size=12)
         g.set_ylabels("Cancelled Orders",
            size=12)
         g.add_legend()
         plt.show()
```

```
<Figure size 576x360 with 0 Axes>
```



Exercise: Explore https://seaborn.pydata.org/examples/index.html to discover additional ideas for visualization. We will be utilizing some of these approaches as they become relevant.

Part 3: Introduction to Modelling

A fundamental step in many data analyses is to model raw, observable data, i.e. to replace the complex processes that generate these data with a mathematically and computationally simpler fiction.

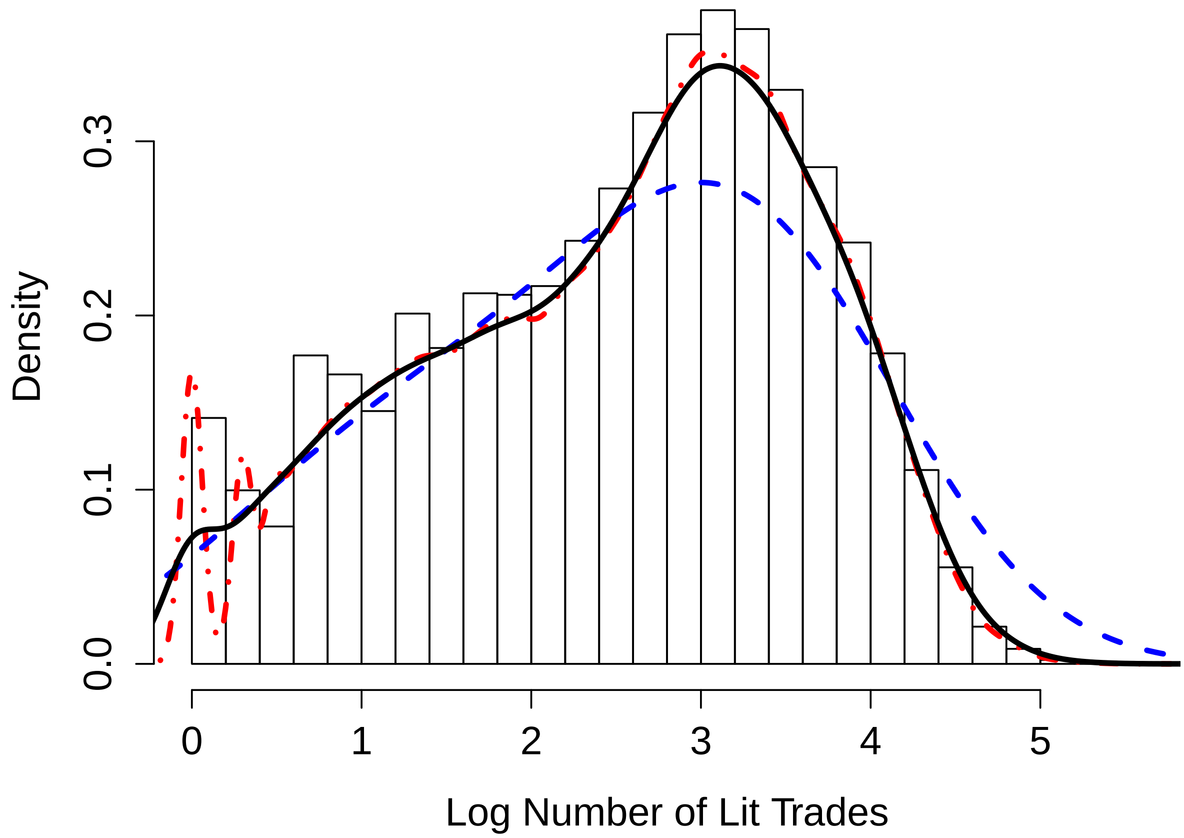
You have likely heard the expression, "All models are wrong; some are useful."[1] That is a good mentality to take towards the modelling process: We do not believe that the data were actually generated by, for example, drawing from a normal distribution. Instead, it **may** be the case that the normal provides a sufficiently accurate approximation to that process.

[1]For a history of this saying, see https://en.wikipedia.org/wiki/All_models_are_wrong.

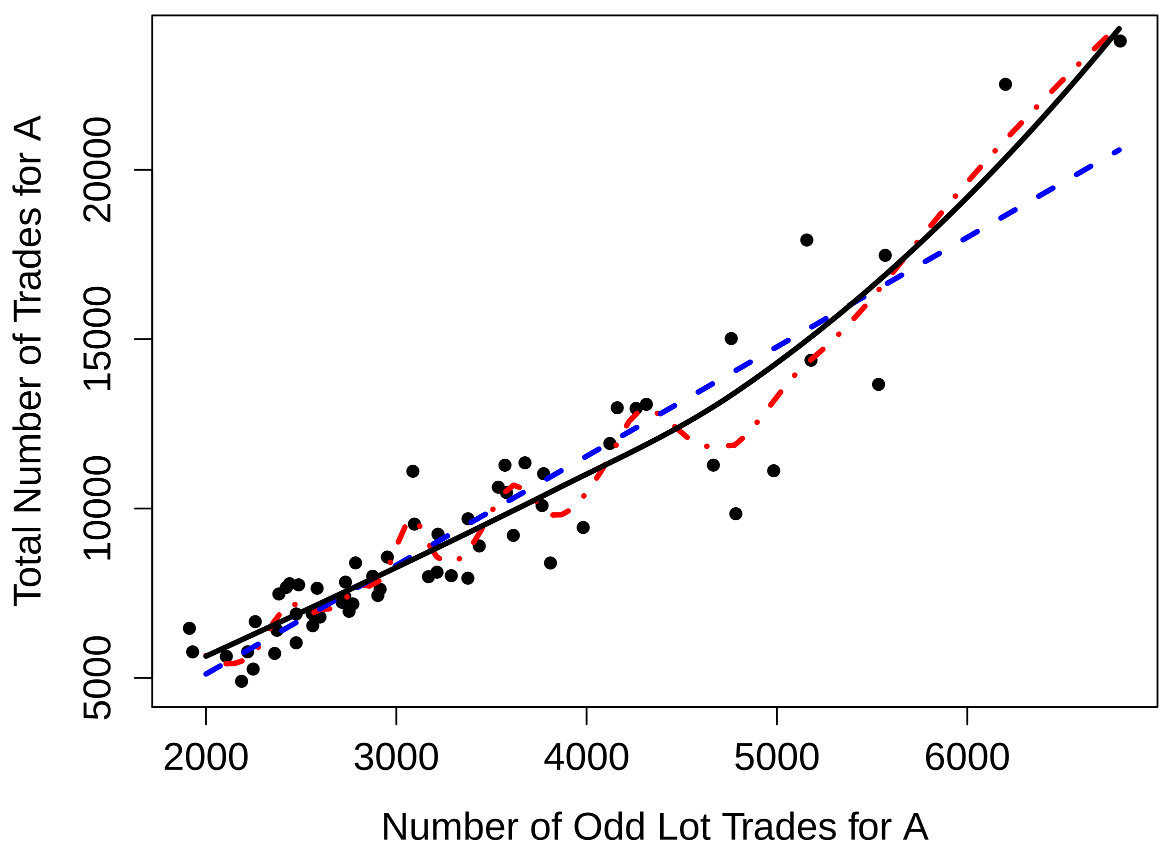
A key heuristic in the statistical modelling process is that of smoothness.

We believe that most relationships, and most distributions, are fundamentally smooth, i.e., we don't try to fit to every little fluctuation that our sample may possess.

Of course, this idea can be taken too far, and oversmoothing can lead to missing important features that are present in data.


Density
0.0
0.1
0.2
0.3
0
1
2
3
4
5
Log Number of Lit Trades

1


Total Number of Trades for A
5000
10000
15000
20000
2000
3000
4000
5000
6000
Number of Odd Lot Trades for A

1

Exercise: Explain how the three proposed models on the previous slides seem to perform, in regards to the appropriate amount of smoothing of the histogram and scatter plot.

Exercise: Explain why overfitting is such a significant concern.

In many situations, there are one or more unknown population quantities that are of interest to estimate. For example,

- The probability that the return exceeds c tomorrow
- The "beta" for some stock over the past year.
- The volatility for some stock over the past three months.
- The mean of some distribution of interest.
- The variance of some distribution of interest.
- The λ parameter from some Exponential distribution.

The model serves as a means to estimating such a quantity, since we imagine that the true model, if known, would provide the true value of the unknown. An estimated model provides an estimate of the unknown.

From a statistical standpoint, we say that a probability model is incompletely specified. I.e., a particular model is assumed, but there are still unknown components, often real-valued parameters.

We use available data (a sample) to estimate these parameters.

The quantities of interest such as those listed above are either the parameters themselves, or some function of those parameters.

Example: In a certain situation, an individual is comfortable assuming that $X_1, X_2, \ldots X_n$ are independent and identically distributed (i.i.d.) with the Normal distribution with mean μ and variance σ^2. These parameters are unknown, and must be estimated from data.

Exercise: Suppose someone wants to estimate the probability that tomorrow's return will be less than c. Describe steps as to how a model could be constructed and the quantity of interest estimated.

Terminology

The parameters are the constants that distinguish probability models in the same "family." Examples of "families" include "normal," "exponential," and so forth.

A statistic is a function of the RVs being used to model the observable data. Examples include the sample mean, the sample variance, etc. A statistic is a **random variable**.

An estimator is a statistic which is used to estimate an unknown parameter or an unknown function of a parameter. It is also a **random variable**.

The "hat notation" is standard for denoting an estimator: $\widehat{\beta}$ is an estimator for β, for example. θ is often used to denote a generic parameter, $\widehat{\theta}$ is its estimator.

Exercise: Explain the sense in which an estimator is appropriately thought of as a random variable.

Performance of Estimators

Our objective is to estimate unknowns, i.e., to construct estimators. For any situation, there are many choices of estimator, so we require methods to assess the performance of an estimator.

The “performance” is closely tied to the distribution of the estimator. The distributions we learned in Probability (plus some more) are useful for this purpose.

An ideal estimator $\widehat{\theta}$ for θ would have two properties:

1. The estimator should be accurate, i.e., its distribution would be centered on the true value θ.

2. The estimator should be precise, i.e., its distribution has a small spread (small variance)

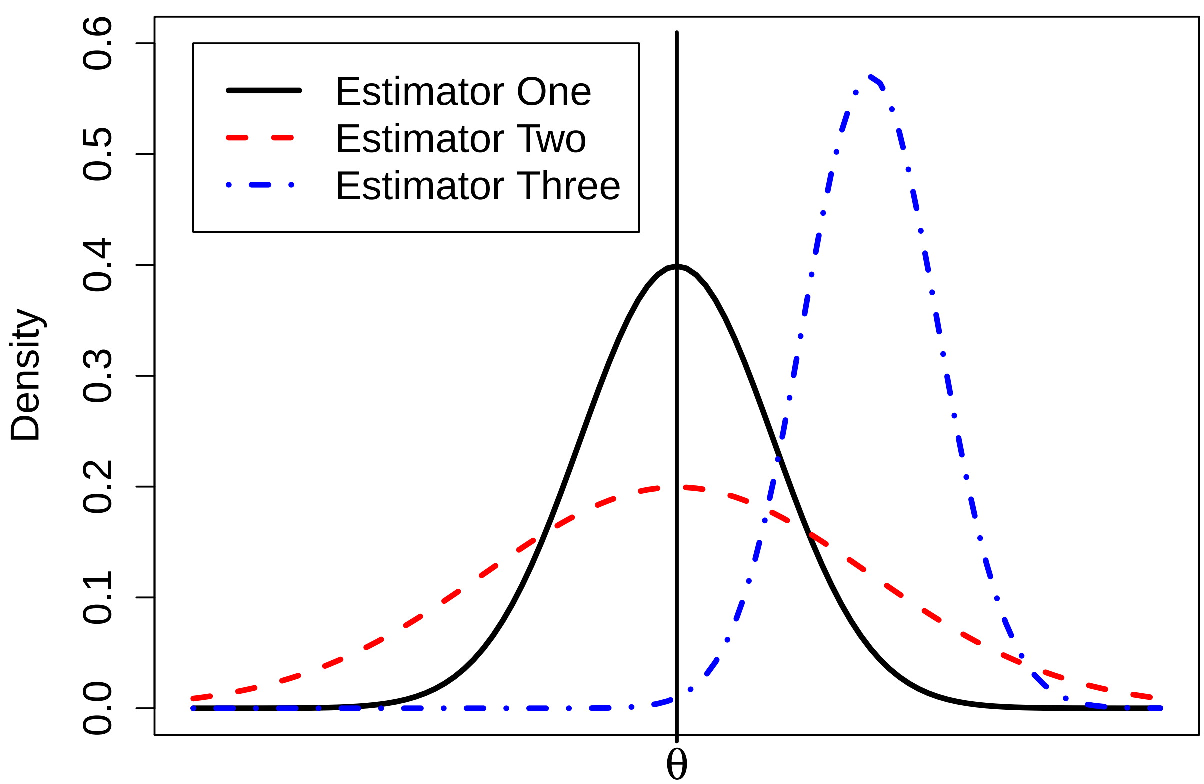
Example: The figure on the following page compares the distributions of three hypothetical estimators for a parameter θ, call them $\widehat{\theta}_1$, $\widehat{\theta}_2$, and $\widehat{\theta}_3$.

We note that $E(\widehat{\theta}_1) = \theta$, so it is optimally accurate. Also, $E(\widehat{\theta}_2) = \theta$.

But, $V(\widehat{\theta}_1) < V(\widehat{\theta}_2)$, so $\widehat{\theta}_1$ is more precise than $\widehat{\theta}_2$.

So, we would prefer $\widehat{\theta}_1$ to $\widehat{\theta}_2$.

Also, $V(\widehat{\theta}_3) < V(\widehat{\theta}_1)$, but $\widehat{\theta}_3$ has terrible accuracy.



An estimator $\widehat{\theta}$ is said to be an unbiased estimator for θ if $E(\widehat{\theta}) = \theta$.

The variability in an estimator (its precision) is naturally quantified by its variance, $V(\widehat{\theta})$.

The standard error (SE) of $\widehat{\theta}$ is the square root of its variance. The SE is often reported along with an estimate in order to give a sense of the amount of error one should anticipate in that estimate.

An estimator $\widehat{\theta}$ is said to be the minimum variance unbiased estimator for θ (MVUE) if $\widehat{\theta}$ is unbiased for θ and for any other unbiased estimator $\widehat{\theta}'$, $V(\widehat{\theta}) \leq V(\widehat{\theta}')$.

Exercise: Suppose that X has the binomial(n, p) distribution. It is common to estimate p using $\widehat{p} = X/n$, the sample proportion. Is this estimator unbiased? What is its variance and standard error?

There is often a "tradeoff" between bias and variance (accuracy and precision). This is called the bias-variance tradeoff.

In some cases, allowing some bias leads to reduced variance.

This tradeoff is made formal via the mean squared error (MSE). For an estimator $\widehat{\theta}$ for θ,

$$\mathrm{MSE}_\theta(\widehat{\theta}) = E\Big(\big(\widehat{\theta} - \theta\big)^2\Big)$$

i.e., it is the expected squared error in estimating θ when using $\widehat{\theta}$.

One can show that

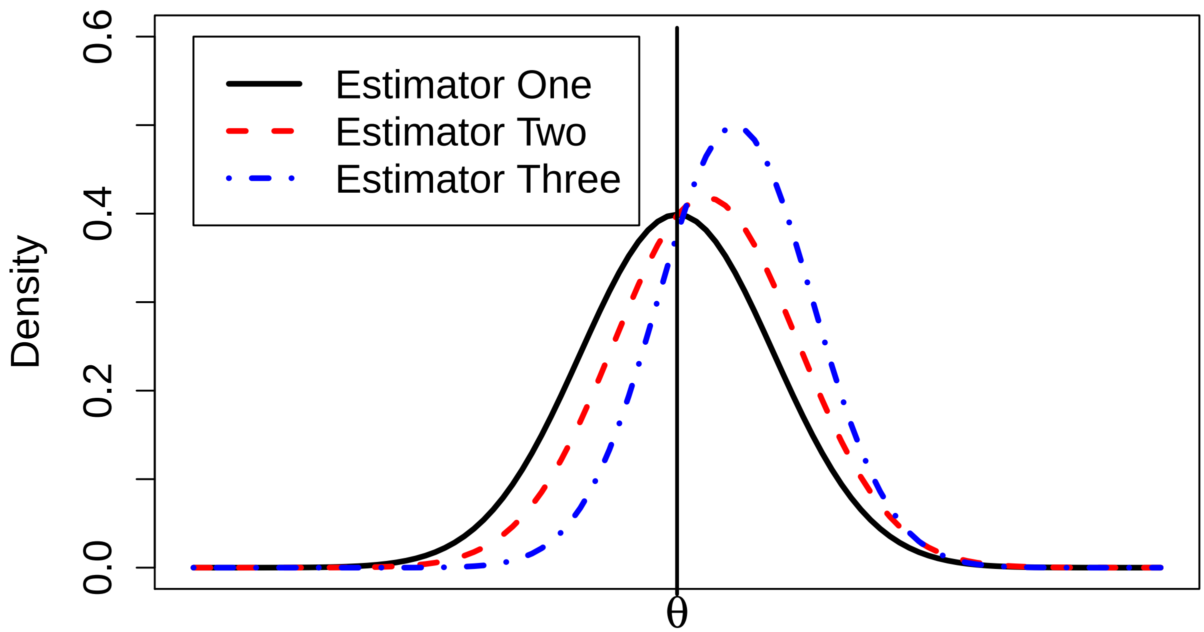
$$\mathrm{MSE}_\theta(\widehat{\theta}) = V(\widehat{\theta}) + \mathrm{bias}_\theta^2(\widehat{\theta})$$

where

$$\mathrm{bias}_\theta(\widehat{\theta}) = E(\widehat{\theta}) - \theta$$

Exercise: How does the bias/variance tradeoff relate to our initial discussion of smoothness?

In the example below, all three estimators have the same MSE.



Important Cases

There are a few estimation situations that worth special attention.

Case 1: The first we have already explored, that of estimating a population proportion. We found that the sample proportion is unbiased and has a standard error of

$$\sqrt{\frac{p(1-p)}{n}}$$

Of course, we do not know the value of p, so typically it is replaced by its best estimate and the approximate SE is reported as

$$\sqrt{\frac{\widehat{p}(1-\widehat{p})}{n}}$$

Exercise: We have derived the mean and variance of the sample proportion, but what is its distribution?

Case 2: Suppose that $X_1, X_2, \ldots, X_n$ are random variables and $E(X_i) = \mu$ and $V(X_i) = \sigma^2$ for all i. Further assume that these random variables are uncorrelated (meaning that every pair of random variables is uncorrelated).

Then, the standard estimator for μ is the sample mean, $\overline{X}$, and the standard estimator for σ^2 is the sample variance,

$$s^2 = \sum_{i=1}^{n} \left(X_i - \overline{X}\right)^2 \Big/ (n-1)$$

Note that the case where $X_1, X_2, \ldots, X_n$ are i.i.d. is a special instance of this situation.

Exercise: Derive the mean, variance, and standard error of the sample mean. What is its MSE as an estimator of μ?

Exercise: If we did assume that $X_1, X_2, \ldots, X_n$ are i.i.d., what could we say about the distribution of the sample mean?

Exercise: As defined, the sample variance is an unbiased estimator of σ^2. (The proof is not difficult, but a little tedious.)

Consider the following question: What choice of a minimizes the quantity

$$\sum_{i=1}^{n} (X_i - a)^2 \ ?$$

How does this result suggest it would be a good idea to estimate σ^2 by dividing the sum of squares by a constant less than n?

Case 3: Suppose that $X_1, X_2, \ldots, X_n$ are i.i.d. with the normal distribution with mean μ and variance σ^2.

Of course, this is a special instance of "Case 2" presented above, and the sample mean and sample variance remain unbiased estimators of μ and σ^2, respectively.

We can make some stronger statements, as well.

First, we know that $\overline{X}$ has **exactly** the normal distribution with mean μ and variance σ^2/n for **any** sample size.

Further results depend on properties of probability distributions that we will present now.

The Chi-squared distribution with k degrees of freedom is a special case of the Gamma distribution. In particular, if X has the Gamma$(k/2, 1/2)$ distribution, then X has the Chi-square distribution with k degrees of freedom.

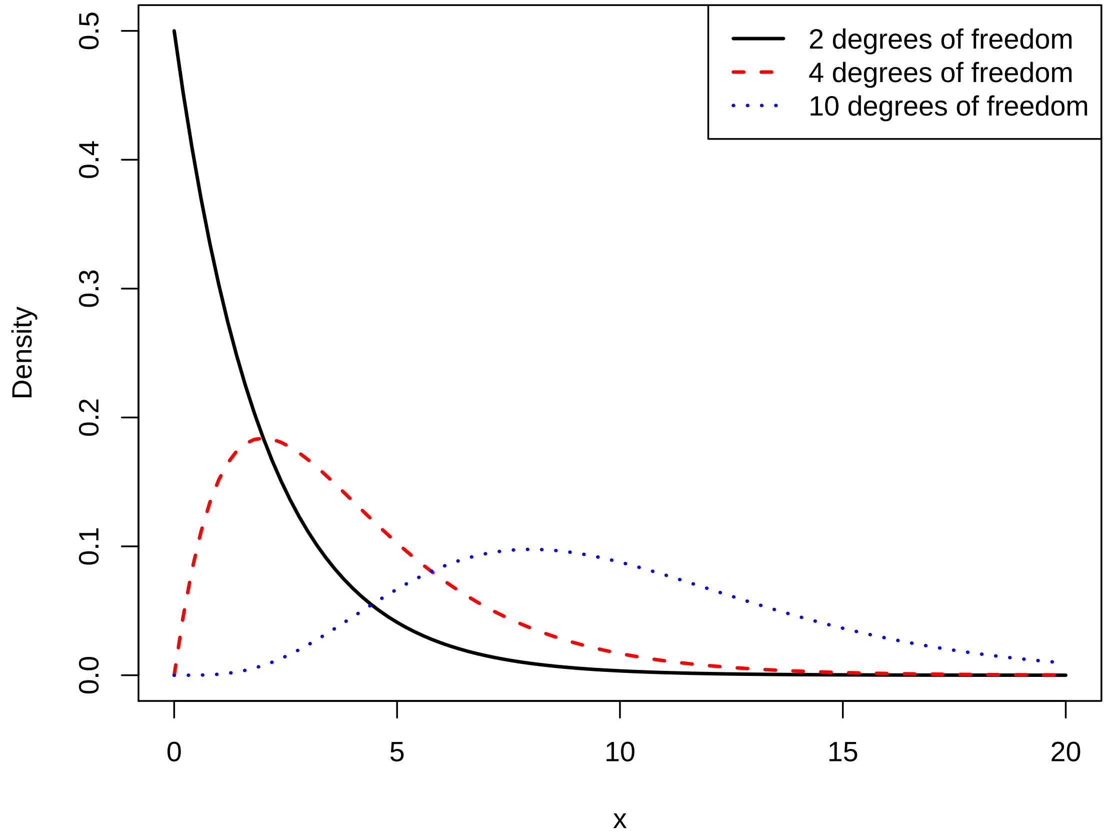
Also, if $Z_1, Z_2, \ldots, Z_k$ are i.i.d. standard normal random variables, then

$$X = \sum_{i=1}^{k} Z_i^2$$

has the Chi-squared distribution with k degrees of freedom.

If X has this distribution, then $E(X) = k$ and $V(X) = 2k$.

Plots of pdfs from the Chi-squared distribution are shown on the following slide.



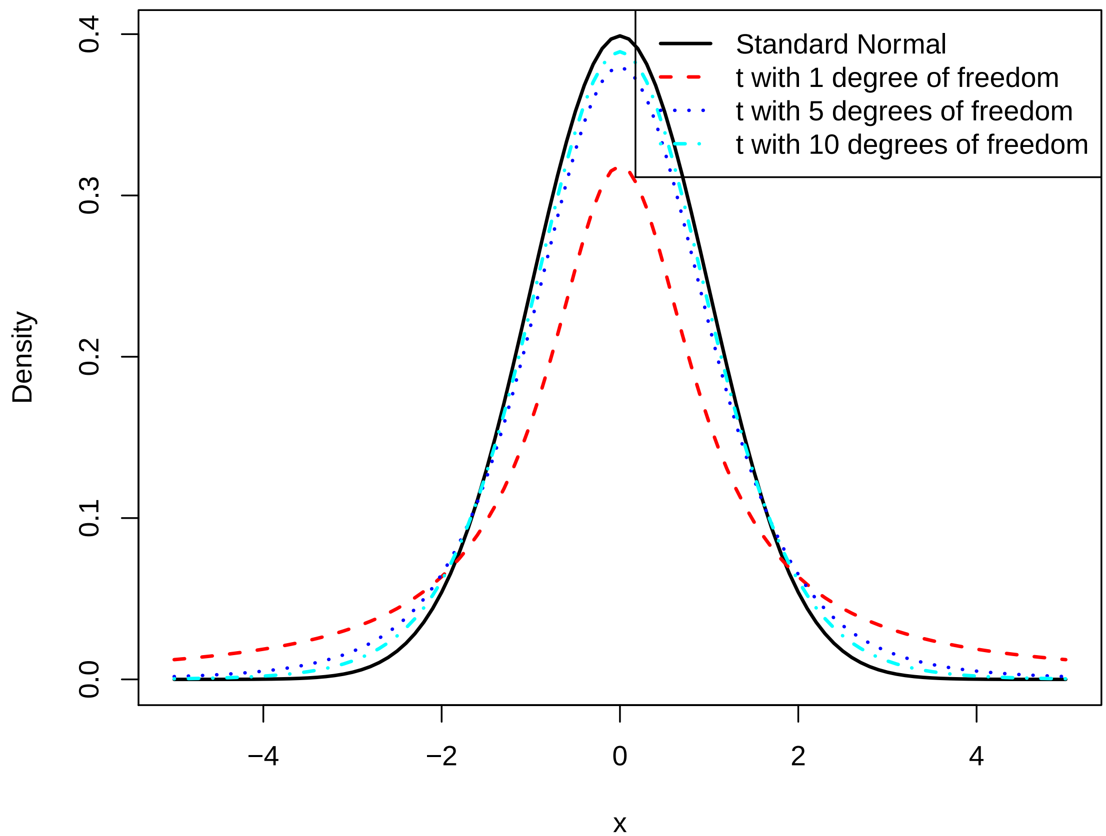
The *t-distribution with ν degrees of freedom* is defined to be a random variable T that can be constructed as

$$T = \frac{Z}{\sqrt{X/\nu}}$$

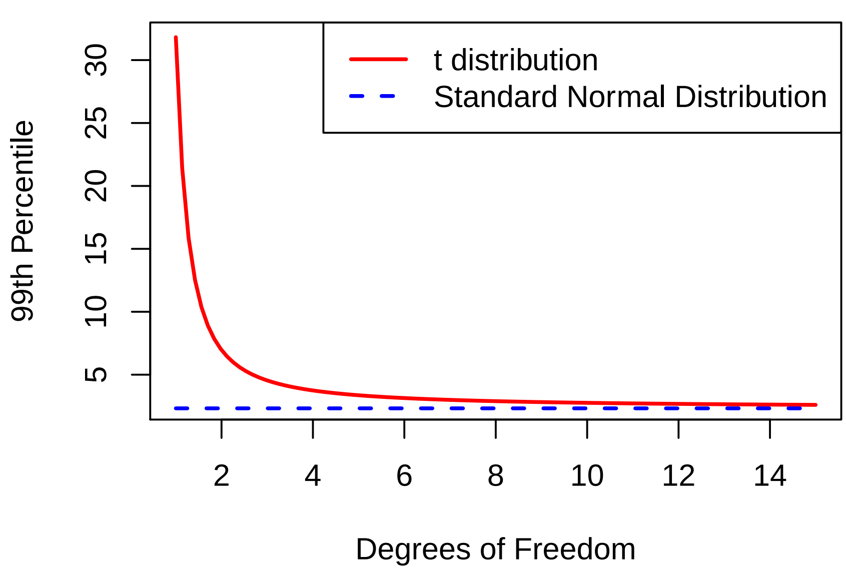
where Z is a random variable with the standard normal distribution, and X is a random variable with the Chi-squared distribution with ν degrees of freedom. Further, Z and X are assumed independent.

This distribution has a "normal-shaped" pdf, but has heavier tails than the normal distribution. As ν increases, the distribution converges to the standard normal distribution.

It holds that $E(T) = 0$ when $\nu \geq 2$ and that $V(T) = \nu/(\nu - 2)$ when $\nu \geq 3$.



The figure below makes clearer how much additional probability is in the tails of the t-distribution compared to the standard normal. For $\nu = 1$, 1% of the observations will be larger than 30. The 99th percentile of the standard normal distribution is 2.33.



Now we can state the four **classic results** that hold under our "Case 3:"

Assuming that $X_1, X_2, \ldots, X_n$ are i.i.d. Normal(μ, σ^2), then

1. $\overline{X}$ and s^2 are independent.
2. The quantity
$$\frac{\overline{X} - \mu}{\sigma/\sqrt{n}}$$
has the standard normal distribution.
3. The quantity
$$\frac{(n-1)s^2}{\sigma^2}$$
has the Chi-squared distribution with $n-1$ degrees of freedom.
4. The quantity
$$\frac{\overline{X} - \mu}{s/\sqrt{n}}$$
has the t-distribution with $n-1$ degrees of freedom.

Exercise: Under "Case 3" it is easier to show that s^2 is an unbiased estimator of σ^2, and its also easy to derive the variance of this estimator. Do these now.

Confidence Intervals

The already-mentioned standard error (SE) is a useful tool for quantifying error in an estimate, but a common alternative is the use of confidence intervals for unknown parameters.

Formally, we call (L, U) a $100(1-\alpha)\%$ confidence interval for θ if

$$P(L \leq \theta \leq U) = 1 - \alpha$$

Hence, if want a "95% confidence interval," set $\alpha = 0.05$. Other common choices are 90% and 99% confidence intervals (α = 0.10 and $\alpha = 0.01$, respectively).

An appropriate statement regarding a confidence interval is, for example, "I am 95% confident that the true value of θ is between 10.6 and 11.7."

It is **not** appropriate to say that "The probability that θ is between 10.6 and 11.7 is 0.95." This is incorrect because there is nothing random in that statement: θ, 10.6 and 11.7 are all constants.

The randomness is in the procedure itself. If one constructs 95% confidence intervals in a wide range of situations, then 95% of those intervals will cover the true value of the parameter.

There is an alternative approach to statistical inference, Bayesian inference, in which parameters are treated as random variables. More on that later.

Exercise: Use our "classic results" above for "Case 3" to derive confidence intervals for μ and σ^2 in that situation. For estimating μ, onsider both the instances where σ^2 is known and unknown.

The next slides present four important confidence intervals that are commonly used.

Confidence Interval for Population Mean, μ, when n is large $(n \geq 30)$

Assume that the sample is i.i.d. from a distribution with mean μ and variance σ^2, with both μ and σ^2 unknown. Then, a $100(1-\alpha)\%$ confidence interval for μ is

$$\overline{X} \pm z_{\alpha/2}\left(\frac{s}{\sqrt{n}}\right)$$

where z_a is such that $P(Z > z_a) = a$ when Z has the standard normal distribution. In R notation, z_a equals `qnorm(1-a)`.

Confidence Interval for Population Mean, μ, when n is small $(n < 30)$

Assume that the sample is i.i.d. from the normal distribution with mean μ and variance σ^2, with both μ and σ^2 unknown. Then, a $100(1-\alpha)\%$ confidence interval for μ is

$$\overline{X} \pm t_{\alpha/2,n-1}\left(\frac{s}{\sqrt{n}}\right)$$

where $t_{a,v}$ is such that $P(T > t_{a,v}) = a$ when T has the t-distribution with v degrees of freedom. In R notation, $t_{a,v}$ equals `qt(1-a,v)`.

Confidence Interval for Population Variance, σ^2

Assume that the sample is i.i.d. from the normal distribution with mean μ and variance σ^2, with both μ and σ^2 unknown. Then, a $100(1-\alpha)\%$ confidence interval for σ^2 is

$$\left(\frac{(n-1)\,s^2}{\chi^2_{\alpha/2,n-1}}, \frac{(n-1)\,s^2}{\chi^2_{(1-\alpha/2),n-1}}\right)$$

where $\chi^2_{a,v}$ is such that $P(U > \chi^2_{a,v}) = a$ when U has the chi-squared distribution with v degrees of freedom. In R notation, $\chi^2_{a,v}$ equals `qchisq(1-a,v)`.

Confidence Interval for Population Proportion, p

Assume that n is large enough that you are confident that $np \geq 5$ and $n(1-p) \geq 5$. Let $\widehat{p} = X/n$ where X is binomial(n,p), and p is unknown. Then, a $100(1-\alpha)\%$ confidence interval for p is

$$\widehat{p} \pm z_{\alpha/2}\sqrt{\frac{\widehat{p}(1-\widehat{p})}{n}}$$

where z_a is such that $P(Z > z_a) = a$ when Z has the standard normal distribution. In R notation, z_a equals `qnorm(1-a)`.

Exercise: A sample of size 50 is drawn from a population with unknown mean and variance. It holds that $\overline{x} = 14.6$ and $s = 1.3$. Estimate the population mean. State both the standard error for the estimator and a 90% confidence interval for μ.

Part 4: Constructing Estimators

Summary

Recall that typically a probability model is incompletely specified. I.e., a particular model is assumed, but there are still unknown components, often real-valued parameters. We use available data (a sample) to estimate these parameters.

In this Part we describe methods for constructing estimators for unknown parameters.

Terminology Review

The parameters are the constants that distinguish probability models in the same "family." Examples of "families" include "normal," "exponential," and so forth.

A statistic is a function of the RVs being used to model the observable data. Examples include the sample mean, the sample variance, etc. A statistic is a **random variable**.

An estimator is a statistic which is used to estimate an unknown parameter. It is also a **random variable**.

The "hat notation" is standard for denoting an estimator: $\widehat{\beta}$ is an estimator for β, for example. θ is often used to denote a generic parameter, $\widehat{\theta}$ is its estimator.

An estimator $\widehat{\theta}$ is said to be an unbiased estimator for θ if $E(\widehat{\theta}) = \theta$.

The variability in an estimator (its precision) is naturally quantified by its variance, $V(\widehat{\theta})$.

The standard error (SE) of $\widehat{\theta}$ is the square root of its variance. The SE is often reported along with an estimate in order to give a sense of the amount of error one should anticipate in that estimate.

The Method of Moments

Simple cases such as estimating the population mean and estimating the population proportion can be misleading: In those cases it is "obvious" what the natural estimator is, since one can use the sample mean or sample proportion as the estimator, respectively.

In other cases, the choice of estimator is not so clear.

We first consider a simple approach to constructing estimators, the method of moments.

First, recall that for any positive integer k, the quantity $E(X^k)$ is called the k^{th} moment for the distribution of X. We might call this the "k^{th} population moment" to be more clear.

Imagine that I have random variables $X_1, X_2, \ldots, X_n$ that model the observable sample. Then the k^{th} sample moment is

$$M_k = \sum_{i=1}^{n} X_i^k \Big/ n$$

Note that M_k is our best estimator for $E(X^k)$, for instance $E(M_k) = E(X^k)$

Now, suppose that $X_1, X_2, \ldots, X_n$ are i.i.d. each with density $f_X(x; \boldsymbol{\theta})$.

Here, $\boldsymbol{\theta} = (\theta_1, \theta_2, \ldots, \theta_p)$ is a vector of parameters.

Set up a system of p equations, setting the first p sample moments equal to the first p population moments:

$$\begin{aligned} M_1 &= E(X) \\ M_2 &= E(X^2) \\ &\vdots \\ M_p &= E(X^p) \end{aligned}$$

We note that the sample moments are all quantities we can determine from the available sample. We also note that the population moments are functions of the unknown parameters.

We can now solve this system of equations for $\theta_1, \theta_2, \ldots, \theta_p$:

$$\begin{aligned} \theta_1 &= g_1(X_1, X_2, \ldots, X_n) \\ \theta_2 &= g_2(X_1, X_2, \ldots, X_n) \\ \vdots & \quad \vdots \\ \theta_p &= g_p(X_1, X_2, \ldots, X_n) \end{aligned}$$

These equations are built on the (incorrect) belief that $M_i = E(X^i)$.

In fact, $M_i \approx E(X^i)$, so

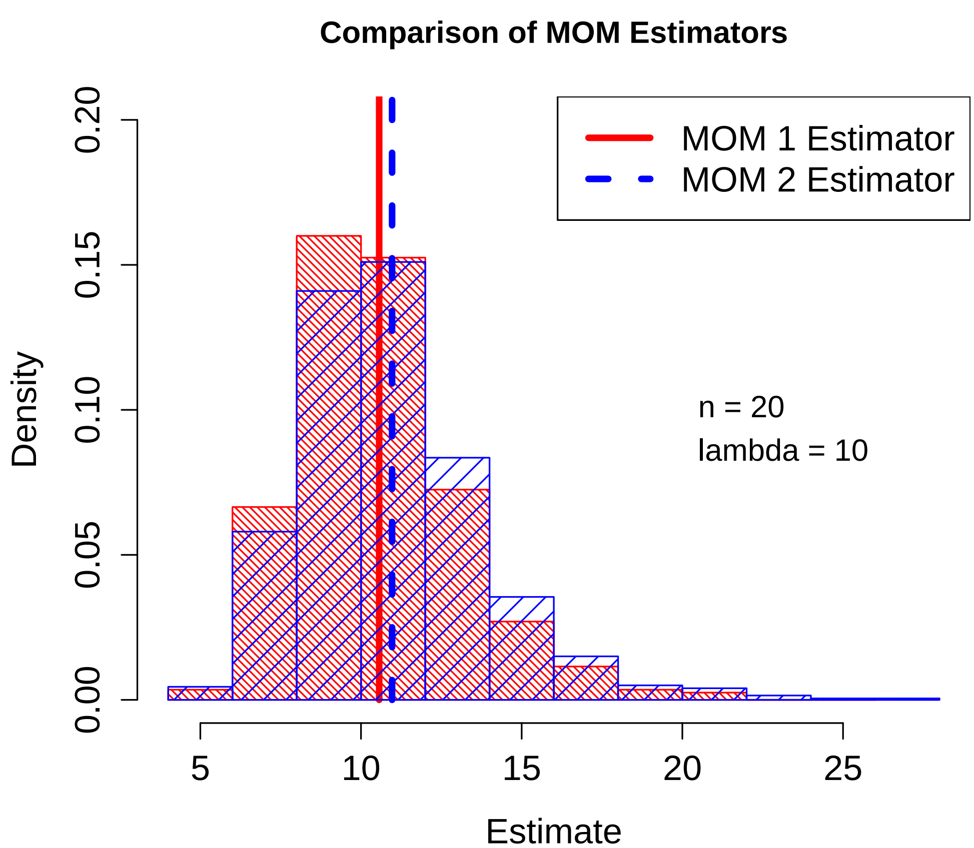
$$\theta_i \approx g_i(X_1, X_2, \ldots, X_n)$$

and we construct our estimator as

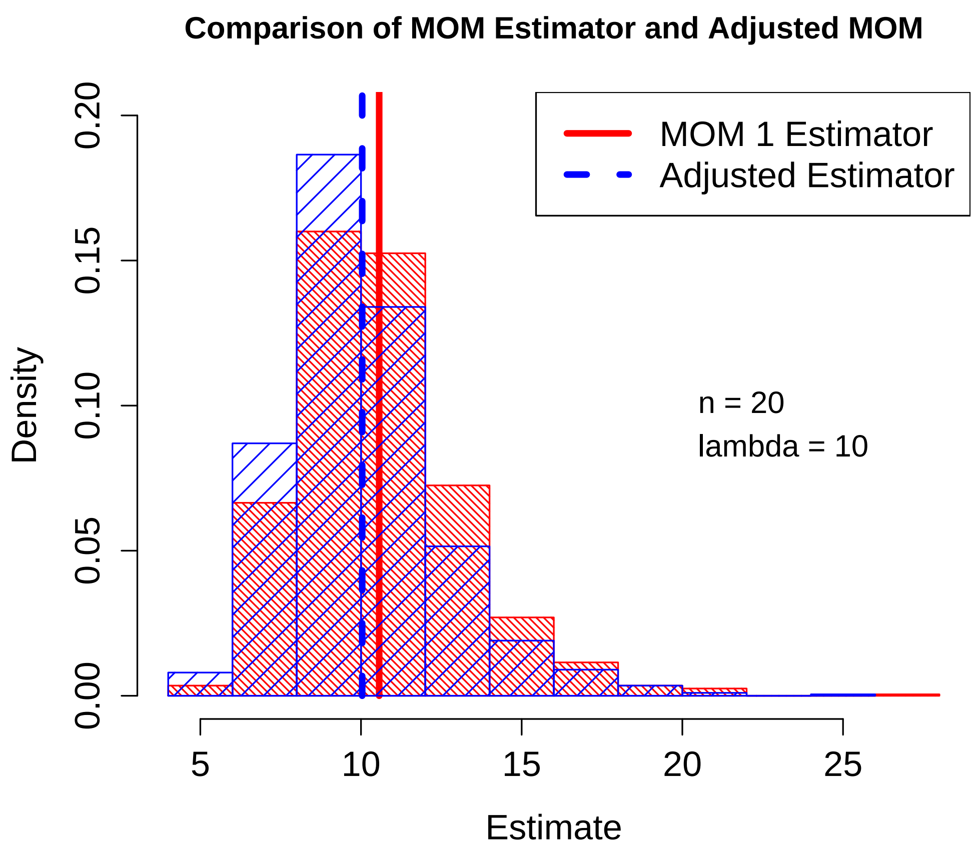
$$\widehat{\theta}_{i,\text{MOM}} = g_i(X_1, X_2, \ldots, X_n)$$

Exercise: Suppose that $X_1, X_2, \ldots, X_n$ are i.i.d. from the Exponential(λ) distribution. (Recall that this means that $E(X_i) = 1/\lambda$.) What is the method of moments estimator for λ?

Exercise: What is the result if, instead of the first moment, the second moment is used to construct the estimator for λ in the previous example?



Exercise: Is the method of moments estimator found above (using the first moment) an unbiased estimator for λ? Can you adjust it to make it an unbiased estimator?



The comparisons made above between the three estimators are based **entirely on simulations**. There is **no** observed data involved at all. This is the frequentist approach to assessing estimators: **how does the estimator perform in repeated use?**.

As seen above, simulations are a useful tool for cases where the mean and variance derivations are not available.

Exercise: Interpret the output below. What is the approximate MSE of each of the three estimators?

```
> print(c(mean(mom1), mean(mom2), mean(adju)))
[1] 10.56553 10.96759 10.03725
```

```
> print(c(var(mom1), var(mom2), var(adju)))
[1] 6.574420 8.224773 5.933414
```

Maximum Likelihood Estimation

Maximum likeihood estimation (MLE) is a powerful, general approach to constructing estimators.

We will begin the explanations with simple, one-dimensional examples, but the concepts extend to complex, high-dimensional models in a natural manner.

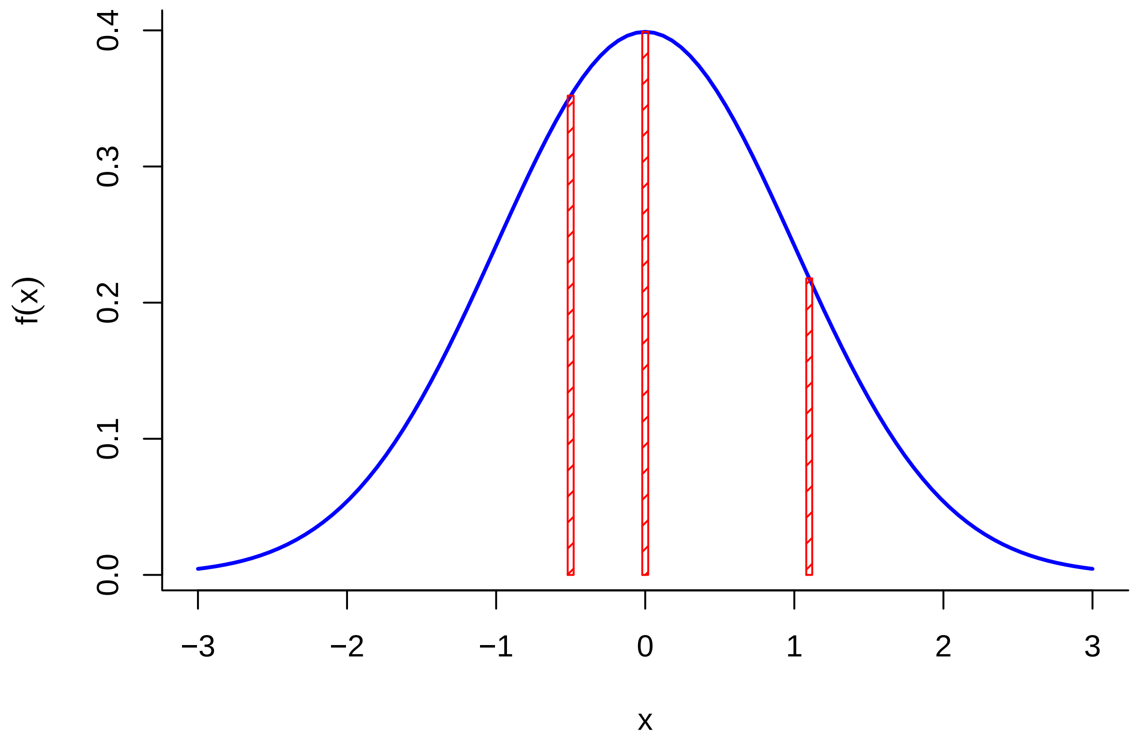
Importantly, there is also a general way of calculating standard errors of MLEs, and, under a wide range of conditions, the MLE is approximately normally distributed. This makes inference with MLEs straightforward.

Basic Setup

Recall that if X is a continuous random variable with density f_X, that does **not** mean that $P(X = a) = f_X(a)$, i.e., the density does **not** return probabilities. Indeed, $P(X = a) = 0$ for all a.

It is true that the density is higher in regions where there is the most probability: For fixed, small $\epsilon > 0$, if we want to find a to maximize $P(a - \epsilon \leq X \leq a + \epsilon)$, then we would look for the a that maximizes $f_X(a)$.

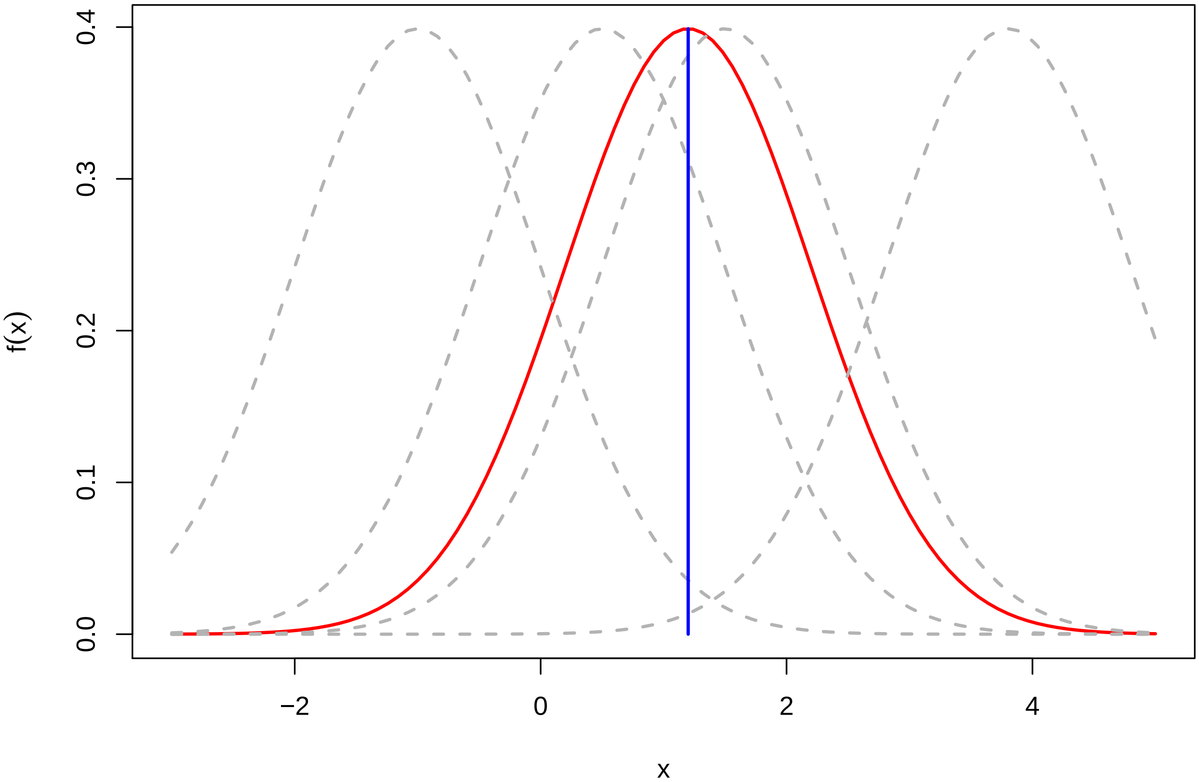
See the figure on the following slide. The standard normal density is depicted. The most likely values are close to zero, where the density is maximized.



As previously stated, we use probability distributions such as the normal to "model" the real processes that generate data. So, if we assume that our observable data is drawn from the standard normal distribution, the density above specifies where we are more or less likely to see our data value.

We can turn this idea around: Suppose that we observe data $x = 1.2$, and we are willing to assume that it came from a normal distribution with mean μ and variance one. What value of μ makes the observation $x = 1.2$ the most likely?

See the figure on the following slide. Different choices of μ are displayed. It's not difficult to conclude that when $\mu = 1.2$, the likelihood is maximized.



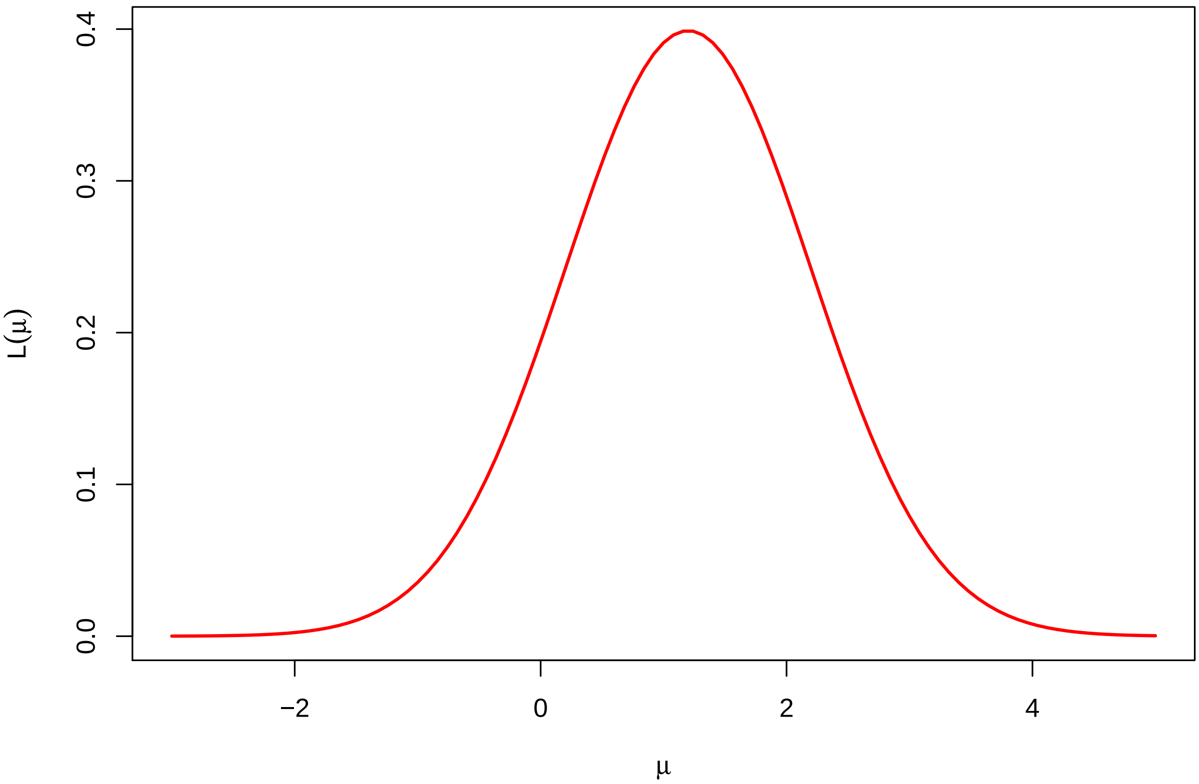
The likelihood function is constructed by calculating the distribution for the data, and then **holding the data constant** and allowing the parameter(s) to vary.

In the simple example above, we assume that we observe a single data point x that was generated from a normal distribution with mean μ and variance one. Hence, the distribution is

$$f(x;\mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2}\right)$$

If we fix the data $x = 1.2$, and think of this as a function of μ, the we arrive at our likelihood function:

$$L(\mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(1.2-\mu)^2}{2}\right)$$



The maximum likelihod estimator for the parameter is the choice of the parameter which maximizes the likelihood function.

Heuristically, we ask: Among all possible choices for the parameter, which one makes our observed data the most likely?

It is worth stressing that the likelihood function is **not** itself a probability distribution. It is **not** specifying the relative “probabilities” for different values of the parameter.

Remember that in the frequentist approach, the unknown parameter is a constant, and not random.

Exercise: An urn contains black and white balls. All that we know is that the balls have a color ratio of 1 to 3. (We do not know if there are more white or more black balls.) We draw three balls **with replacement** from the urn. Let X equal the number of black draws. Derive the likelihood function and the MLE as a function of X.

Exercise: Now suppose that θ equals the proportion of balls in the urn which are black. We know that $0 < \theta < 1$. We draw ten times with replacement, and count seven black balls and three white balls. Derive the MLE for θ.

The preceding examples involved a single, observable random variable. In most cases, we have a sequence of random variables in the model.

A common case is where $X_1, X_2, \ldots, X_n$ are i.i.d. with density $f_X(x;\theta)$, where θ is the parameter to be estimated. In this case, the joint distribution of the random variables is

$$\prod_{i=1}^{n} f_{X_i}(x_i;\theta)$$

and the likelihood function is

$$L(\theta) = \prod_{i=1}^{n} f_{X_i}(x_i;\theta)$$

Finding the MLE Analytically

In some situations the MLE can be derived analytically, usually via calculus, but sometimes via direct inspection of the likelihood.

One "trick" that is often very helpful: The likelihood function is positive at its maximum. Hence, the likelihood is maximized at the same θ as does the logarithm of the likelihood.

Hence, we often work with the log likelihood instead of the likelihood. This is often denoted $\ell(\theta) = \log L(\theta)$.

In the case where $X_1, X_2, \ldots, X_n$ are i.i.d. $f_X(x;\theta)$, then

$$\ell(\theta) = \log \prod_{i=1}^{n} f_{X_i}(x_i;\theta) = \sum_{i=1}^{n} \log f_{X_i}(x_i;\theta)$$

Exercise: Suppose that $X_1, X_2, \ldots, X_n$ are i.i.d. with the Exponential(λ) distribution. Derive the MLE for λ.

Exercise: Suppose that $X_1, X_2, \ldots, X_n$ are i.i.d. with the Beta$(\alpha, 1)$ distribution. Derive the MLE for α. Compare this with the MOM estimator.

Exercise: Assume that $X_1, X_2, \ldots, X_n$ are i.i.d. with the Normal(μ, σ^2) distribution. Derive the MLE for μ and σ^2.

Exercise: Suppose that $X_1, X_2, \ldots, X_n$ are i.i.d. Uniform$(0, \theta)$. Derive the MLE for θ. What is the MOM in this case?

Part 5: Properties of MLEs

Summary

In this Part we explore some of the important properties possessed by maximum likelihood estimators.

Briefly stated, we will see that, asymptotically, these estimators cannot be beat, in the sense that they have minimal MSE, and also are approximatley normally distributed with variance that is easy to calculate. This makes quantifying the error in the estimators straightforward.

Invariance of the MLE

The invariance result can be stated as follows: If $\widehat{\theta}_{\mathrm{MLE}}$ is the MLE for θ, then $g(\widehat{\theta}_{\mathrm{MLE}})$ is the MLE for $g(\theta)$.

This is a useful, important result because it is often the case that we are truly interested in estimating some function of the parameter θ, not θ itself.

Using this, we can say, for example, that if $X_1, X_2, \ldots, X_n$ are i.i.d. normal(μ, σ^2), then the MLE for σ is

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(X_i - \overline{X}\right)^2}$$

and the MLE for the signal to noise ratio μ/σ is

$$\overline{X} \Big/ \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(X_i - \overline{X}\right)^2}$$

One consequence of this property is that estimation is not affected by how one chooses to parameterize a model.

For example, some choose to represent the Exponential distribution using the "mean parameterization" which leads to $E(X) = \mu$, whereas we have used the "rate parameterization" which leads to $E(X) = 1/\lambda$. Because of invariance, these two approaches will lead to the same estimates, i.e., $\widehat{\mu} = 1/\widehat{\lambda}$.

Exercise: Does unbiasedness have this invariance property?

Exercise: Suppose that $X_1, X_2, \ldots, X_n$ are i.i.d. Exponential(λ). What is the MLE of the tail probability $P(X_i > a)$?

Consistency of the MLE

An estimator $\widehat{\theta}$ is consistent for θ if as the sample size n increases, the estimator converges to the true value of the parameter.

Formally, we would write: For any $\epsilon > 0$,

$$\lim_{n\to\infty} P(|\widehat{\theta} - \theta_0| > \epsilon) = 0$$

This is a fairly low standard placed on an estimator, and, fortunately, under certain "regularity conditions," the MLE is consistent.

Asymptotic Normality of the MLE

Another great property of the MLE: The MLE is asymptotically normal, with variance that is not difficult to calculate, under certain "regularity conditions." (See page 516 from Casella and Berger, Second Edition.)

This will allow an easy route to:

1. Calculate standard errors for MLEs
2. Constructing confidence intervals for parameters
3. Constructing confidence intervals for general functions of parameters (via the Delta method)
4. Constructing hypothesis tests for parameters

To start, we focus on the case where the parameter θ is one-dimensional.

The formal result can be stated as follows:

If $X_1, X_2, \ldots, X_n$ are i.i.d. $f(x; \theta_0)$, then, under suitable regularity conditions, as n increases,

$$\sqrt{n}\left(\widehat{\theta} - \theta_0\right) \longrightarrow N\big(0, I^{-1}(\theta_0)\big)$$

where

$$I(\theta_0) = E\left(-\frac{\partial^2 \log f(X; \theta)}{\partial \theta^2}\right)$$

is the Fisher Information.

Exercise: Explain the practical value of the above result.

Exercise: Show that

$$nI(\theta) = E\left[-\frac{\partial^2}{\partial\theta^2}\log L(\theta)\right]$$

The previous result shows that the variance of the MLE is approximately equal to the reciprocal of the expected value of the negative second derivative of the log likelihood at its peak.

And, “the negative of the second derivative” is the amount of curvature at the peak of the likelihood.

If the log likelihood has a “sharp” peak, then $nI(\theta)$ will be large, and the variance of $\widehat{\theta}$ will be small.

Exercise: Assume $X_1, X_2, \ldots, X_n$ are i.i.d. $N(\mu, \sigma^2)$, where σ^2 is known. We know that $\overline{X}$ is the MLE for μ. Derive the Fisher Information for μ, and use it to approximate the standard error and distribution of $\overline{X}$.

Exercise: Assume that $X_1, X_2, \ldots, X_n$ are i.i.d. Exponential(λ). Derive the Fisher Information for λ and the approximate distribution of its MLE.

Exercise: Assume that $X_1, X_2, \ldots, X_n$ are i.i.d. Poisson(λ). Derive the Fisher Information for λ and the approximate distribution of its MLE.

Efficiency of the MLE

It is possible to show that the lower bound on the variance of any unbiased estimator is $1/nI(\theta_0)$. This is called the Cramer-Rao lower bound.

Since the MLE is asymptotically unbiased, we know that we are achieving the Cramer-Rao lower bound, at least asymptotically.

Hence, we are minimizing the MSE (asymptotically). This is objectively the best rationale for utilizing maximum likelihood estimators.

The Delta Method

Often, our objective is not to estimate θ, but instead some function of θ, call it $g(\theta)$. For example, often want to estimate the probability of some event A. This probability is not a parameter, but it is a function of θ.

From the invariance property we know that the MLE of $g(\theta)$ is $g(\widehat{\theta})$. But, we also need to calculate a SE or construct a confidence interval for this estimate.

One approach would be to "reparameterize" the model in terms of $g(\theta)$, and then find $I(g(\theta))$, and then use $1/I(g(\theta))$ as the variance, etc. Fortunately, there is an easier way…

Recall the Delta Method:

> If Y_n is approximately $N(\mu, \sigma_n^2)$ and $\sigma_n^2 \to 0$ as n increases, then $g(Y_n)$ is approximately $N(g(\mu), (g'(\mu))^2\sigma_n^2)$, assuming that $g'(\mu)$ exists and is not zero.

Here we will use this result with the MLE for θ in place of Y_n.

Then, $\mu = \theta_0$ and $\sigma_n^2 = 1/nI(\theta_0)$.

We can conclude that $g(\hat{\theta})$ is approximately

$$N\left(g(\theta_0),\ g'(\theta_0)^2\left(\frac{1}{nI(\theta_0)}\right)\right)$$

In practice, we use the approximate variance:

$$g'(\hat{\theta})^2\left(\frac{1}{nI(\hat{\theta})}\right)$$

Exercise: Suppose that $X_1, X_2, \ldots, X_n$ are i.i.d. Exponential(λ). We seek to estimate $\tau = P(X_i > a)$ for some constant $a > 0$. Construct the MLE and find its standard error. Also construct a confidence interval for τ.

The Multidimensional Case

Now we consider the case where $\boldsymbol{\theta} = (\theta_1, \theta_2, \ldots, \theta_p)$.

We define the Fisher Information matrix to be the p by p matrix whose (i, j) entry is

$$I_{ij}(\boldsymbol{\theta}) = E\left[-\frac{\partial^2}{\partial\theta_i\partial\theta_j} \log f(x; \boldsymbol{\theta})\right]$$

The Main Result: Under appropriate "regularity conditions," if $X_1, X_2, \ldots, X_n$ are i.i.d. $f(x; \boldsymbol{\theta})$ then

$$\sqrt{n}(\widehat{\boldsymbol{\theta}} - \boldsymbol{\theta}_0) \longrightarrow N_p\big(0, \mathbf{I}^{-1}(\boldsymbol{\theta}_0)\big)$$

Recall that N_p denotes the p-dimensional multivariate normal distribution.

Practical Version: Under "regularity conditions," $\widehat{\boldsymbol{\theta}}$ is approximately normal with variance $\mathbf{I}^{-1}(\widehat{\boldsymbol{\theta}})/n$

Exercise: Suppose that $X_1, X_2, \ldots, X_n$ are i.i.d. $N(\mu, \sigma^2)$. Derive the joint distribution of the MLE for $\boldsymbol{\theta} = (\mu, \sigma^2)$.

Multivariate Delta Method

We can extend the Delta method to cases where the parameter is multidimensional.

In general, suppose that $\mathbf{Y}_n$ is approximately $N_p(\boldsymbol{\mu}, \boldsymbol{\Sigma}/n)$. Now consider an m-dimensional function $\mathbf{g}()$ where $m \leq p$:

$$\mathbf{g}(\mathbf{Y}_n) = \begin{bmatrix} g_1(\mathbf{Y}_n) \\ g_2(\mathbf{Y}_n) \\ \vdots \\ g_m(\mathbf{Y}_n) \end{bmatrix}$$

Then, $\mathbf{g}(\mathbf{Y}_n)$ is approximately $N_m(\mathbf{g}(\boldsymbol{\mu}), \mathbf{G}\boldsymbol{\Sigma}\mathbf{G}^T/n)$ where the (i, j) entry of $\mathbf{G}$ is $\partial g_i(\mathbf{y})/\partial y_j$. This all assumes that $\mathbf{G}$ is of full rank.

Exercise: Place our parameter estimation problem into the framework of the above statement of the Delta method.

Case Study: Geometric Brownian Motion

This Part concludes with a simple, but important, example of how careful derivation of the MLE and its distribution can lead to useful insight.

We begin by defining some standard concepts regarding Brownian motion.

The Standard Brownian Motion (Wiener Process)

A stochastic process $\{W(t) : t \geq 0\}$ is a (standard) Brownian motion if the following hold:

1. $W(0) = 0$. (almost surely)
2. $W(t)$ is continuous as a function of t. (almost surely)
3. $W(t_4) - W(t_3)$ and $W(t_2) - W(t_1)$ are independent for $t_1 \leq t_2 \leq t_3 \leq t_4$.
4. The distribution of $W(t_2) - W(t_1)$ depends only on $|t_1 - t_2|$ for t_1, t_2.
5. $W(t)$ is Normal$(0, t)$ for all t.

Brownian Motion with Drift and Scaling

A stochastic process $\{B(t) : t \geq 0\}$ is a Brownian Motion with drift μ and scaling σ^2 if $\{W(t) : t \geq 0\}$ is a standard Brownian motion and

$$B(t) = \mu t + \sigma W(t).$$

Note that this implies the following:

1. $B(0) = 0$. (almost surely)
2. $B(t)$ is continuous as a function of t. (almost surely)
3. $B(t_4) - B(t_3)$ and $B(t_2) - B(t_1)$ are independent for $t_1 \leq t_2 \leq t_3 \leq t_4$.
4. The distribution of $B(t_2) - B(t_1)$ depends only on $|t_1 - t_2|$ for t_1, t_2.
5. $B(t)$ is Normal$(\mu t, \sigma^2 t)$ for all t.

Geometric Brownian Motion

A stochastic process $\{S(t) \colon t \geq 0\}$ is a geometric Brownian motion if $\{B(t) \colon t \geq 0\}$ is a Brownian motion with drift μ and scaling σ^2 and

$$S(t) = S(0)\, e^{B(t)}$$

where $S(0) > 0$ is the initial value. Define $\alpha = \mu + \sigma^2/2$.

Note that this implies the following:

1. $S(t)$ is continuous as a function of t. (almost surely)
2. $S(t_4)/S(t_3)$ and $S(t_2)/S(t_1)$ are independent for any $t_1 \leq t_2 \leq t_3 \leq t_4$.
3. The distribution of $S(t_2)/S(t_1)$ depends only on $|t_1 - t_2|$ for any t_1, t_2.
4. $S(t)$ is Lognormal$(\log(S(0)) + \mu t, \sigma^2 t)$ for all t.
5. $S(t)$ has expected value $S(0)e^{\alpha t}$.
6. $\log(S(t_2)/S(t_1))$ is Normal$(\mu(t_2 - t_1), \sigma^2(t_2 - t_1))$ for all $t_1 \leq t_2$.

Let's consider the geometric Brownian motion model. Can we use data (observed prices) to estimate α and σ^2?

Suppose we observe the stock price over the period $[0, T]$. We will model the price as a geometric Brownian motion $\{S(t) : t \geq 0\}$.

Divide the time interval into n equal-width subintervals, with $\Delta = T/n$.

Define, for $i = 1, 2, \ldots, n$,

$$Y_i = \log\left(\frac{S(t_{i-1} + \Delta)}{S(t_{i-1})}\right)$$

where $t_i = i/n$, the times at which the price is observed.

From Property 6 of a geometric Brownian motion, $Y_1, Y_2, \ldots, Y_n$ are i.i.d. $N(\Delta\mu, \Delta\sigma^2)$.

Exercise: Find the MLE of $\boldsymbol{\theta} = (\alpha, \sigma^2)$. Approximate the distribution of $\boldsymbol{\theta}$.

Exercise: What does this result suggest regarding how large n should be chosen for the best estimation of σ^2? What about for α?

Is it, in fact, a good idea to sample more frequently in order to estimate σ^2? See Ait-Sahalia, Mykland, and Zhang (2005), "How Often to Sample a Continuous-Time Process in the Presence of Market Microstructure Noise," *The Review of Financial Studies*, Volume 18.

Part 6: Simple Linear Regression Theory

This lecture provides an overview of the most basic regression model, the **simple linear regression model**. There is a single **covariate/predcitor** x used to predict the **response** Y. This model will serve as a building block for the more complex (and more useful) models we will present afterwards.

The simple linear regression model is typically written using notation

$$Y_i = \beta_0 + \beta_1 x_i + \epsilon_i, \quad i = 1, 2, \ldots, n$$

where β_0 is the **intercept** and β_1 is the **slope** for the regression line.

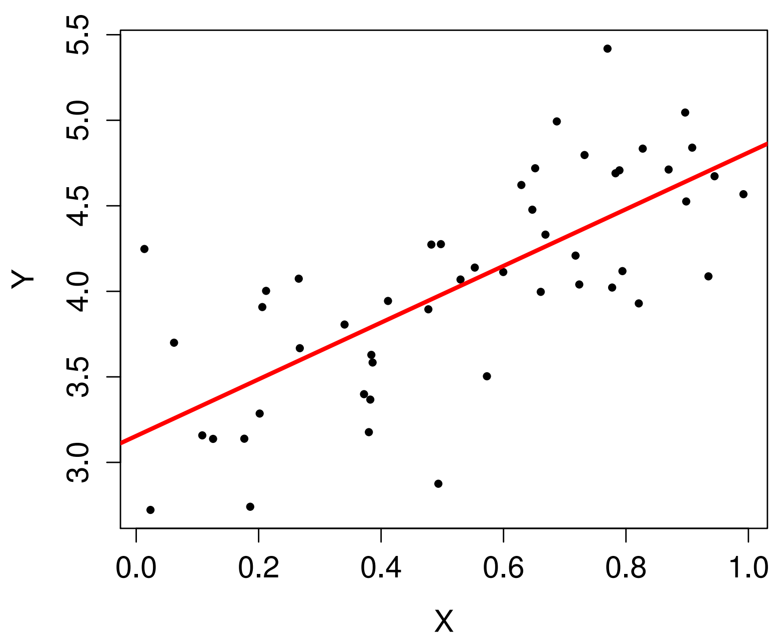


Figure 1: Synthetic scatter plot with $n = 50$ and a fitted regression line, $Y = 3.15 + 1.65x$.

The ϵ_i are random variables capturing the **irreducible error**, the scatter around the regression line.

The exact assumptions placed on the ϵ_i will vary depending on the situation, but the base assumptions that are almost always associated with simple linear regression are as follows:

1. $E(\epsilon_i) = 0$ for all i
2. $V(\epsilon_i) = \sigma^2$ for all i
3. The ϵ_i are uncorrelated, i.e., $\mathrm{Cov}(\epsilon_i, \epsilon_j) = 0$ for all $i \neq j$.

Additional assumptions, **if justified**, lead to stronger results. In particular, one often assumes that the ϵ_i are normally distributed.

Example: This model is sometimes written as

$$Y = \alpha + \beta x + \epsilon.$$

This notation is the origin for references to the "beta" for a stock.

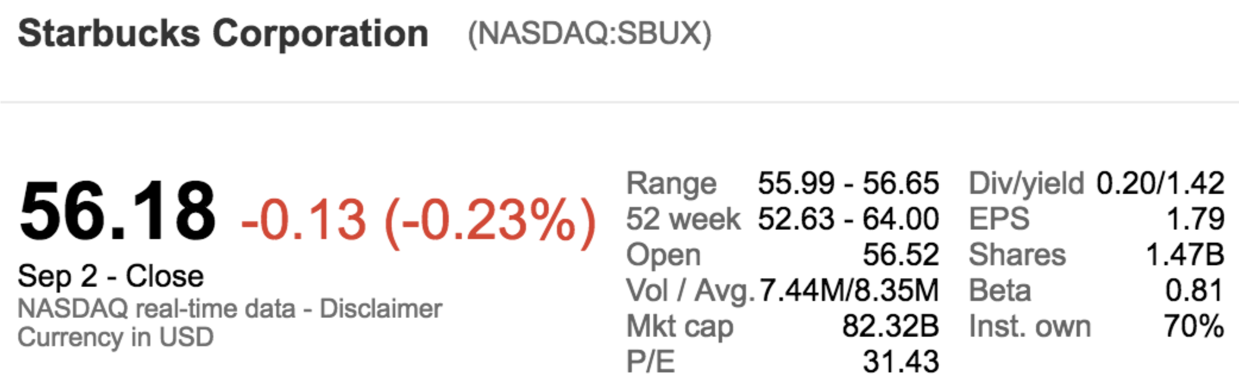


Figure 2: Screen shot from Google Finance, taken September 3, 2016.

In particular, the "beta" is found my performing a simple linear regression where the response is the excess returns for that equity, and the predictor is the excess returns for the market (as measured by some standard index such as S & P 500).

Figure 3 shows an example of the use of simple linear regression with the objective of calculating the "beta" for Google (GOOG), using daily market from 2014.

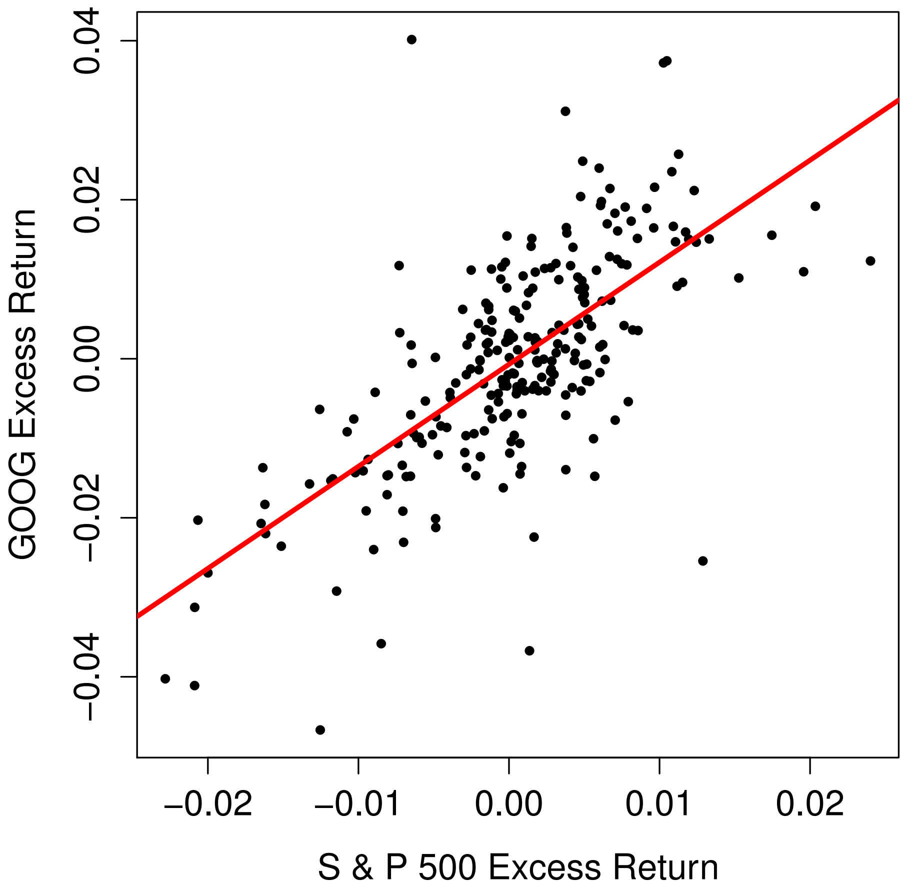


Figure 3: Daily excess returns for GOOG versus daily excess returns for S & P 500 for 2014.

Estimation via a Sample

In practice, we rely on a sample of n pairs (x_i, y_i) for $i = 1, 2, \ldots, n$, assumed to be drawn from a larger population. These will be used to estimate β_0, β_1 and σ^2. The standard notation for these estimates will be $\widehat{\beta}_0$, $\widehat{\beta}_1$, and $\widehat{\sigma}^2$.

The sample is naturally depicted in a **scatter plot**, with example shown in Figure 4.

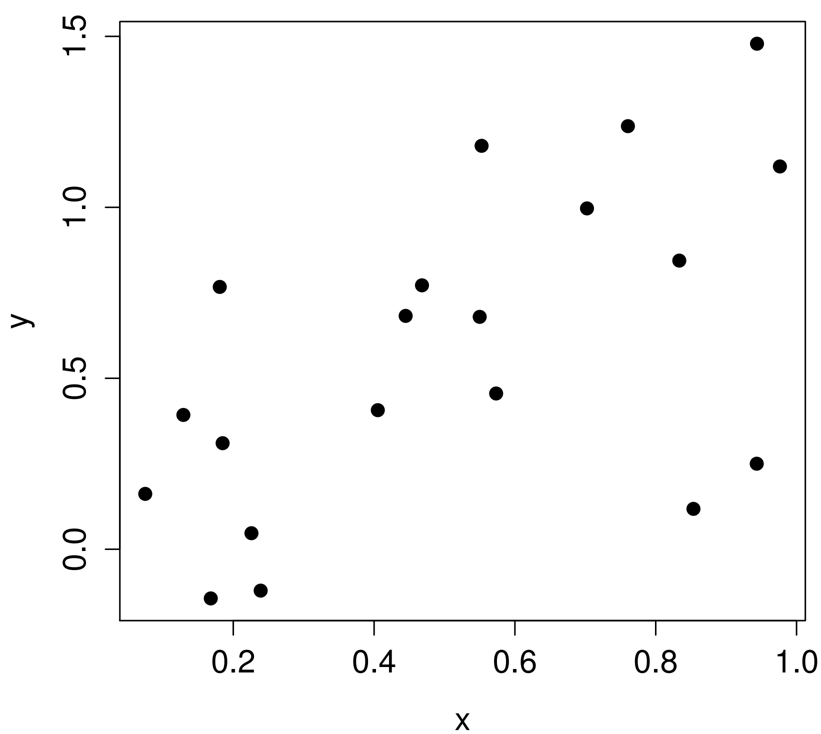


Figure 4: A simple scatter plot, with $n = 20$.

Some additional notation we should establish immediately: For any estimated regression line, the **fitted values** are

$$\widehat{y}_i = \widehat{\beta}_0 + \widehat{\beta}_1 x_i, \quad i = 1, 2, \ldots, n$$

and the **residuals** are

$$\widehat{\epsilon}_i = y_i - \widehat{y}_i, \quad i = 1, 2, \ldots, n.$$

Exercise: Use the scatter plot below, with a fit regression line superimposed, to demonstrate the fitted values and the residuals.

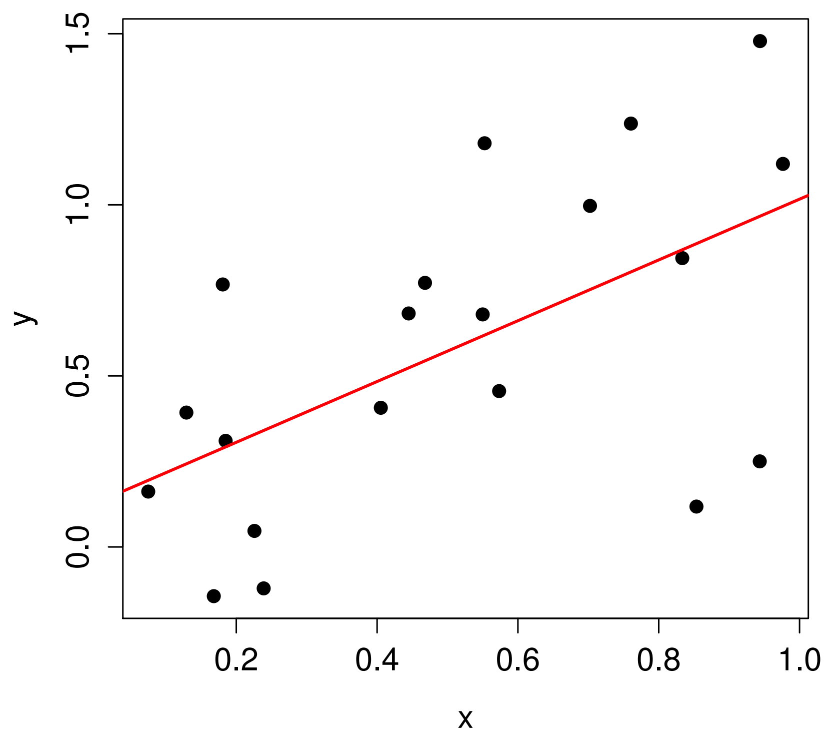


Figure 5: A simple scatter plot, with $n = 20$, with a regression line fit shown.

The Sample Covariance and Correlation

⋆ **Exercise:** Contrast covariance and correlation.

Recall that the covariance (and a related quantity, correlation) are measures of the **linear** association between two random variables X and Y:

$$\mathrm{Cov}(X,Y) = \mathrm{E}[(X - \mathrm{E}(X))(Y - \mathrm{E}(Y))]$$

and

$$\mathrm{Corr}(X,Y) = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y}$$

Not surprisingly, sample versions of these quantities are relevant to simple linear regression.

First, the **sample covariance** between observations of n pairs (x_i, y_i):

$$s_{xy} = \left(\frac{1}{n-1}\right) \sum_{i=1}^{n} (x_i - \overline{x})(y_i - \overline{y})$$

Note the similarity between this and the usual sample variance:

$$s_x^2 = \left(\frac{1}{n-1}\right) \sum_{i=1}^{n} (x_i - \overline{x})^2$$

Then, the **sample correlation** is

$$r_{xy} = \frac{s_{xy}}{s_x s_y}.$$

This quantity is typically just denoted with r. Figure 6 shows some examples of scatter plots, and the value of r for each. It is useful for building some intuition as to what different values of r mean.

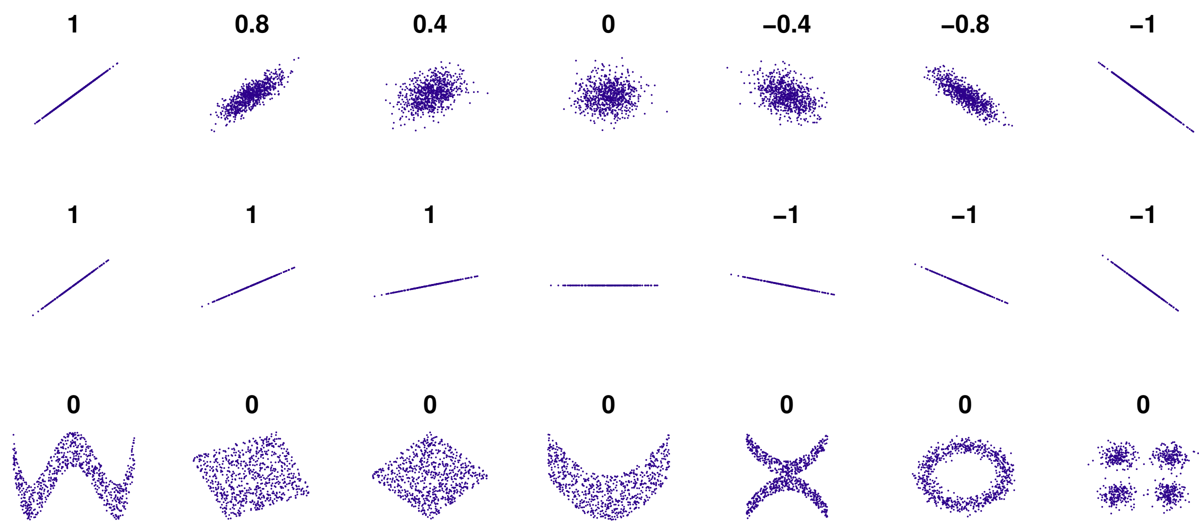
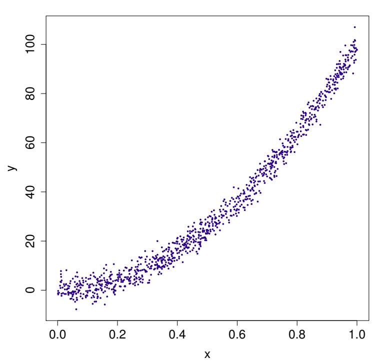


Figure 6: Examples of scatter plots, and corresponding values for r.

Exercise: Consider the scatter plot below. What, approximately, is the sample correlation?



The Parameter Estimates

⋆ **Exercise:** What route would you use to derive $\widehat{\beta}_0$ and $\widehat{\beta}_1$?

If we were to follow through with this, we would find that the optimal estimates would result from minimizing the **residual sum of squares (RSS)**:

$$\mathrm{RSS}(\beta_0, \beta_1) = \sum_{i=1}^{n} \left(y_i - (\beta_0 + \beta_1 x_i)\right)^2$$

And a little bit of calculus would show that

$$\widehat{\beta}_1 = \frac{\sum_i \left(y_i - \overline{y}\right)\left(x_i - \overline{x}\right)}{\sum_i \left(x_i - \overline{x}\right)^2} = \frac{s_{xy}}{s_x^2} = r_{xy}\left(\frac{s_y}{s_x}\right)$$

and

$$\widehat{\beta}_0 = \overline{y} - \widehat{\beta}_1 \overline{x}.$$

⋆ **Exercise:** Why do we use **least squares** to estimate the parameters in regression?

⋆ **Exercise:** Suppose that the roles of the predictor and response variables were interchanged in this regression model. What would be the effect on the slope of the regression line?

Figure 7 illustrates this idea. On the same scatter plot, two regression lines are shown: The "dashed-dotted" line is the regression line for predicting GOOG excess return from the S&P 500 excess return. The "dashed" line is the regression line for predicting S&P 500 excess return from GOOG excess return.

The solid line is the **SD Line.** This has a slope equal to s_y/s_x.

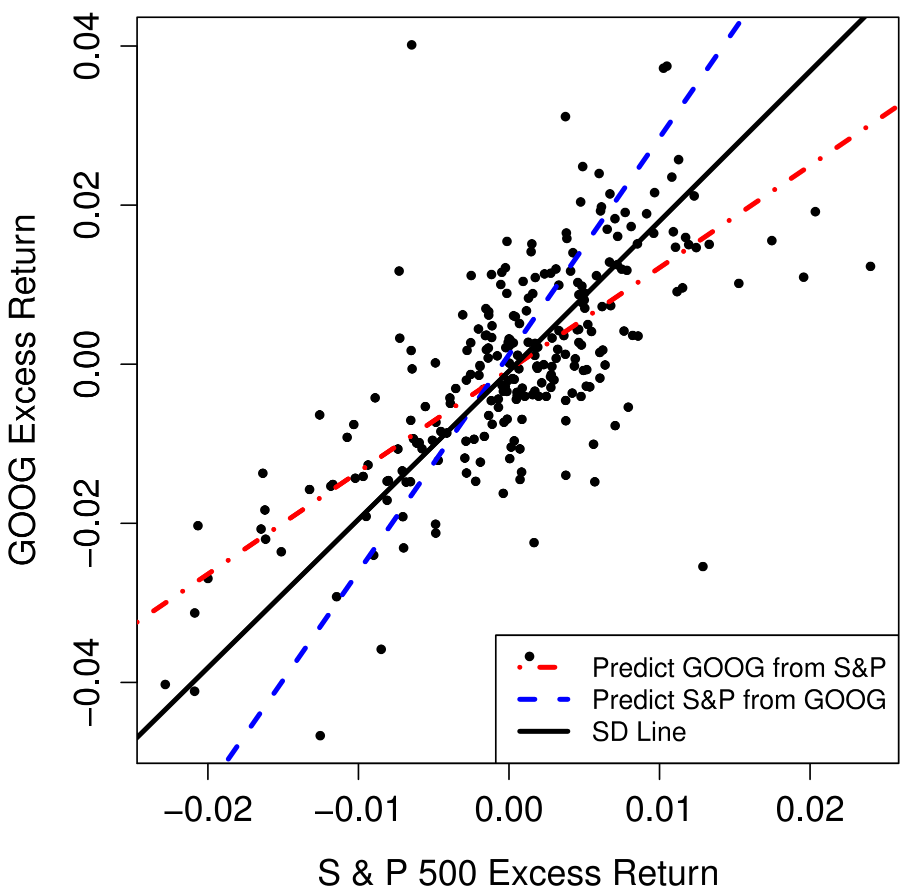


Figure 7: Daily excess returns for GOOG versus daily excess returns for S & P 500 for Jan. to Nov. 2012.

Estimating σ^2

The MLE for σ^2 can be derived to be

$$\widehat{\sigma}^2_{\text{MLE}} = \left(\frac{1}{n}\right) \sum_{i=1}^{n} (y_i - \widehat{y}_i)^2 = \text{RSS}\Big/ n.$$

This estimator is biased, but it can be adjusted to obtain an unbiased estimator

$$\widehat{\sigma}^2 = \left(\frac{n}{n-2}\right) \sigma^2_{\text{MLE}} = \left(\frac{1}{n-2}\right) \sum_{i=1}^{n} (y_i - \widehat{y}_i)^2 = \text{RSS}\Big/ (n-2).$$

This is the estimator we will typically use, and what is reported by R and other software packages.

Exercise: Is there intuition as to why we divide by $(n-2)$?

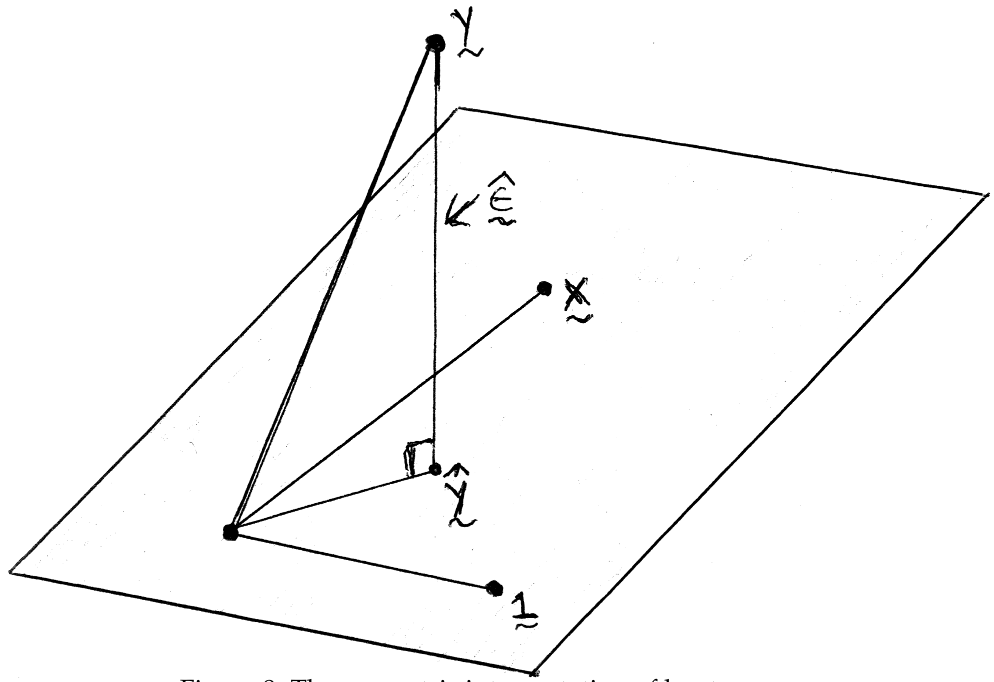


Figure 8: The geometric interpretation of least squares.

Figure 8 is a very useful geometric depiction of what happens when we do least squares. The vector of fitted values $\hat{\mathbf{y}}$ is the orthogonal projection of $\mathbf{y}$ onto the space spanned by the vectors $\mathbf{1}$ and $\mathbf{x}$. (The rectangle represents all of the possible vectors that can be formed from linear combinations of $\mathbf{1}$ and $\mathbf{x}$.)

⋆ **Exercise:** What is the sum of the residuals in OLS?

Inference Regarding β_1

Within the context of this simple model, the primary objective is to perform inference regarding β_1, since it captures information regarding the relationship (or lack thereof) between the response and the single predictor.

Here are some key results in this direction. Keep in mind that these are all true under the assumptions put forth for the simple linear regression model.

- $\widehat{\beta}_1$ is an unbiased estimator of β_1.
- The variance of $\widehat{\beta}_1$ is
$$V(\widehat{\beta}_1) = \frac{\sigma^2}{(n-1)s_x^2} \approx \frac{\widehat{\sigma}^2}{(n-1)s_x^2}.$$
This means that the standard error is
$$\mathrm{SE}(\widehat{\beta}_1) \approx \frac{\widehat{\sigma}}{s_x\sqrt{(n-1)}}.$$
- Suppose that the true value of the slope is β_{10}. Then
$$T = \frac{\widehat{\beta}_1 - \beta_{10}}{\mathrm{SE}(\widehat{\beta}_1)}$$
has the t-distribution with $(n-2)$ degrees of freedom. Of course, when n is sufficiently large, T will be approximately standard normal.

Exercise: Construct a $100(1-\alpha)\%$ confidence interval for β_1.

A common (overused) objective is to test for evidence if one can claim that $\beta_1 \neq 0$. It is important to be mindful of the following fact: If the confidence interval for β_1 includes zero, this is often not because $\beta_1 = 0$, but instead because the standard error for $\widehat{\beta}_1$ is too large to detect the deviation from zero.

⋆ **Exercise:** How can the standard error of $\widehat{\beta}_1$ be decreased?

One of the most common mistakes made when constructing models is to blindly remove a predictor from the model solely because there is not strong evidence that its cofficient does not equal zero. Model building (the process of deciding which covariates to include) should comprise many factors, including our knowledge of the situation, the importance that we are placing on constructing a **parsimonious** model, **and** the results of formal statistical inference procedures.

The Coefficient of Determination, R^2

When working with regression models it is very common to hear references to "the R^2 for the model."

This is also called the **coefficient of determination**.

The idea is simple: R^2 equals the proportion of the sum of squared response which is accounted for by the model that has been fit, relative to the model with no covariates.

In particular:

$$R^2 = 1 - \left[\sum_{i=1}^{n} (y_i - \widehat{y}_i)^2 \Big/ \sum_{i=1}^{n} (y_i - \overline{y})^2 \right]$$

Note that $0 \leq R^2 \leq 1$.

In the case of simple linear regression (a single predictor), R^2 does indeed equal the sample correlation (r) squared.

R^2 is overused. In particular, note that it is possible for R^2 to be close to one, but the model be a terrible fit. And it is possible for a great-fitting model to have R^2 close to zero. See Figure 9.

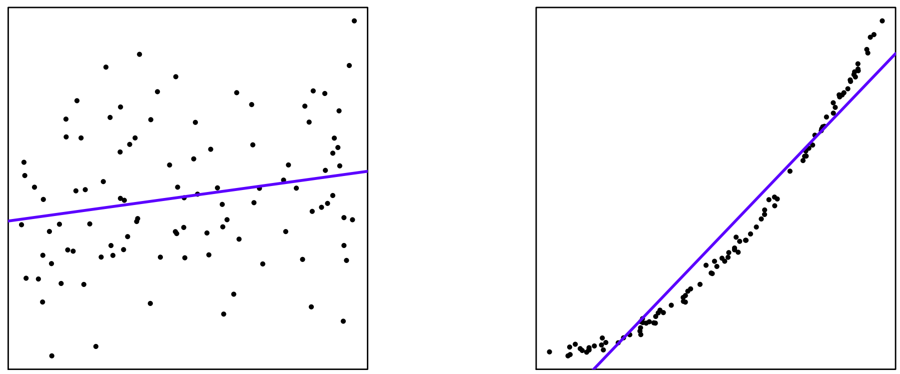


Figure 9: The scatter plot on the left has $R^2 \approx 0.05$, while that on the right has $R^2 \approx 0.95$. The data in the left plot were simulated from the normal linear model.

In short, R^2 should not be reiled upon as a diagnostic to judge the quality of the model fit. **If** the model is a good fit, **then** R^2 says something about the **predictive power** of the model.

⋆ **Exercise:** Suppose that, by mistake, each observed pair (response and predictor) is included twice in the analysis. What is the effect on $\widehat{\beta}_i$, the t-statistics, and R^2?

Part 7: Multiple Regression

In this part, we will extend from the simple linear regression model to **multiple regression**.

These models comprise the familiar linear models that take the form

$$Y = \beta_0 + \sum_{j=1}^{p} \beta_j x_j + \epsilon$$

where ϵ are assumed to be uncorrelated with mean zero and variance σ^2.

Assuming that the ϵ are normally distributed brings additional nice properties, but is generally not essential.

We will go through the fundamental steps of fitting such a model via an example.

Example: Factor Models

References to the "beta" for a stock derive from the slope of the regression line fit to a scatter plot where excess return for that equity is the response, and excess market return is the predictor. Models of this form are an important component of **Capital Asset Pricing Model (CAPM)**.

The term **Factor Model** is used to indicate any linear model for excess return on an equity. The predictors in the model are the **factors**.

Quoting Ruppert, examples of factors include

1. excess returns on the market portfolio beyond the risk-free return
2. growth rate of the GDP;
3. interest rate on short term Treasury bills or changes in this rate;
4. inflation rate or changes in this rate;
5. interest rate spreads;
6. return on some portfolio of stocks;
7. the difference between the returns on two portfolios.

The CAPM only includes the factor for excess market return.

The **Fama-French Three Factor Model** adds two factors.

The first is **small minus big (SMB)**, the difference in returns between portfolios of small and large stocks.

The second is **high minus low (HML)**, the difference in returns between portfolios of high and low book-to-market stocks.

Kenneth French maintains a web site with data on the three factors

http://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html

This Python function allows for easy access to the daily factor data.

```
In [15]: import pandas_datareader.data as web
         import datetime, re, copy

         def getFamaFrench(from_date, to_date):

             three_fac = web.DataReader(\
                 "F-F_Research_Data_Factors_daily",
                 "famafrench")
             f = copy.copy(three_fac[0])
             f.rename( columns={c:re.sub(r'[0-9\-\s]','',c)\
                 for c in f.columns}, inplace=True)

             datetime.datetime.strptime(from_date, "%Y-%m-%d")
             datetime.datetime.strptime(to_date, "%Y-%m-%d")

             out = f[(f.index > datetime.datetime.strptime\
                 (from_date, "%Y-%m-%d")) &
                     (f.index < datetime.datetime.strptime\
                 (to_date, "%Y-%m-%d"))]

             return out
```

An example, and the first few rows of the data returned are as follows. (These factors are periodically revised, and hence the exact numbers may be different at the present time. This will affect the subsequent analysis, as well.)

```
In [16]: ffhold = getFamaFrench(from_date="2016-1-1",
                               to_date="2016-6-30")

         ffhold.iloc[0:10,]
```

```
Out[16]:             MktRF   SMB   HML   RF
         Date
         2016-01-04  -1.59 -0.84  0.53  0.0
         2016-01-05   0.12 -0.22  0.02  0.0
         2016-01-06  -1.35 -0.12 -0.01  0.0
         2016-01-07  -2.44 -0.28  0.08  0.0
         2016-01-08  -1.11 -0.48 -0.03  0.0
         2016-01-11  -0.06 -0.64  0.34  0.0
         2016-01-12   0.72 -0.40 -0.80  0.0
         2016-01-13  -2.67 -0.70  0.79  0.0
         2016-01-14   1.65 -0.01 -0.39  0.0
         2016-01-15  -2.14  0.42 -0.21  0.0
```

Let's get the data for PNC for the same time period. The Python package `yfinance` makes this simple.

```
In [17]: import yfinance as yf

         PNC = yf.download("PNC", "2016-01-01",
                 "2016-06-30")
```

```
[*********************100%***********************]  1 of 1 down
```

Now find the excess returns for PNC. Note that the Fama-French data file includes the risk free rate, with name `RF`:

```
In [18]: ffhold['PNCexret'] = \
          100*PNC['Adj Close'].pct_change() - ffhold['RF']
```

The first row does not have a return, so remove it.

```
In [19]: ffhold = ffhold[1:]
```

The three factor model we fit is

$$\text{PNC.Ex.Ret.} = \beta_0 + \beta_1 \,\text{Mkt.Ex.Ret.} + \beta_2 \,\text{SMB} + \beta_3 \,\text{HML} + \epsilon$$

A Python command for fitting a linear regression is `ols` in package `statsmodels`.

```
In [20]: import statsmodels.api as sm

         ff3modPNC = sm.OLS.from_formula\
           (formula="PNCexret ~ MktRF + SMB + HML",
              data=ffhold).fit()
```

The key information from the fit of the model is stored in the object `ff3modPNC`. For example, to see the parameter estimates and their standard errors for the estimators:

```
In [21]: ff3modPNC.params

Out[21]: Intercept   -0.151925
         MktRF        1.215999
         SMB         -0.011464
         HML          0.798166
         dtype: float64

In [22]: ff3modPNC.bse

Out[22]: Intercept    0.070081
         MktRF        0.070531
         SMB          0.139419
         HML          0.120677
         dtype: float64
```

You can obtain the residuals from `ff3modPNC.resid` and the fitted values from `ff3modPNC.fittedvalues`.

Look at `ff3modPNC.summary()` for a more complete summary.

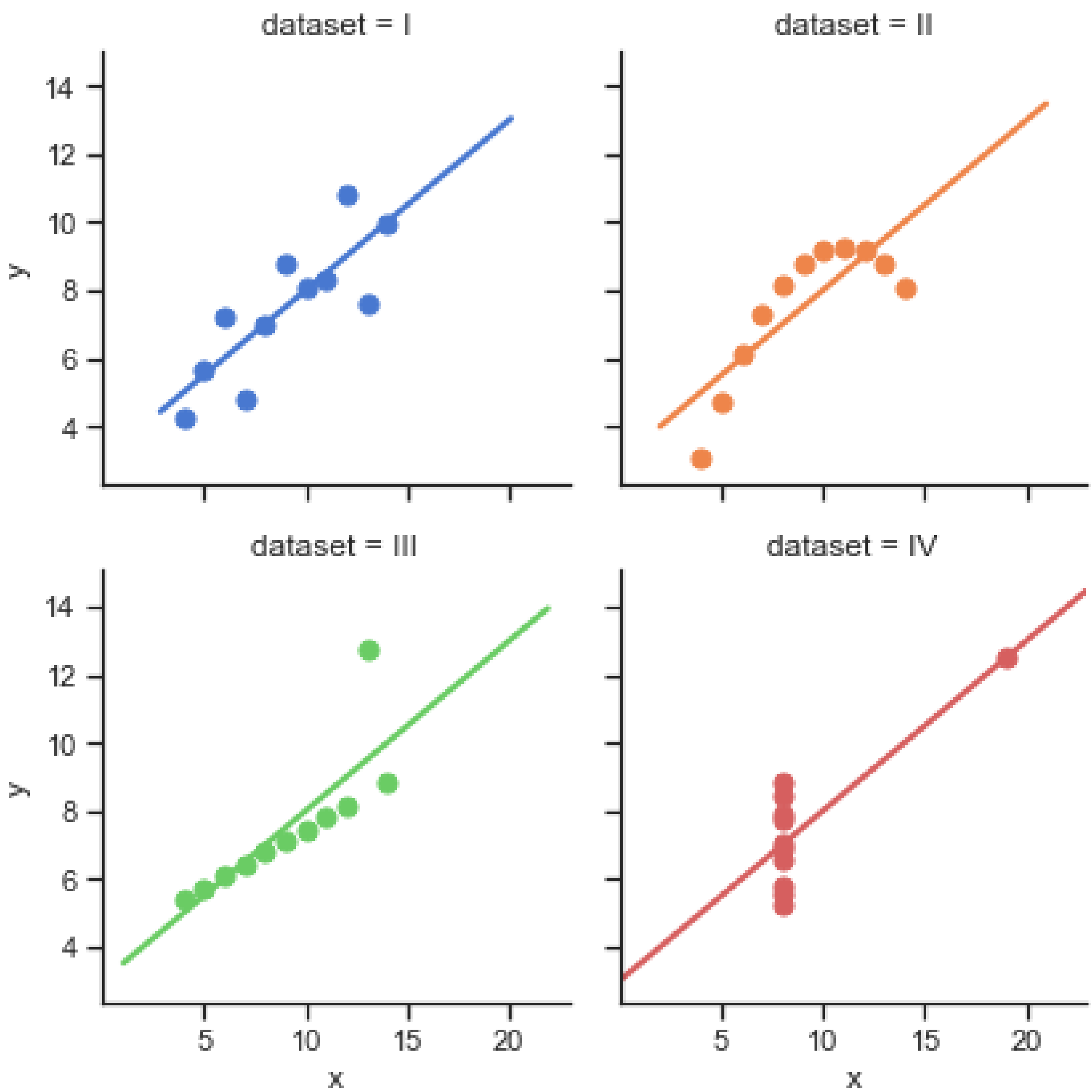
Diagnostic Plots

The figure above shows four scatter plots constructed by Francis Anscombe for a 1973 research article. These are called **Anscombe's Quartet**. This is, in part, based on https://seaborn.pydata.org/examples/anscombes_quartet.html.

In [23]:
```
import seaborn as sns
sns.set(style="ticks")

ansdf = sns.load_dataset("anscombe")

hold = sns.lmplot(x="x", y="y", col="dataset",
        hue="dataset", data=ansdf, col_wrap=2,
        ci=None, palette="muted", height=3,
        scatter_kws={"s": 50, "alpha": 1})
```


dataset = I
dataset = II
dataset = III
dataset = IV
y
x
4
6
8
10
12
14
5
10
15
20

Exercise: Find the sample mean and standard deviation for both x and y in all four cases, to find the correlation between x and y in each case, and then use that to find the slope of the regression line for each.

Exercise: This can be taken a step further: We could find that the residual sum of squares in all four cases is exactly the same. What does this imply about the inference for β_1 in all four cases?

Exercise: What is the message we take away from this example?

A very useful diagnostic plot is that of the **residuals versus the fitted values**. There are a wide range of models for which this is an invaluable tool.

Exercise: If the assumptions of the model are correct, what should we see in the plot of residuals versus fitted values?

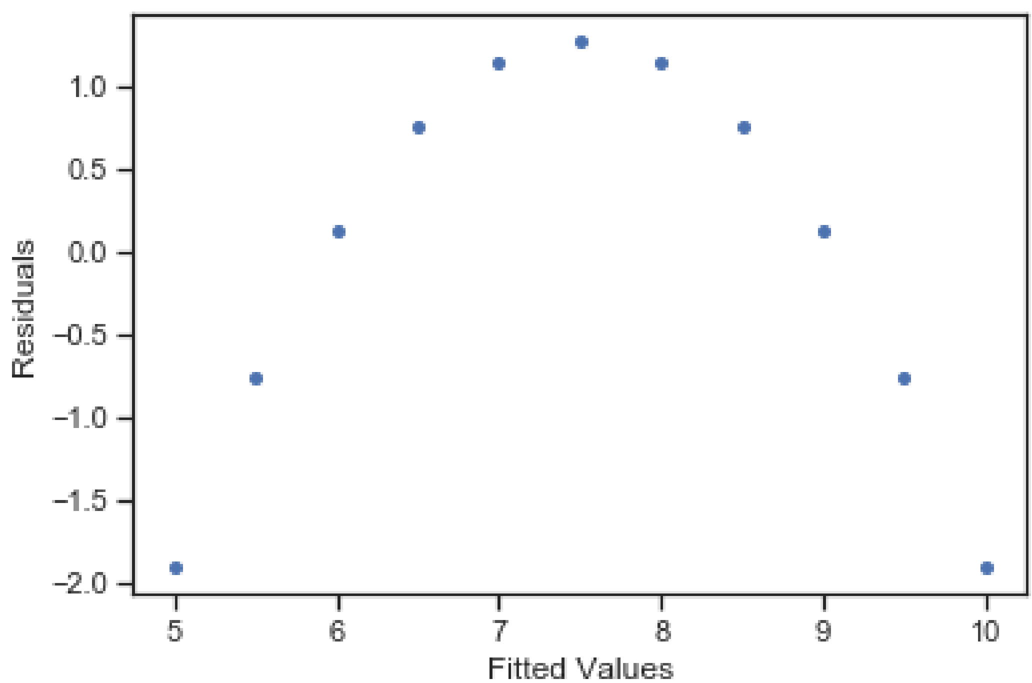
Exercise: Use the code below to explore the plot of residuals versus fitted values in the case of Anscombe's Quartet. Comment on the results.

```
In [24]: ansgrp = ansdf.groupby(ansdf.dataset)
         onegrp = ansgrp.get_group("II")

         ansfit = sm.OLS.from_formula\
           (formula="y~x", data=onegrp).fit()

         import matplotlib.pyplot as plt

         plt.scatter(ansfit.fittedvalues,
             ansfit.resid, s=16)
         plt.xlabel("Fitted Values")
         plt.ylabel("Residuals")
         plt.show()
```



Exercise: Your colleague says, "I found the correlation between the residuals and fitted values to be equal to zero, so there is clearly no relationship between the residuals and fitted values. So, the model must be fine." Do you agree with this logic?

The residuals are also often plotted against each of the predictors. Our assumption is that the irreducible error is "random scatter" around the regression line/plane. If this assumption is true, there should be no remaining pattern evident in these plots.

The Normal Probability Plot

We have already mentioned that when the irreducible errors can be assumed to be iid normal, then there are additional appealing results that can be stated:

1. The least squares estimator can be justified as being the maximum likelihood estimators.
2. The least squares estimators are exactly normal.
3. The least squares estimators, when appropriately scaled, have a t-distribution, which can be used to construct confidence intervals and hypothesis tests.

Overall, however, the value of this assumption is not that great. First, least squares can be motivated even if it does not lead to the MLE. Second, there are versions of the central limit theorem that state that, assuming the errors are independent (and some regularity conditions), the least squares estimator will be approximately normal for "large" n.

Still, it can be useful to assess the normality of the residuals, as a means of assessing the normality of the irreducible error. The standard tool for doing this is the **normal probability plot**, which is a particular version of a **quantile-quantile plot**, or **QQ plot**.

In a QQ plot, one compares the sorted sample values versus the corresponding percentiles of the distribution being tested. More specifically, the i^{th} order statistic is plotted versus the $100(i/(n+1)$ percentile of the distribution, for $i = 1, 2, \ldots, n$. If the sample was drawn from that distribution, then the plot should show (approximately) a 45-degree line.

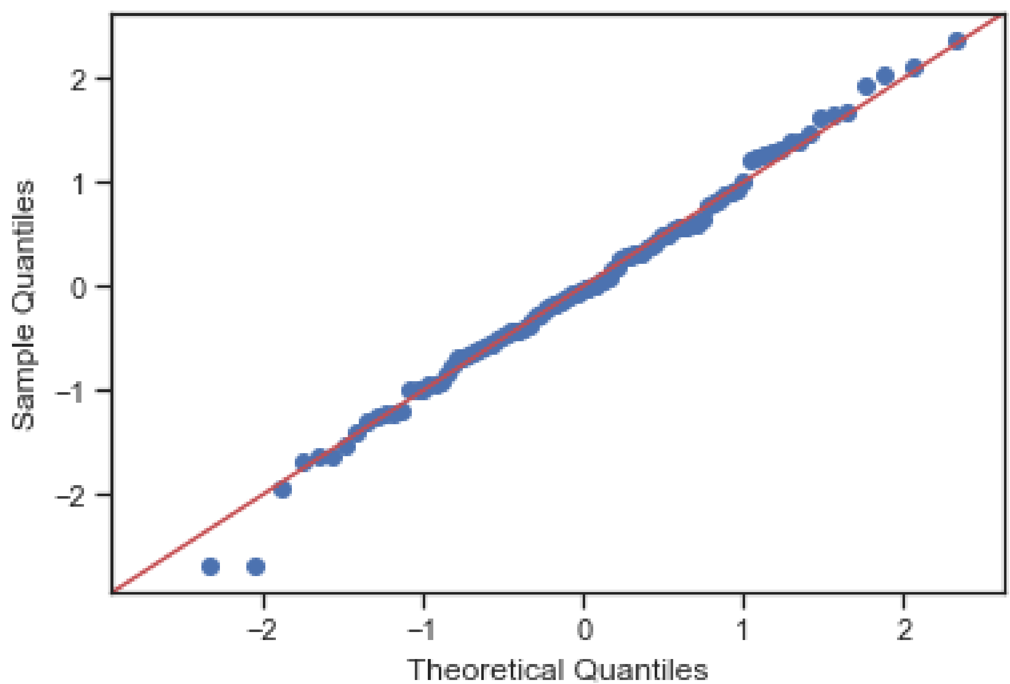
This may require the estimation of parameters of the distribution; if so, then use the best estimates available (e.g., MLE). Fortunately, in the normal case, this is unnecessary because a linear transformation of any normal is also normal. Hence, comparison with the standard normal is sufficient.

In Python, this plot can be constructed using `qqplot()` found in `statsmodels`. Note the use of the `Fit` argument to force Python to compare with the best fitting distribution within the specified family. (One can obtain fitted parameters using the `fit()` method.)

```
In [25]: from statsmodels.graphics.gofplots import qqplot
         import scipy.stats.distributions as spdists

         x = spdists.t.rvs(30,size=100)
         theodist = spdists.norm

         qqplot(x, dist=theodist, line="45", fit=True)
         plt.show()
```

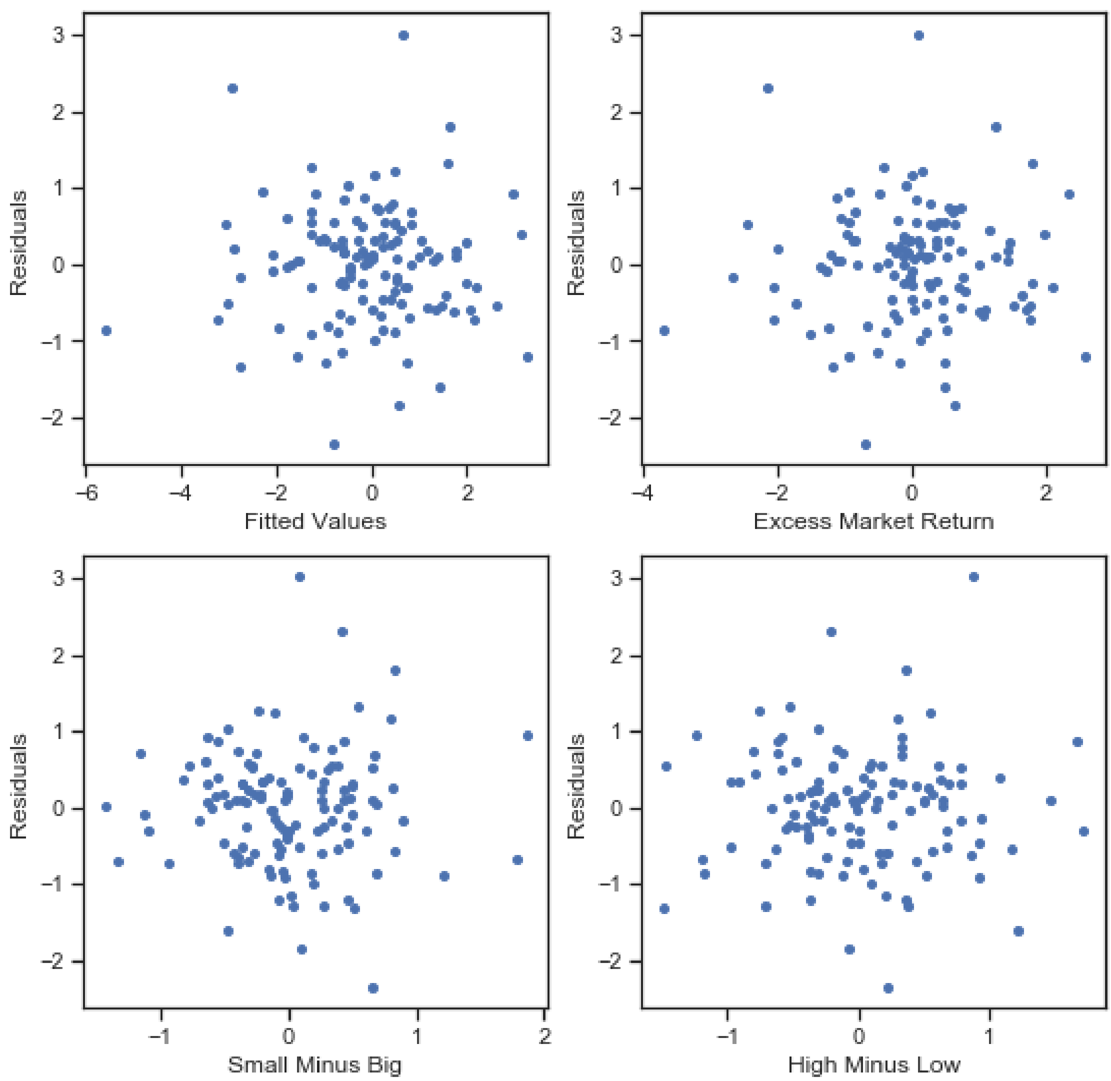


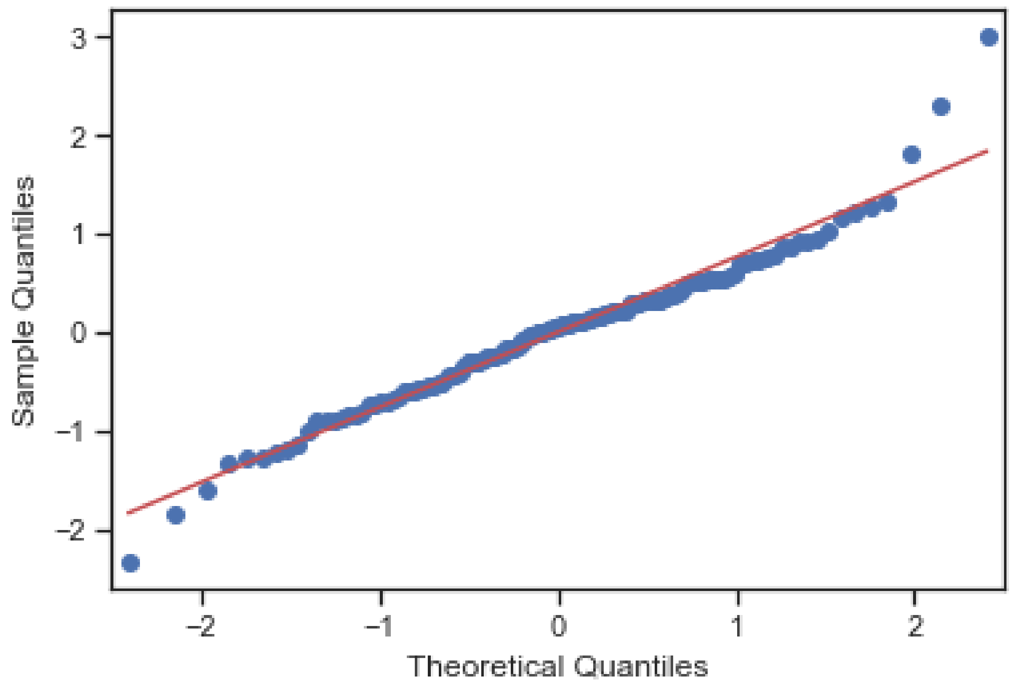
Exercise: Use the above code to try different examples and explore the use of this tool.

Exercise: Explain why checking the normal probability plot of the residuals may be a good idea.

Let's now return to our PNC example. The figure below shows three diagnostic plots:

1. Plot of residuals versus fitted values.
2. Plot of residuals versus the predictors.
3. Normal probability plot of the residuals.





Exercise: Comment on the quality of the fit.

Linear Regression in Matrix Notation

First, recall our simple linear regression model:

$$Y_i = \beta_0 + \beta_1 X_i + \epsilon_i, \quad i = 1, 2, \ldots, n$$

We can rewrite this model in matrix notation by letting

$$\mathbf{Y} = \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{bmatrix} \qquad \mathbf{X} = \begin{bmatrix} 1 & X_1 \\ 1 & X_2 \\ \vdots & \vdots \\ 1 & X_n \end{bmatrix} \qquad \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} \qquad \boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix}$$

and then noting that

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

When moving to multiple regression, the main change we must make is to add columns to the matrix $\mathbf{X}$, commonly called the **design matrix**, and add entries to $\boldsymbol{\beta}$:

$$\mathbf{X} = \begin{bmatrix} 1 & X_{11} & X_{12} & \cdots & X_{1,p} \\ 1 & X_{21} & X_{22} & \cdots & X_{2,p} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & X_{n1} & X_{n2} & \cdots & X_{n,p} \end{bmatrix} \qquad \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}$$

One can show that the least squares estimators (now a vector) are

$$\widehat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}.$$

and can now define the vector of fitted values and residuals as

$$\widehat{\mathbf{Y}} = \begin{bmatrix} \widehat{Y}_1 \\ \widehat{Y}_2 \\ \vdots \\ \widehat{Y}_n \end{bmatrix} \quad \text{and} \quad \widehat{\boldsymbol{\epsilon}} = \begin{bmatrix} \widehat{\epsilon}_1 \\ \widehat{\epsilon}_2 \\ \vdots \\ \widehat{\epsilon}_n \end{bmatrix}$$

and it follows that

$$\widehat{\mathbf{Y}} = \mathbf{X}\widehat{\boldsymbol{\beta}} \quad \text{and} \quad \widehat{\boldsymbol{\epsilon}} = \mathbf{Y} - \widehat{\mathbf{Y}}.$$

The Hat Matrix

We note that we could write

$$\widehat{\mathbf{Y}} = \mathbf{X}\widehat{\boldsymbol{\beta}} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y} = \mathbf{H}\mathbf{Y}$$

where

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$$

is called the **hat matrix** (because it puts a "hat" on $\mathbf{Y}$).

The diagonal entries of $\mathbf{H}$ are referred to as the **leverages**.

Comment: Mathematically, $\mathbf{H}$ is a **projection matrix**, as it "projects" the vector of observed responses $\mathbf{Y}$ onto the space of all vectors which are linear combinations of the columns of $\mathbf{X}$. In fact, we note that $\mathbf{H}$ is symmetric and **idempotent**, meaning that $\mathbf{H}\mathbf{H} = \mathbf{H}$.

Exercise: Show that $\mathbf{I} - \mathbf{H}$ is also symmetric and idempotent.

Additional Results

A range of important results can be proven:

1. The least squares estimator for $\boldsymbol{\beta}$ is $\widehat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$.
2. The variance of $\widehat{\boldsymbol{\beta}}$ is $\sigma^2(\mathbf{X}^T\mathbf{X})^{-1}$.
3. $\widehat{\mathbf{Y}} = \mathbf{X}\widehat{\boldsymbol{\beta}} = \mathbf{H}\mathbf{Y}$, where the (symmetric, idempotent) hat matrix $\mathbf{H}$ is defined as above.
4. $\widehat{\boldsymbol{\epsilon}} = \mathbf{Y} - \widehat{\mathbf{Y}} = (\mathbf{I} - \mathbf{H})\mathbf{Y}$.
5. If $\boldsymbol{\epsilon}$ is assumed normal with mean zero and variance $\sigma^2\,\mathbf{I}$, the following hold:
   - $\widehat{\boldsymbol{\beta}}$ is the maximum likelihood estimator.
   - $\mathbf{Y}$ is multivariate normal with mean $\mathbf{X}\boldsymbol{\beta}$ and covariance $\sigma^2\,\mathbf{I}$.
   - $\widehat{\boldsymbol{\beta}}$ is multivariate normal with mean $\boldsymbol{\beta}$ and covariance $\sigma^2(\mathbf{X}^T\mathbf{X})^{-1}$.
   - $\widehat{\mathbf{Y}}$ is multivariate normal with mean $\mathbf{X}\boldsymbol{\beta}$ and covariance $\sigma^2\,\mathbf{H}$.
   - $\widehat{\boldsymbol{\epsilon}}$ is multivariate normal with mean zero and covar. $\sigma^2(\mathbf{I} - \mathbf{H})$.

Degrees of Freedom

One key thing that does change when moving to multiple regression is the number of **degrees of freedom** in the residual vector.

Recall that we had seen previously that with simple linear regression, it held that

$$\sum_i \widehat{\epsilon}_i = 0 \text{ and } \sum_i \widehat{\epsilon}_i X_i = 0.$$

This result is one way of seeing why the residuals only have $n-2$ degrees of freedom in the case of simple linear regression: There are two constraints placed on the vector of residuals.

But, these facts could be written more compactly using matrix notation: $\mathbf{X}^T \widehat{\boldsymbol{\epsilon}} = \mathbf{0}$, where $\mathbf{0}$ is a vector consisting entirely of zeros.

In fact, this result extends to multiple regression: The vector of residuals $\widehat{\boldsymbol{\epsilon}}$ is orthogonal to **every column** of $\mathbf{X}$, i.e., $\mathbf{X}^T \widehat{\boldsymbol{\epsilon}} = \mathbf{0}$.

We now can state that there are $n - (p+1)$ **degrees of freedom in the residuals**, and hence need to update some of our previous results. (You will note that simple linear regression corresponds to the case where $p = 1$.)

In particular:

1. The unbiased estimator of σ^2 is
$$\widehat{\sigma}^2 = \text{RSS} \Big/ (n-p-1)$$
2. When the errors of i.i.d. normal, the statistic
$$\frac{\widehat{\beta}_i - \beta_i}{\widehat{\text{SE}}(\widehat{\beta}_i)}$$
has the t-distribution with $n-p-1$ degrees of freedom. Hence, a $100(1-\alpha)\%$ confidence interval for β_i is formed as
$$\widehat{\beta}_i \pm t_{\alpha/2, n-p-1}\, \widehat{\text{SE}}(\widehat{\beta}_i)$$
and hypothesis tests concerning β_i should compare the test statistic (the "t value") with the t-distribution with $n-p-1$ degrees of freedom.
3. For large n, we can appeal to the central limit theorem and construct an approximate $100(1-\alpha)\%$ confidence interval for β_i using
$$\widehat{\beta}_i \pm z_{\alpha/2}\, \widehat{\text{SE}}(\widehat{\beta}_i)$$

Basic Variable Selection

The process of **variable selection** in regression consists of making a decision among all of the possible subsets of covariates to find that model with the best ability to **generalize** to the population from which the training sample was drawn.

Various criterion have been proposed for comparing models.

A proven, general strategy for predictor selection is to compare the values of the **Akaike Information Criterion (AIC)** across different models. AIC is defined as

$$\text{AIC} = n \log(\text{RSS}) + 2p$$

where p is the number of β parameters in the model.

Exercise: Justify the form for AIC. How does it balance model fit with model complexity?

Technical Comment:

The actual value of AIC is not important, it is only the ordering of the models that it provides which is important.

Different texts and software packages use different definitions of AIC in the context of linear regression. In all cases, these expressions should be a monotonically increasing function of the definition given above. Hence, these definitions do not differ in ways that are of practical importance, i.e., the ordering of models that results will be the same.

The value of AIC reported by Python for the model fit above is as follows:

```
In [9]: ff3modPNC.aic

Out[9]: 292.7014744341992
```

while the value using the formula calculated above is

```
In [10]: import numpy as np

         ff3modPNC.nobs*np.log(sum(ff3modPNC.resid**2)) + 2*4

Out[10]: 535.5412709766572
```

These differ by the factor

$$n(\log(2\pi/n) + 1)$$

The reason for this difference will be discussed later when we use AIC in general situations.

Example (Section 11.3 in Ruppert and Matteson)

The **forward rate function** $r(t)$ returns the current forecast of bond interest rates at time t in the future. The current price of of a zero-coupon bond with maturity T can be found by discounting the face value of the bond by the forward rates "averaged" over the period until maturity:

$$P(T) = \text{Face Value of Bond} \times \exp\left(-\int_0^T r(t)\,dt\right)$$

(The face value of the bond is the amount to be paid to the holder at maturity.)

Simple derivation shows that

$$r(T) = -\frac{\partial}{\partial T}\log P(T).$$

Hence, we can estimate $r(t)$ using current zero-coupon bond prices.

In particular, consider a grid of observable maturity dates $T_0 < T_1 < \cdots < T_n$ and approximate the derivative of $\log P(T)$ at T_i using

$$\frac{\log(P(T_i)) - \log(P(T_{i-1}))}{T_i - T_{i-1}}$$

for $i = 1, 2, \ldots, n$.

This sets up a natural regression problem where the response is

$$Y_i = -\frac{\log(P(T_i)) - \log(P(T_{i-1}))}{T_i - T_{i-1}}$$

and our simple model assumes that

$$Y_i = r(T_i) + \epsilon_i$$

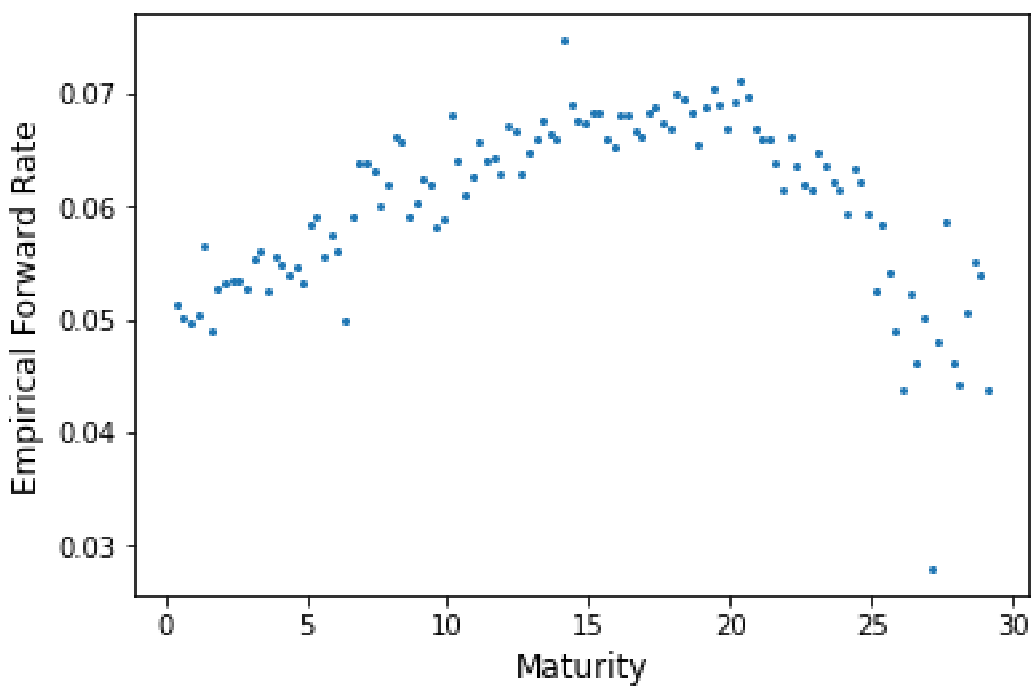
where the ϵ_i are iid, normally distributed, with mean zero. Hence, in this example we have a single covariate, time to maturity (T).

The data we have can be read in and visualized as follows:

```
In [11]: import pandas as pd

         forratedat = pd.read_csv(
         "http://www.stat.cmu.edu/~cschafer/MSCF/forratedat.txt
             sep=" ")

         fig, axs = plt.subplots(1, 1, figsize=[6,4])
         axs.plot(forratedat['maturity'],forratedat['emp'],
             marker='.', ms=4, linestyle="none")
         axs.set_xlabel("Maturity", size=12)
         axs.set_ylabel("Empirical Forward Rate", size=12)
         plt.show()
```



We will consider polynomial models of the form

$$Y_i = \sum_{j=0}^{p} \beta_j T_i^j + \epsilon_i$$

with a goal of finding the best subset of the p polynomial terms to include in the model. (As is usually the case, we do not consider removing the intercept β_0.)

The "full" model (largest possible model) we'll consider includes up to the ninth power.

We will **scale** maturity to have mean zero and variance one:

```
In [12]: forratedat['maturity'] = (forratedat['maturity'] - \
                    forratedat['maturity'].mean()) / \
                    (forratedat['maturity'].std())
```

Exercise: Why is scaling the predictor a good idea in this case?

Now, we can fit the full model:

```
In [13]: forratedat['maturity2'] = forratedat['maturity'] ** 2
         forratedat['maturity3'] = forratedat['maturity'] ** 3
         forratedat['maturity4'] = forratedat['maturity'] ** 4
         forratedat['maturity5'] = forratedat['maturity'] ** 5
         forratedat['maturity6'] = forratedat['maturity'] ** 6
         forratedat['maturity7'] = forratedat['maturity'] ** 7
         forratedat['maturity8'] = forratedat['maturity'] ** 8
         forratedat['maturity9'] = forratedat['maturity'] ** 9

         import statsmodels.formula.api as smf

         X_with_ones = sm.add_constant(
             forratedat.drop('emp', axis=1))
         fullmod = smf.OLS(endog=forratedat['emp'],
             exog=X_with_ones).fit()
```

```
/Users/chadschafer/anaconda3/lib/python3.7/site-packages/numpy/
  return ptp(axis=axis, out=out, **kwargs)
```

Exercise: Inspect `fullmod.summary()` and comment on the parameter estimates. Do you think that this model actually needs this many terms, or are we at risk of overfitting?

Exhaustive Search

In cases where the number of predictors is small, an exhaustive search over all possible models is possible.

Exercise: If a model has p candidate predictors, how many models are compared in the exhaustive search?

```
In [14]: import itertools
         from tqdm import tnrange, tqdm_notebook

         X = forratedat.drop('emp', axis=1)
         df = {}

         for k in tnrange(1, len(X.columns)+1):
             for combo in \
               itertools.combinations(X.columns, k):
                 X_with_ones = sm.add_constant(X[list(combo)])
                 model = smf.OLS(endog=forratedat['emp'],
                         exog=X_with_ones).fit()
                 df[str(combo)] = model.aic

         df = pd.Series(df)
```

```
HBox(children=(IntProgress(value=0, max=9), HTML(value='')))
```

Exercise: Parse the Python code in the previous example.

If you look at `df.idxmin()`, we see that the AIC-optimal model is

$$Y_i = \beta_0 + \beta_1 T_i + \beta_4 T_i^4 + \beta_5 T_i^5 + \beta_6 T_i^6 + \beta_7 T_i^7 + \epsilon_i$$

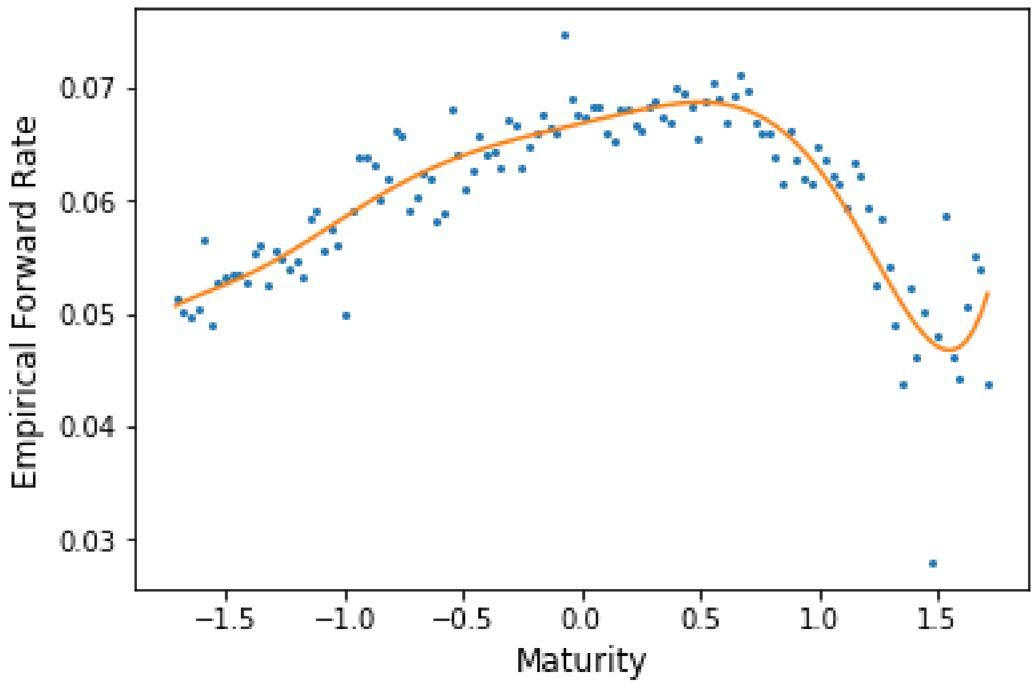
Let's fit that model.

Exercise: Which fit has a larger value of R^2, the AIC-optimal model, or the full model?

```
In [15]: X_with_ones = \
            sm.add_constant(X.iloc[:,[0,3,4,5,6]])

         AICoptmodel = smf.OLS(\
             endog=forratedat['emp'],
             exog=X_with_ones).fit()

         fig, axs = plt.subplots(1, 1, figsize=[6,4])
         axs.plot(forratedat['maturity'],forratedat['emp'],
             marker='.', ms=4, linestyle="none")
         axs.set_xlabel("Maturity", size=12)
         axs.set_ylabel("Empirical Forward Rate", size=12)
         axs.plot(forratedat['maturity'],
             AICoptmodel.fittedvalues,ms=10)
         plt.show()
```



Exercise: Comment on the quality of the fit of the model.

Stepwise Regression

An exhaustive search is not feasible when there is a large number of predictors.

A classic strategy for dealing with the large number of possible models is to use a **stepwise variable selection** procedure.

With this method, one (typically) starts with the largest model under consideration, and then takes a sequence of **steps**: At each step one predictor is either **added** or **dropped**, depending on if a **forward** or **backward** selection procedure is used, respecitvely.

The decision for which step to take (i.e., which covariate to add or drop) is based on AIC: You take the step which reduces AIC the most.

Once AIC no longer changes, the process stops, and you have your final model.

This algorithm is far from perfect, but is useful in the appropriate situations.

```
In [16]: features_chosen = []
         remaining_features = list(X.columns.values)
         best_AIC = np.inf

         for i in range(1, k+1):
             add_feature = False

             for combo in itertools.combinations\
                (remaining_features, 1):
                  X_with_ones = sm.add_constant(\
                X[list(combo) + features_chosen])
                  model = smf.OLS(endog=forratedat\
                ['emp'], exog=X_with_ones).fit()
                  this_AIC = model.aic

                  if this_AIC < best_AIC:
                      add_feature = True
                      best_AIC = this_AIC
                      best_feature = combo[0]

             if add_feature:
                 features_chosen.append(best_feature)
                 remaining_features.remove(best_feature)
```

The resulting model fit is

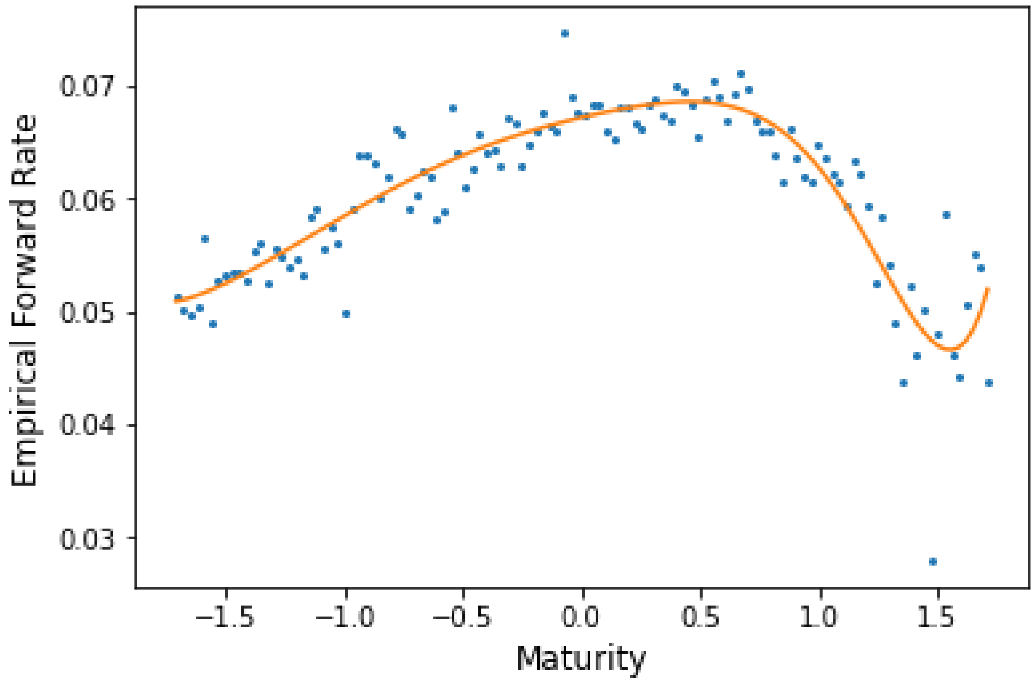
$$Y_i = \beta_0 + \beta_1 T_i + \beta_2 T_i^2 + \beta_4 T_i^4 + \beta_5 T_i^5 + \beta_7 T_i^7 + \beta_8 T_i^8 + \epsilon_i$$

We can see that this is quite similar to the model fit before, although not identical.

```
In [17]: X_with_ones = \
            sm.add_constant(X.iloc[:,[0,1,3,4,6,7]])

         FSWoptmodel = smf.OLS(\
             endog=forratedat['emp'],
             exog=X_with_ones).fit()

         fig, axs = plt.subplots(1, 1, figsize=[6,4])
         axs.plot(forratedat['maturity'],forratedat['emp'],
             marker='.', ms=4, linestyle="none")
         axs.set_xlabel("Maturity", size=12)
         axs.set_ylabel("Empirical Forward Rate", size=12)
         axs.plot(forratedat['maturity'],
             FSWoptmodel.fittedvalues,ms=10)
         plt.show()
```


0.07
0.06
0.05
0.04
0.03
Empirical Forward Rate
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
Maturity

Cross Validation

We've discussed the use of AIC for the purpose of **model selection**, i.e.~the process of determining which covariates should be included in the model.

An important alternative is **leave-one-out cross validation**.

In this approach, one imagines refitting the model n times, each time excluding one of the n observations.

At iteration i, observation i is excluded. The fitted model is then used to predict the response for this observation. Call this $\widehat{y}_{(-i)}$.

Finally, we calculate the quantity

$$\text{PRESS} = \sum_{i=1}^{n} \left(y_i - \widehat{y}_{(-i)}\right)^2$$

where **PRESS** stands for **Prediction Error Sum of Squares**.

The motivation for this is as follows: Once observation i is removed from the fit, we have removed the **influence** that this observation has on the parameter estimates.

Now, we can think of observation i as being a "new" observation, and we can ask: "How well does the model, containing only these covariates, predict the response for this case?"

The quantity $(y_i - \widehat{y}_{(-i)})^2$ quantifies the amount of error in this prediction.

Exercise: Which is larger: PRESS or RSS?

Fortunately, it is not actually necessary to fit n models in order calculate PRESS.

The remarkable result is as follows:

Let $\mathbf{Y}$ be a vector consisting of the n responses, and let $\widehat{\mathbf{Y}}$ be a vector consisting of the n fitted values (from the full model).

If it is the case that $\widehat{\mathbf{Y}} = \mathbf{H}\mathbf{Y}$, then

$$y_i - \widehat{y}_{(-i)} = \frac{\widehat{\epsilon}_i}{1 - h_{ii}}$$

where h_{ii} is the i^{th} diagonal element of $\mathbf{H}$.

In the case of linear regression, it is the case that

$$\mathbf{H} = \mathbf{X}\left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T$$

where $\mathbf{X}$ whose columns hold the values of the predictors. See Ruppert and Matteson for the details.

This matrix $\mathbf{H}$ is previously mentioned **hat matrix**.

The diagonal elements h_{ii} are called the **leverages**.

The leverages can be found in Python using the following.

```
In [18]: import statsmodels.stats.outliers_influence \
            as outliers_influence

         levs = outliers_influence.OLSInfluence(fullmod)\
            .hat_matrix_diag
```

and then PRESS can be calculated as

```
In [19]: PRESS = ((fullmod.resid / (1 - levs))**2).sum()
         print(PRESS)
```

```
0.0019740282761601747
```

The following code performs an exhaustive search for the optimal model when using PRESS.

```
In [20]: X = forratedat.drop('emp', axis=1)
         df2 = {}

         for k in tnrange(1, len(X.columns)+1):
             for combo in \
               itertools.combinations(X.columns, k):
                 X_with_ones = \
                    sm.add_constant(X[list(combo)])
                 model = smf.OLS(endog=forratedat['emp'],
                    exog=X_with_ones).fit()
                 levs = outliers_influence.\
                    OLSInfluence(model).hat_matrix_diag
                 df2[str(combo)] = \
                    ((model.resid / (1 - levs))**2).sum()

         df2 = pd.Series(df2)
```

```
HBox(children=(IntProgress(value=0, max=9), HTML(value='')))
```

The resulting model fit is

$$Y_i = \beta_0 + \beta_1 T_i + \beta_3 T_i^3 + \beta_4 T_i^4 + \beta_6 T_i^6 + \beta_7 T_i^7 + \epsilon_i$$

This is, again, a sightly different model, but it yields almost identical predictions for the response.

AIC versus PRESS

AIC and PRESS will often give similar, if not the same, result for the model choice.

There are a couple main reasons to prefer PRESS to AIC: First, the quantity being estimated is by PRESS is a very natural measure of prediction performance. Second, the theory behind AIC is based on the assumption that the disributional assumption for the response is correct.

A main reason to choose AIC over PRESS is that AIC is more stable. The estimate that PRESS provides of the expected prediction error is subject to large variance. Hence, PRESS may not perform well for small to moderate sample sizes.

Although PRESS is simple to calculate in the case of linear regression, this will not always be the case. In general, AIC will be easier to calculate than PRESS.

As is often the case, our past experience will often guide our choice of criterion.

Robust Regression

In the least squares approach to regression, we seek the $\boldsymbol{\beta}$ that minimizes the residual sum of squares

$$\sum_{i=1}^{n} \left(y_i - \mathbf{x}_i^T \boldsymbol{\beta}\right)^2.$$

Of course, this is equivalent to minimizing

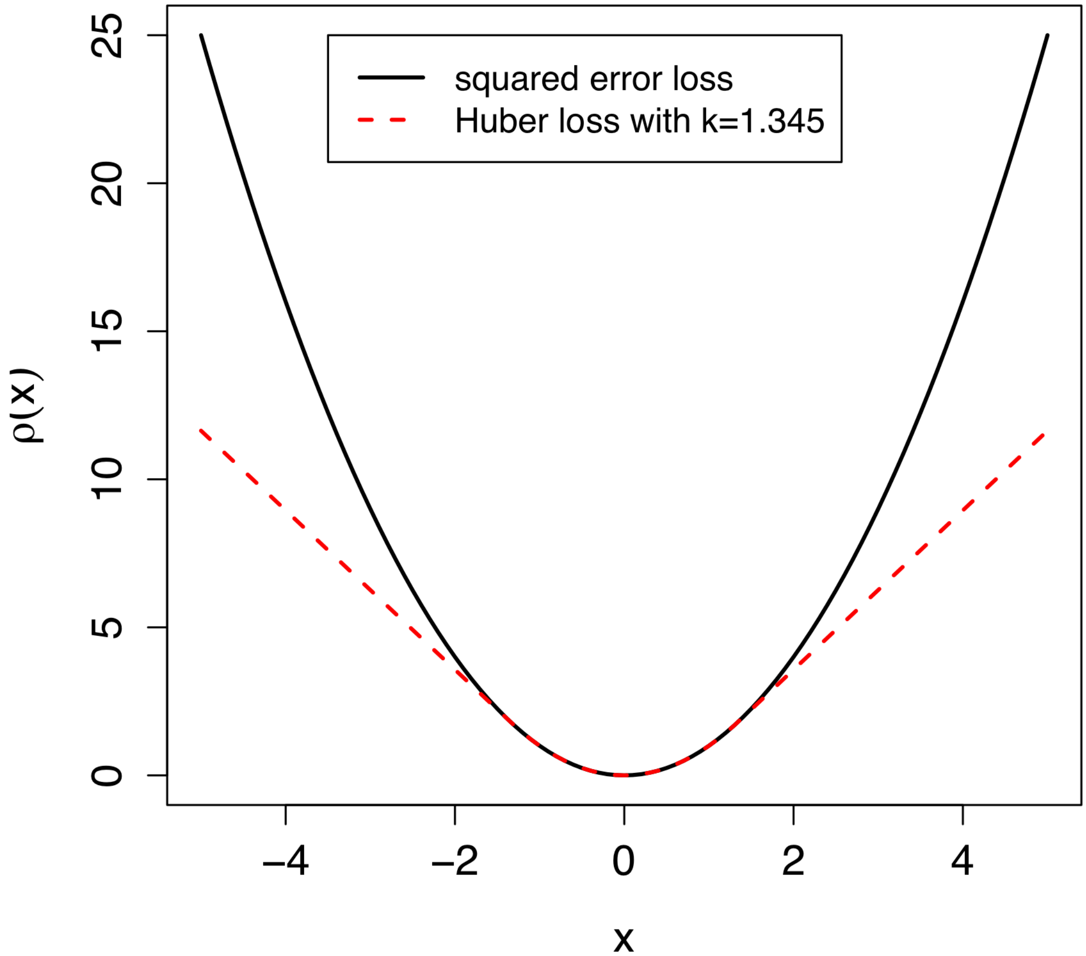
$$\frac{1}{\sigma^2} \sum_{i=1}^{n} \left(y_i - \mathbf{x}_i^T \boldsymbol{\beta}\right)^2 = \sum_{i=1}^{n} \left(\frac{y_i - \mathbf{x}_i^T \boldsymbol{\beta}}{\sigma}\right)^2 = \sum_{i=1}^{n} \rho\left(\frac{y_i - \mathbf{x}_i^T \boldsymbol{\beta}}{\sigma}\right)$$

where $\rho(x) = x^2$.

The general concern with $\rho(x) = x^2$ is that it may place too much weight on extreme observations: Think about how quickly the function x^2 increases as $|x|$ increases.

The choice $\rho(x) = x^2$ may be optimal when the errors are normal, but it is not very **robust** to deviations from this assumption.

If we generated errors from the t distribution with four degrees of freedom, for instance, we are likely to see a few extreme errors. These observations would have a large influence on the fit of the model, if we proceeded with $\rho(x) = x^2$.



Comparison of Loss Functions.

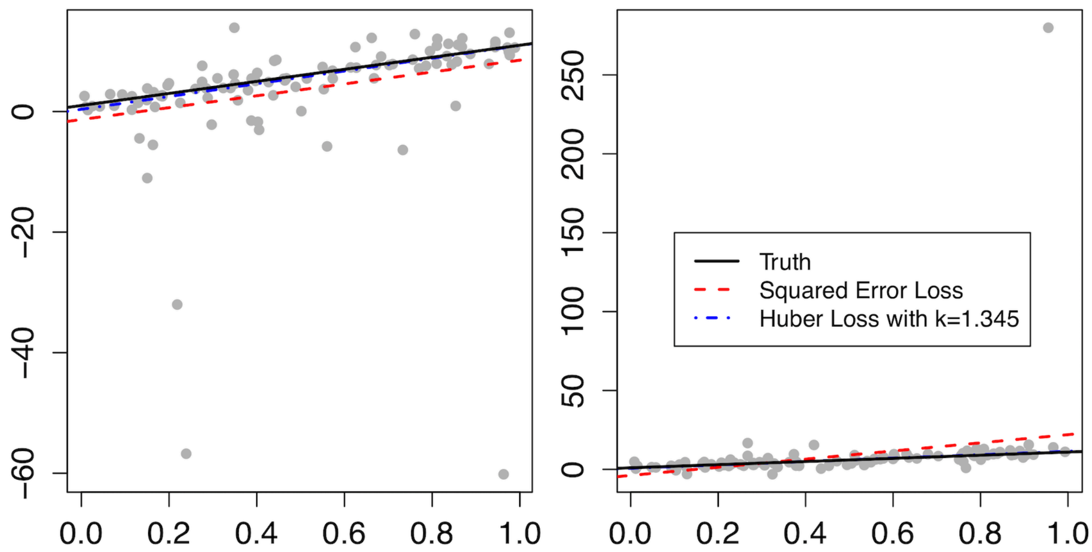
There are many possible choices for ρ, but here we will focus on a popular one, the **Huber loss function**, which is defined as

$$\rho_{\text{Hub}}(x) = \begin{cases} x^2 & \text{if } |x| < k \\ k(2|x| - k) & \text{otherwise} \end{cases}$$

where $k > 0$ is set by the user. For technical reasons, the default choice is $k = 1.345$.

The figure above compares squared error loss with the Huber loss function.

Exercise: What is the effect of varying k in the Huber loss function?



The effect of outliers on the fit.

Example. Here the truth is that $\beta_0 = 1$ and $\beta_1 = 10$. The sample size is $n = 100$. The error has the t distribution with a single degree of freedom, so there is great potential for extreme observations.

The figure above shows two simulated data sets under these conditions, each with the true line (solid), the regression line estimated using least squares (dashed), and the regression line estimated using the Huber loss (dashed-dot).

It is clear that in both instances that the Huber loss function effectively downweights the extreme observations, and, as a result, results in a closer approximation to the truth.

These examples are, admittedly, chosen to be extreme to illustrate the point. Nevertheless, robust regression is useful in situations where it is suspected that the error distribution is not normal, and there is concern over extreme observations.

Robust regression is implemented in Python using `RLM` from `statsmodels`.

The argument `M` sets the loss function. (This falls into the general class of problems called **M-estimation**.) The default setting is `M=sm.robust.norms.HuberT`, which uses the Huber loss function with $k = 1.345$.

```
In [21]: ff3modPNCrob = sm.RLM.from_formula\
             (formula="PNCexret ~ MktRF + SMB + HML",
                data=ffhold).fit()
```

Exercise: Compare the results of this fit with least squares. Have the coefficients changed much?

Part 8: Regression Topics

Heteroskedasticity

To this point, an assumption of our model has been that all of the model errors have the same variance; this condition is called **homoskedasticity**.

The opposite case, the presence of **heteroskedasticity** is a serious concern. The validity of the confidence intervals, hypothesis tests, and so forth rests on this assumption.

The plot of residuals versus fitted values is a useful tool for diagnosing heteroskedasticity.

Exercise: What shapes would one expect to see in the plot of residuals versus fitted values when heteroskedasticity is present?

There are two major strategies for dealing with heteroskedasticity.

First, the response variable can be **transformed**, meaning you would use $g(y)$ instead of y as the response variable. The two most popular choices for $g()$ are the logarithm, and the square root.

There is an accompanying Jupyter notebook that shows a simple example in which the log transformation helps with heteroskedasticity.

The other major strategy for dealing with heteroskedasticity is to use **weighted least squares**. Briefly stated, instead of minimizing the residual sum of squares, one minimizes

$$\text{WRSS}(\beta) = \sum_{i=1}^{n} w_i (y_i - \widehat{y}_i)^2$$

where the weights are chosen to deemphasize those observations for which the variance is larger.

In fact, the optimal choice is $w_i = 1/\sigma_i^2$, where σ_i^2 is the variance for the i^{th} model error.

One would not typically know the σ_i^2, but it is sometimes the case that you are willing to assume that $\sigma_i^2 = \sigma^2/m_i$. Then, the weight can be chosen as $w_i = m_i$. Or, the σ_i^2 could be estimated from the residuals.

Exercise: Explain why it may be the case that you would be willing to assume that $\sigma_i^2 = \sigma^2/m_i$.

Influential Observations

One source of concern is the possibility that one (or a few) observations are unduly **influential** on the resulting model fit. Of course, we expect that each data point should have some effect on the resulting fit. But no one observation should be overly influential; we'd prefer that each contribute a roughly equal amount to determining the final estimates.

A natural way of quantifying the influence of observation j is to consider the following question:

> By how much do the fitted values $\widehat{Y}_1, \widehat{Y}_2, \dots, \widehat{Y}_n$ change if observation j is excluded from the training set, i.e., it is not used in fitting the model?

This question is answered by **Cook's Distance**.

First, let $\widehat{Y}_{i(-j)}$ denote the fitted value for observation i when using the model that **excludes** observation j from the fitting. Cook's Distance for observation j is then

$$D_j = \sum_{i=1}^{n} \left(\widehat{Y}_i - \widehat{Y}_{i(-j)}\right)^2 \Big/ (p+1)\widehat{\sigma}^2$$

In Python, we can find Cook's Distance using

```
holdlm.get_influence().cooks_distance[0]
```

where `holdlm` is the output from the `statsmodels` function used previously.

Fortunately, calculating Cook's Distance for an observation does not actually require refitting the model with that observation removed. It is possible to show that (in the case of simple linear regression),

$$D_j = \frac{\widehat{\epsilon}_j^2}{(p+1)\widehat{\sigma}^2}\left[\frac{h_{jj}}{(1-h_{jj})^2}\right]$$

where h_{jj} is the previously-defined i^{th} leverage.

Exercise: Consider the given expression for D_j. What does it say about what it takes to make an observation influential?

Handling Influential Observations

Influential observations are almost always an **outlier** in some sense; this is what causes them to have more influence than other observations.

First check to see if the influential point is not the result of a mistake. If so, the error should be corrected or, if that is not possible, the observation removed. But, **it is a mistake to arbitrarily remove an observation from a training set.** Instead, consider the following:

1. One should verify that there truly is a linear relationship between the response and predictor. A nonlinear model may be able fit more flexibly to the feature(s) that led to this outlier.
2. It may be the case that this observation is sufficiently different from the others that it would make sense to redefine the population of interest in such a way that this outlying (and other) cases are handled by a separate model. In other words, it may not be realistic to expect a single model to cover such a wide range of possibilities.
3. A transformation of the predictor and/or response can "pull in" an outlier so that it's not so extreme.
4. An alternative approach to estimating the β_i may be preferable; for example, weighted least squares or robust regression.

Making Predictions

One of the fundamental motivations for using regression models is to make predictions for **future** observations of the response. In other words, consider a situation where you know the value of the predictor variable x, and seek to predict the response.

In the context of our simple linear regression model, if our value for the predictor is x^*, then we would start by finding

$$\widehat{y}^* = \widehat{\beta}_0 + \widehat{\beta}_1 x^*$$

We have plugged in this new value for x into the line that has been fit. This is our best estimator for y^*, the true response for this object.

One should not be satisfied with only this, however: We should quantify the amount of error in this prediction.

In the case of simple linear regression, the **variance in the error in the prediction** is

$$V(\hat{y}^* - y^*) = \sigma^2 + \sigma^2\left[\frac{1}{n} + \frac{(x^* - \overline{x})^2}{(n-1)s_x^2}\right]$$

Exercise: Explain how the above decomposes the error in the prediction into two key sources.

Hence the **standard error in the prediction** is

$$\mathrm{SE}(\hat{y}^*) = \sqrt{\sigma^2 + \sigma^2\left[\frac{1}{n} + \frac{(x^* - \overline{x})^2}{(n-1)s_x^2}\right]}$$

Of course, in practice σ^2 will be replaced with $\hat{\sigma}^2$.

If the errors are assumed to be normally distributed, then a $100(1-\alpha)\%$ **prediction interval for the future response** can be written as

$$\widehat{y}^* \pm t_{\alpha/2,n-2}\,\mathrm{SE}(\widehat{y}^*)$$

Figure 1 shows a simple example of how the prediction interval will appear, as x^* ranges over the extent of the predictor space.

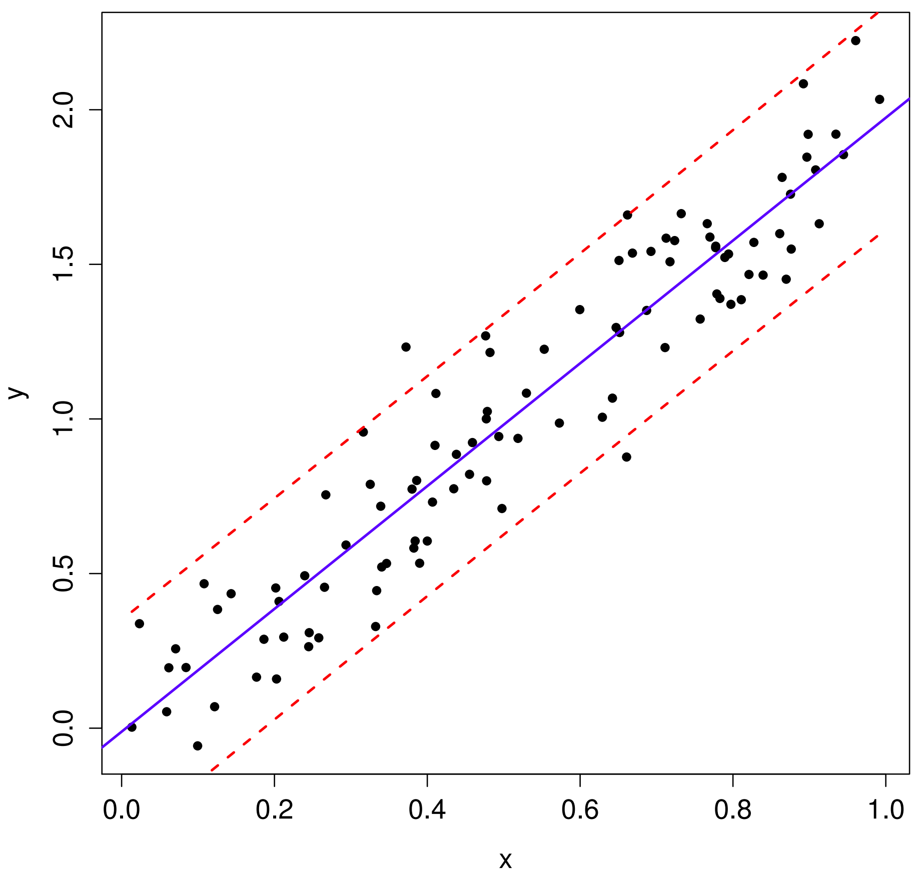


Figure 1: An example showing a regression line (solid line), along with the prediction bands (dashed lines).

Exercise: Discuss the role of the normality assumption in the validity of this interval. If the sample size is large, do you believe that the normality assumption can be relaxed?

Discrete Variables as Predictors

One often encounters variables which are discrete quantities, but still are on the ratio scale.

A primary example is that of a **count variable**, i.e., a variable which counts the number of times somthing occurs. For example, it is standard for credit-granting entities to build models where the response variable is

– the likelihood someone will payoff a loan

and the predictors utilized include

– number of major derogatory reports on credit record
– number of dependents
– number of major credit cards held
– number of active credit accounts

Such predictors **can** be included as predictors just as continuous quantites are; the interpretation remains that same: The expected response increases at a constant rate as the variable increases.

Categorical Predictors

We just discussed using a discrete variable on the ratio scale as a predictor. There are some discrete variables for which this is clearly not a good idea, however. These are cases where the numbers are merely labels for the levels of a **categorical variable**.

Consider a variable which indicates highest degree obtained:

Code	Value
0	Did not finish high school
1	Finished high school
2	Some college
3	Graduated college

Exercise: This variable is on the ordinal scale. What would happen if we mistakenly treated this as a variable on the ratio scale when including it in the regression model?

Here is the tricky part: A categorical variable with k levels adds $k-1$ terms into the regression function.

To see why this is the case, imagine that the model under consideration includes a predictor giving the number of credit cards, along with this education level predictor. The $k-1$ new terms correspond to "shifts" in the regression function relative to the "baseline" category. In particular, let X_1 equal the number of credit cards. Then,

when education level is...	**the regression function is...**
0	$\beta_0 + \beta_1 X_1$
1	$\beta_0 + \beta_1 X_1 + \beta_2$
2	$\beta_0 + \beta_1 X_1 + \beta_3$
3	$\beta_0 + \beta_1 X_1 + \beta_4$

We can write this regression function more compactly as

$$\beta_0 + \beta_1 X_1 + \beta_2 \mathbf{1}_{\{\text{edu}\,=\,1\}} + \beta_3 \mathbf{1}_{\{\text{edu}\,=\,2\}} + \beta_4 \mathbf{1}_{\{\text{edu}\,=\,3\}}$$

where $\mathbf{1}_A$ is the **indicator variable** for the event A.

Exercise: Why not include **every** discrete predictor as being categorical?

An appropriately-defined categorical variable will be handled correctly by software such as Python and R.

```
Coefficients:
                   Estimate Std. Error t value Pr(>|t|)
(Intercept)         35.2647     0.7990  44.137  < 2e-16 ***
NumCards            -2.1467     0.1915 -11.209  < 2e-16 ***
(Education)1         5.4004     0.8424   6.411 5.58e-09 ***
(Education)2         7.8708     0.9308   8.456 3.26e-13 ***
(Education)3         9.5610     0.8292  11.530  < 2e-16 ***
```

Exercise: Consider the sample output above. What is the expected response for a case when education level is "Did not finish high school?" What about when education level is "Some college?"

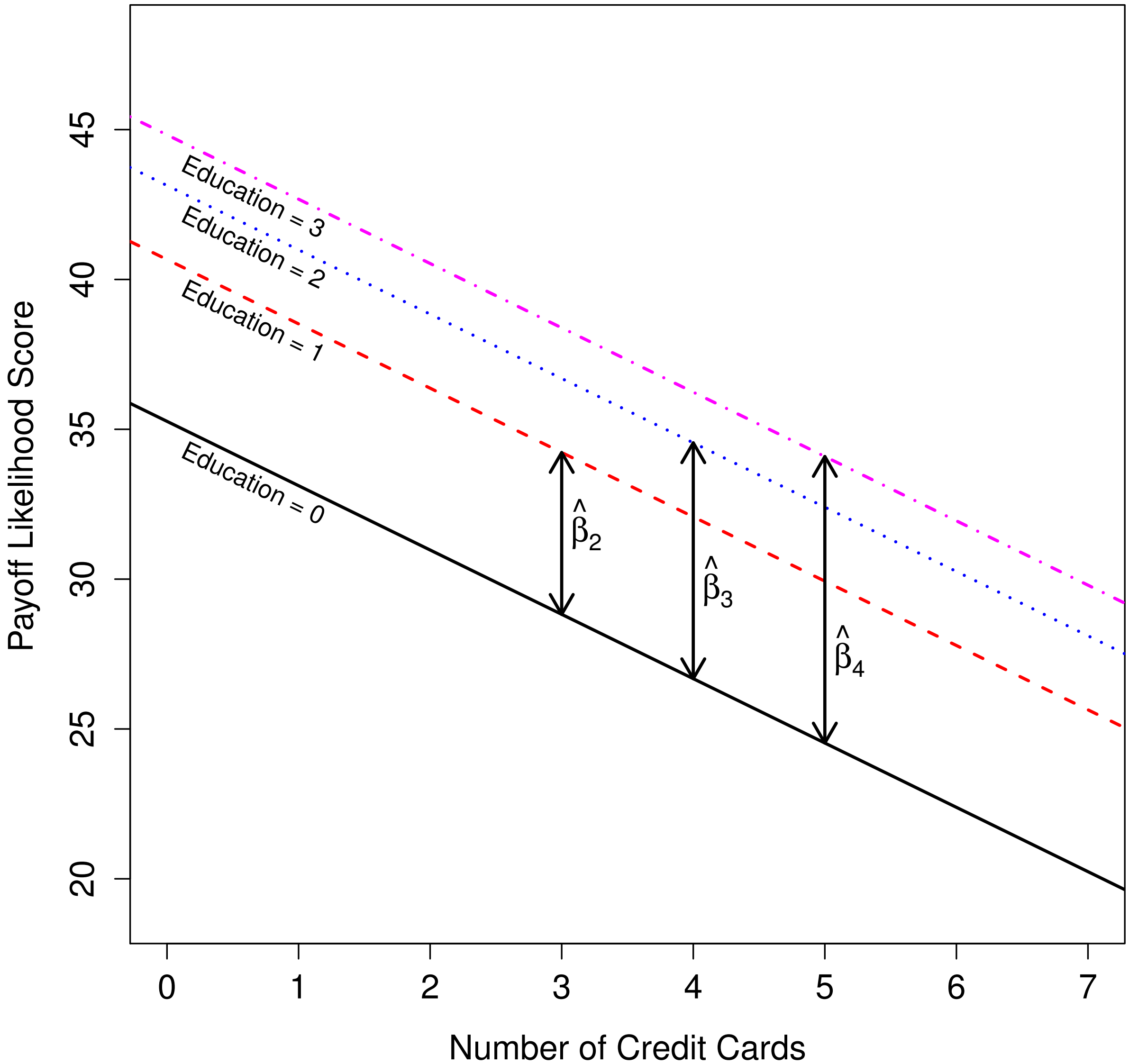


Figure 2: The fit model in this case. The four lines correspond to the four levels of the Education predictor. The slope of each line is $\widehat{\beta}_1 = -2.15$.

Categorical Variables in the Design Matrix

Remember that a categorical variable with k levels adds $k-1$ β parameters to the model. Hence, it adds $k-1$ **columns** to the design matrix $\mathbf{X}$. In order to create the "shifting" effect, these columns will be filled with zeros and ones.

Consider the following example. The response is the assessment of the likelihood of default. There is a predictor which counts the number of open credit cards. There is another predictor "education level," defined as above. The first few rows of the data set appear as follows:

Observation	Payoff Likelihood	Number of Cards	Education Level
1	40	4	2
2	41	2	3
3	44	1	2
4	28	5	0
5	36	4	1
6	32	2	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$

Assume that the analysis proceeds with the predictor "Number of Cards" untransformed.

The first six rows of the design matrix would be

$$\mathbf{X} = \begin{bmatrix} 1 & 4 & 0 & 1 & 0 \\ 1 & 2 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 5 & 0 & 0 & 0 \\ 1 & 4 & 1 & 0 & 0 \\ 1 & 2 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$

Exercise: Explain the form of the design matrix above. Consider the matrix product $\mathbf{X}\boldsymbol{\beta}$.

More than One Categorical Variable

Exercise: Suppose that a model contains only two predictors, and they are both categorical variables. The first (X_1) has three levels, and the second (X_2) has four levels. How many total β parameters are in the model, and how is each interpreted?

Interactions

Interaction terms are added to the model by taking the product of other predictors.

In the absence of any interactions, we say that the model is **additive** in the predictors.

Exercise: Think carefully about what is being assumed when an additive model is used. In particular, what is the effect on the regression function if a covariate is varied?

Consider a model with two predictor variables X_1 and X_2, but with the interaction term X_1X_2 included. So, the regression function is

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1 X_2$$

Then, if X_1 is held fixed, the effect of varying X_2 is characterized by the slope of $\beta_2 + \beta_3 X_1$. In other words, if X_2 is increased by c, then the regression function increases by $(\beta_2 + \beta_3 X_1)c$.

The key enhancement is that this slope now depends on X_1.

Of course, one may wonder: Why does it make sense to let the slope depend on X_1 **in this particular manner**?

The best way to answer this is to consider the interaction X_1X_2 to be a **first order approximation** to this dependence.

It is much like how we often use straight lines to approximate relationships which are not truly straight: These are useful approximations in the right circumstances.

Exercise: What will happen if you include in the model an interaction between a continuous predictor and a categorical variable?